STAT395: Sampling

Richard Arnold et al.

2026-05-09

Contents

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

##
## Attaching package: 'kableExtra'

## The following object is masked from 'package:dplyr':
##
##     group_rows

## Loading required package: grid

## Loading required package: Matrix

##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack

## Loading required package: survival

##
## Attaching package: 'survey'
```

```
## The following object is masked from 'package:graphics':
##
##     dotchart
##
## Attaching package: 'srvyr'
## The following object is masked from 'package:kableExtra':
##
##     group_rows
## The following object is masked from 'package:stats':
##
##     filter
##
## Attaching package: 'sampling'
## The following objects are masked from 'package:survival':
##
##     cluster, strata
##
## Attaching package: 'mice'
## The following object is masked from 'package:stats':
##
##     filter
## The following objects are masked from 'package:base':
##
##     cbind, rbind
```

Chapter 1

Introduction

These are notes for the Sampling section of STAT395/455. The course overlaps to some extent with STAT392/439, but there is a greater emphasis on R coding, computation, summary and display.

1.1 Lecture Plan

Course Learning Objectives

- Describe the steps in the survey process, including the sources of survey error;
- Draw samples from a sample frame under a variety of sample designs, and compute survey weights;
- Carry out, interpret and present statistical inference procedures on survey data, including adjustments for non-response.

Lecture plan (3 lectures + 1 tutorial per week)

Bring a laptop to the Friday (tutorial) lecture.

Week 1 – Sampling and The Survey Process

Chapters 1-4

- Sampling
- The survey process, sources of survey error
- Populations and Frames
- Sampling in finite and infinite populations; Basics of SRSWOR inference; sample weights;
- **Tutorial:** Sampling basics; Population and Frames

Week 2 – Sample Designs

- Simple random sampling, with and without replacement
- Introduction to sample weights
- Probabiiity proportional to size sampling
- Linear Systematic Random Sampling
- Stratified sampling
- One and Two stage cluster sampling
- Sampling in complex designs

- **Tutorial:** using R to draw samples and compute survey weights

Week 3 - Inference with the R survey package

- Using R to specify sample designs
- Inference in simple and complex designs, including regression
- Definition and interpretation of the design effect
- **Tutorial:** using R to specify designs and draw inferences

Week 4 – Non-response and complex designs

- Post-stratification for non-response adjustment
- Jackknife inference
- Imputation for item non-response
- **Tutorial:** psot-stratification and raking adjustments to weights; imputation; jackknife inference

1.2 Recommended reading

There are lots of books on sample surveys.

- Thomas Lumley's book 'Complex Surveys' (Lumley (2010)) is a good introduction to sample survey analysis with R, and accompanies the `survey` package (Lumley (2004)).
- Groves et al. 'Survey Methodology' (groves.etal) is a good general summary of the important issues in survey design.
- Sarndal, Swensson and Wretman is a classic in the field of the theory of sampling (Särndal et al. (1992))
- Likewise the books by Cochran (1977) and Kish (1965)
- Little and Rubin have a great book on missing data (Little and Rubin (2002))
- More recent are the books by Lohr (1999) and Scheaffer et al. (2006)
- And in 1994 StatsNZ published 'A Guide to Good Survey Design' (Zealand (1995))
- Notes from the STAT392 course on Sample Surveys https://homepages.ecs.vuw.ac.nz/~rarnold/STAT392/SampleSurveysBook/_book/index.html
- A crash guide to using iNZight for sample surveys https://homepages.ecs.vuw.ac.nz/~rarnold/STAT392/iNZightManual/_book/index.html (iNZight is free software, using the `survey` package in the back end.)

And also

- CRAN site on R resources for Official Statistics https://cran.r-project.org/web/views/OfficialStatistics.html
- Online book on using R for survey analysis: https://bookdown.org/jimr1603/Intermediate_R_-_R_for_Survey_Analysis/
- This gallery of R graphs https://r-graph-gallery.com/

Some simple lectures on Sample Surveys

- https://r-project.ro/conference2017/presentations/Camelia_Goga_sondages.pdf
- Notes for epidemiologists: https://www.epirhandbook.com/en/new_pages/survey_analysis.html
- Notes on imputation: https://www.r-bloggers.com/2023/01/imputation-in-r-top-3-ways-for-imputing-missing-data/
- An online book about spatial sampling https://dickbrus.github.io/SpatialSamplingwithR/

- From Qualtrics https://www.qualtrics.com/articles/strategy-research/sampling-methods/

Data

- UC Irvine Machine Learning data repository: http://archive.ics.uci.edu/
- Some free microdata sets: https://asdfree.com/
- Kaggle data repository: https://www.kaggle.com/
  - Airline dataset from Kaggle: https://www.kaggle.com/datasets/thedevastator/airlines-traffic-passenger-statistics [distributed under the Creative Commons Licence 4.0: https://creativecommons.org/licenses/by-nc-sa/4.0/]
- Address file from LINZ: https://data.linz.govt.nz/layer/123113-nz-addresses/
- List of NZ schools https://www.educationcounts.govt.nz/directories/list-of-nz-schools
- Data on Population, GDP and Military Expenditure from 'Our World in Data' https://ourworldindata.org/
- And this list of survey data sources with suggestions for analysis https://asdfree.com/

1.3 R Packages

- `survey` package (Lumley (2004)) with this handy summary of command uses https://stats.oarc.ucla.edu/r/seminars/survey-data-analysis-with-r/
- `srvyr` a dplyr wrapper for the `survey` package and `jtools` which also provides some extensions
- `mice` package for imputation

Chapter 2

Sampling

In statistics the data we analyse is often a subset, i.e. a **sample** of members of a **population**.

> For example we might have a sample of households whose members respond to a questionnaire, or a sample of plants from a field.

In order to select from a population we first enumerate (list) the population in some way and then make a random selection of our subset members from that list. The method by which we do so is called the **sample design**.

Our focus in statistics is often the **estimation** of population characteristics (known as **population parameters**) on the basis of sample characteristics (which are called **sample statistics**). We are always aware that the sample we drew from the population is not the only sample we could have drawn, and therefore our conclusions are subject to uncertainty, which we call **sampling error**.

This uncertainty associated with sampling is the reason we have confidence intervals on our estimates, and why we can neither **prove** nor **disprove** hypotheses.

(One exception is when we do a **census** where we select and get fully accurate data from every member of the population. Then there's no uncertainty left at all, and our sampling errors are zero.)

When generating **simulated** datasets we also generate samples. Simulation is often done as a means of testing out statistical ideas and methods, or (in the case of the bootstrap method) when conventional ways of estimating standard errors (and therefore confidence intervals) don't work.

Before we get into detail of estimation and inference, let's spend some time thinking about how sampling works.

2.1 Random numbers

In both settings (sampling and simulation) we need to start with a method of generating random numbers. We'll start by assuming we have some means of generating a random sequence of numbers uniformly distributed in the interval [0,1]:

$$U_i \sim \text{Uniform}(0,1) \qquad \text{for } i = 1, 2, 3, ...$$

Here 'uniformly distributed' means that all values in the range [0,1] are equally likely to occur.

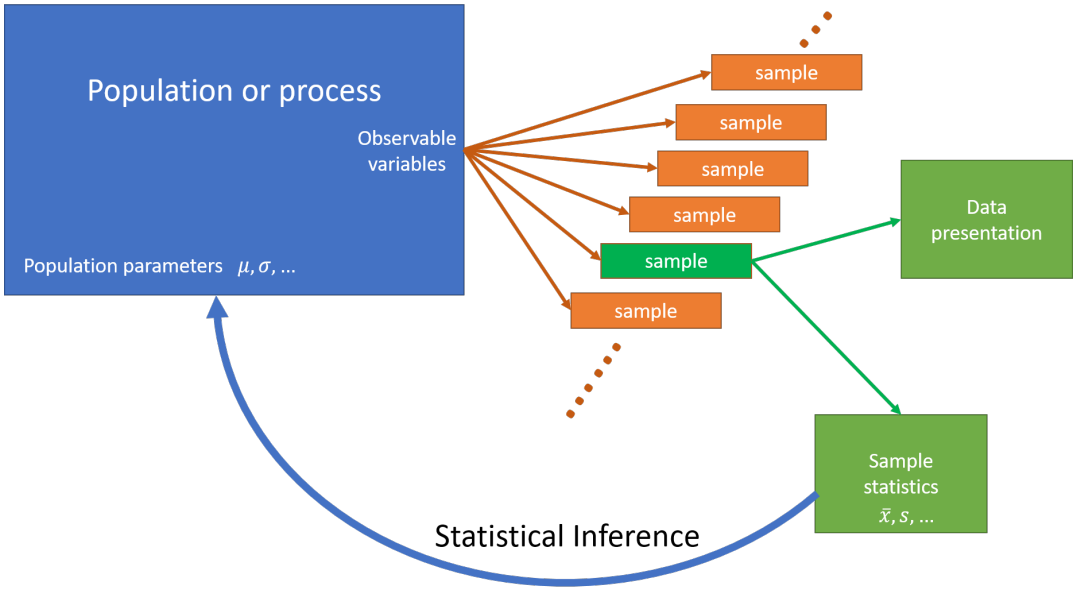


Figure 2.1: Statistical Inference

In R this is done using the `runif()` function. For example to generate 5 random draws from Uniform(0,1):

```
runif(5)
```

```
## [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673
```

Another draw will a different set of random numbers:

```
runif(5)
```

```
## [1] 0.0455565 0.5281055 0.8924190 0.5514350 0.4566147
```

If we draw a histogram of 100 draws it looks a bit lumpy:

```
hist(runif(100), xlab="x", ylab="Frequency")
```

But a histogram of 100,000 draws is very uniform:

```
hist(runif(100000), xlab="x", ylab="Frequency")
```

Although the numbers appear to be random they are in fact generated using formulae, so if one knows the formula the sequence of random numbers generated by `runif()` is completely predictable. However the formula is complex enough that there are no easily discernible patterns in sequences of many millions of draws. These numbers are **psuedo-random**, but they are 'random enough' for our purposes.

Underlying the random number generator is a **seed** value, which initiates the sequence. If we set the seed to a particular value, the sequence of random numbers that will be generated after doing so will always be the same.

```
set.seed(111)
runif(5)
```

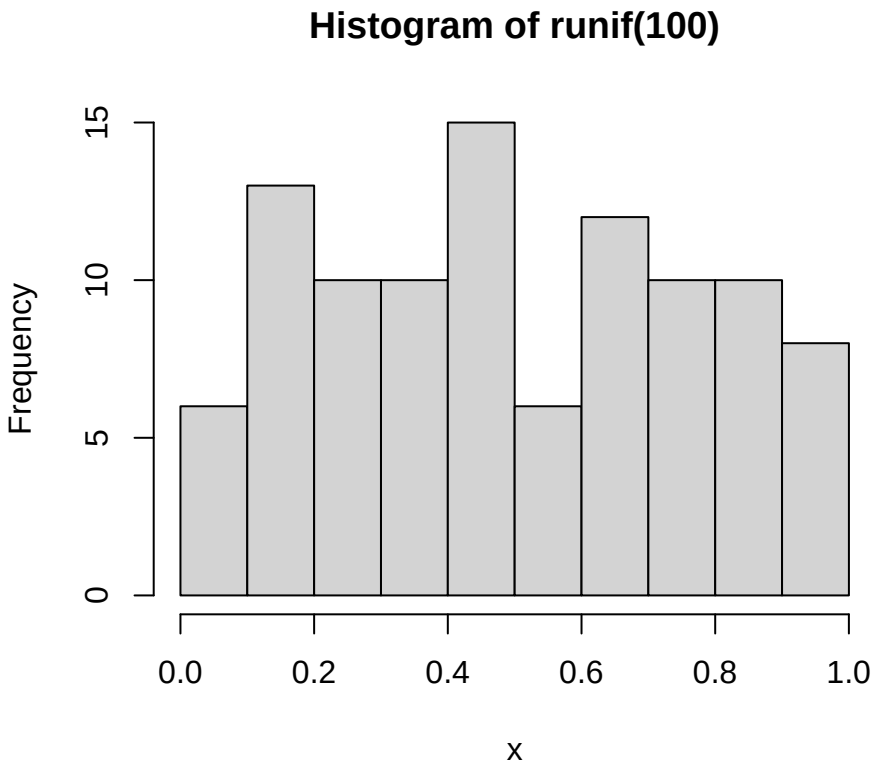


Figure 2.2: 100 draws from Uniform(0,1)

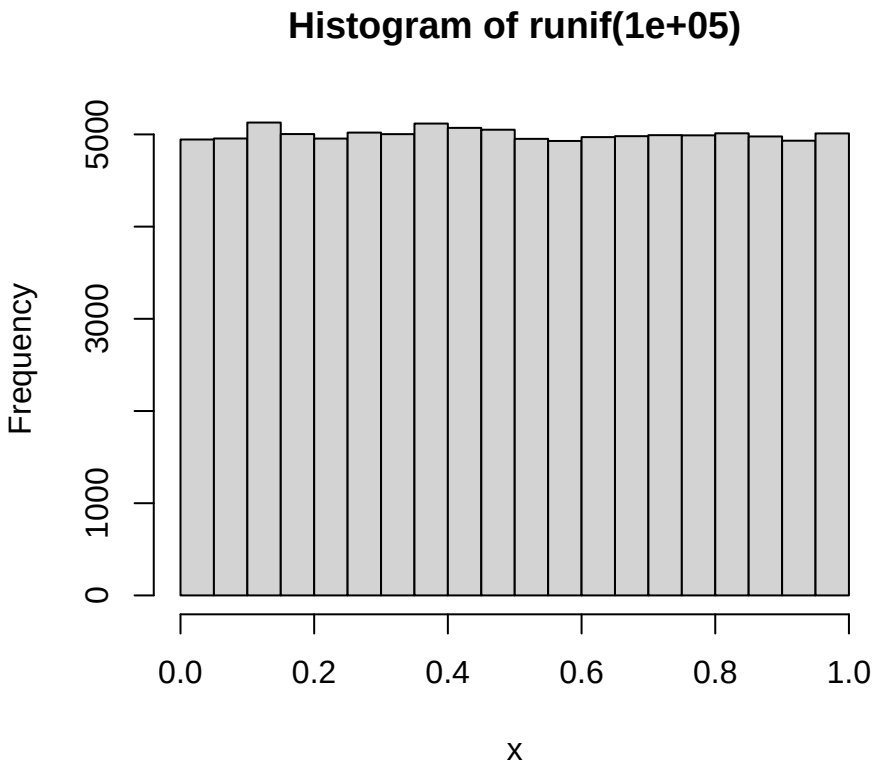


Figure 2.3: 100000 draws from Uniform(0,1)

```
## [1] 0.5929813 0.7264811 0.3704220 0.5149238 0.3776632
```

```
runif(5)
```

```
## [1] 0.41833733 0.01065785 0.53229524 0.43216062 0.09368152
```

```
runif(5)
```

```
## [1] 0.55577991 0.59022849 0.06714114 0.04754785 0.15620252
```

```
set.seed(8)
runif(5)
```

```
## [1] 0.4662952 0.2078233 0.7996580 0.6518713 0.3215092
```

```
set.seed(111)
runif(5)
```

```
## [1] 0.5929813 0.7264811 0.3704220 0.5149238 0.3776632
```

This property can be very useful when testing and comparing algorithms: we can generate an identical sequence of random numbers and check (for example) that two methods of estimation give the same results on two identical randomly generated datasets.

2.2 Sampling from categorical random variables

Draws from a uniform distribution are all we need to simulate/sample from any distribution.

Heads or tails: if the probability of flipping heads is 0.5, we can generate n random numbers between 0 and 1, and then label any values less than 0.5 as Heads and the remainder as Tails.

```
n <- 10
u <- runif(n)
u
```

```
##  [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673 0.0455565 0.5281055
##  [8] 0.8924190 0.5514350 0.4566147
```

```
y <- ifelse(u<=0.5,"Heads","Tails")
y
```

```
##  [1] "Heads" "Tails" "Heads" "Tails" "Tails" "Heads" "Tails" "Tails" "Tails"
## [10] "Heads"
```

```
table(y)
```

```
## y
## Heads Tails
##     4     6
```

```
table(ifelse(runif(10000)<=0.5,"Heads","Tails"))
```

```
##
## Heads Tails
##  5059  4941
```

We can extend this idea to any set of categories.

Red, Green or Blue: Say we have a six sided die: three sides are red, two are green and one is blue. The probabilities of rolling the three colours are therefore $\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{6}$ respectively. These probabilities add to one.

The random variable Y is the colour that the die shows when we roll it.

```
pp <- c(Red=1/2, Green=1/3, Blue=1/6)
sum(pp)
```

```
## [1] 1
```

In general we might have m categories, where the probability of that Y category k is p_k: $\Pr(Y = k) = p_k$. and

$$\sum_{k=1}^{m} p_k = 1$$

We then compute the **cumulative probabilities** c_k, the partial sums of these probabilities:

$$c_k = \sum_{j=1}^{k} p_j$$

The final cumulative probability c_m is always 1 by definition.

Implicitly we also have a value $c_0 = 0$, which is the the empty sum and always equal to zero.

These cumulative probabilities c_k are 'the probability that random variable Y has a category label less than or equal to k:

$$\Pr(Y \leq k) = c_k = \sum_{j=1}^{k} p_j$$

Red, Green or Blue: the cumulative probabilities are

$$\begin{aligned} c_0 &= 0.0000 \\ c_1 &= p_1 = 0.5000 \\ c_2 &= p_1 + p_2 = 0.5000 + 0.3333 = 0.8333 \\ c_3 &= p_1 + p_2 + p_3 = 0.5000 + 0.3333 + 0.1667 = 1.0000 \end{aligned}$$

```
m <- length(pp) # Number of categories
cp <- cumsum(pp)
cp
```

```
##       Red     Green      Blue
## 0.5000000 0.8333333 1.0000000
```

We then take a set of uniform random numbers $u_1, \ldots, u_n$. We then want to find which of the m labels corresponds to each random number u_i.

The rule is that if u_i is less than cumulative probability c_k but greater than c_{k-1}, then we assign the label k.

There are various ways to do this, here's a version using `case_when()` from the `dplyr` package:

```
set.seed(112)
u <- runif(8)
u
```

```
## [1] 0.37667253 0.92201944 0.99187774 0.97723602 0.23629728 0.10706678 0.03915213
## [8] 0.72802690
```

```
y <- case_when(u <= cp[1] ~ "Red",
               u <= cp[2] ~ "Green",
               .default="Blue")   # here .default means "otherwise"
data.frame(round(u,5),y)
```

```
##   round.u..5.     y
## 1     0.37667   Red
## 2     0.92202  Blue
## 3     0.99188  Blue
## 4     0.97724  Blue
## 5     0.23630   Red
## 6     0.10707   Red
## 7     0.03915   Red
## 8     0.72803 Green
```

```
table(y)
```

```
## y
##  Blue Green   Red
##     3     1     4
```

```
table(y)/length(y)
```

```
## y
##  Blue Green   Red
## 0.375 0.125 0.500
```

Note that the categories in the table haven't come out in the Red-Green-Blue order that we want - instead they're in the default alphabetical order of the category labels.

We can fix this by converting `y` to a `factor` object:

```
y <- factor(y,levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
##     4     1     3
```

Here's what is really going on: we if we plot the cumulative probabilities as a staircase – a **cumulative distribution function (CDF)** then we draw u_i from a Uniform(0,1) distribution, and then find where a horizontal line drawn at the level of u_i intersects the CDF: and that is the value of the categorical variable y_i.

The larger the probability p_k the bigger the jump in the staircase, and the larger the probability that category k will be selected.

```
set.seed(112)
n <- 10000
u <- runif(n)
y <- case_when(u <= cp[1] ~ "Red",
               u <= cp[2] ~ "Green",
               .default="Blue")
y <- factor(y, levels=names(pp))
table(y)
```

```
## y
```

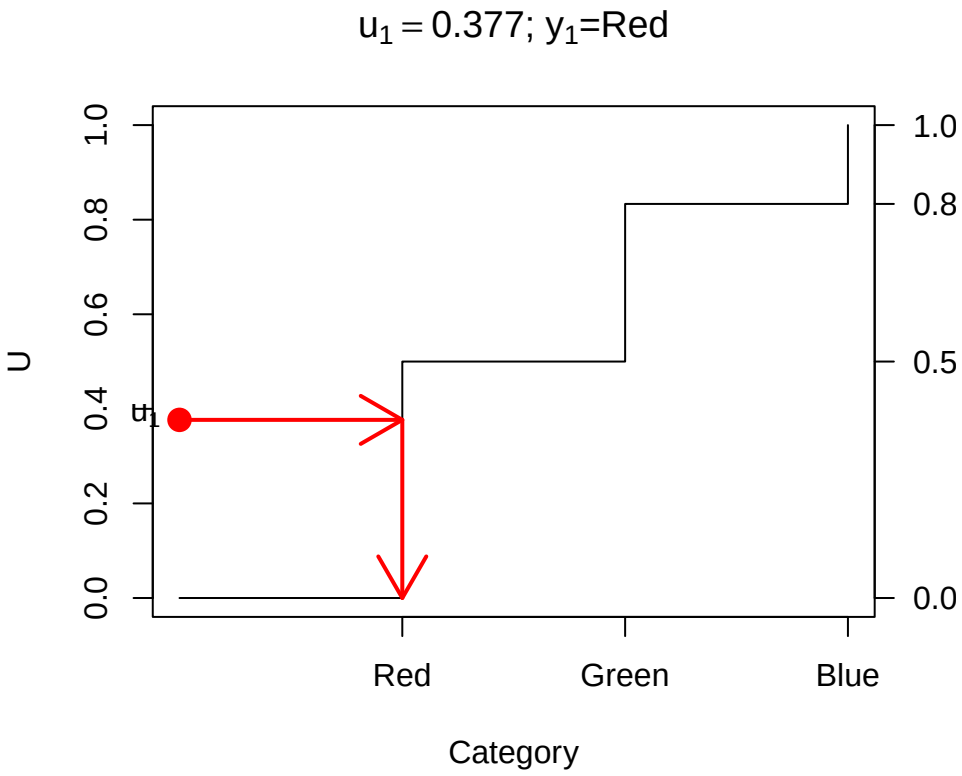


Figure 2.4: Drawing a random category

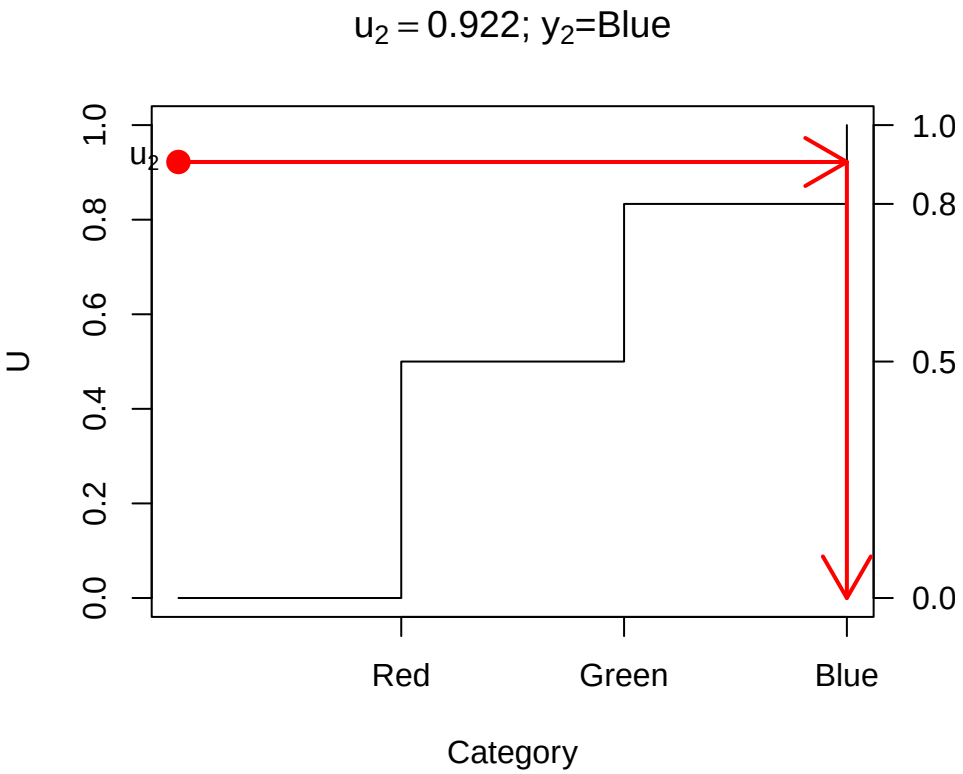


Figure 2.5: Drawing a random category

```
##   Red Green  Blue
##  4976  3336  1688
```

The `sample()` function implements this conveniently: select from the integers 1 to `m`

```
n <- 10000
y <- sample(m, size=n, prob=pp, replace=TRUE)
table(y)
```

```
## y
##    1    2    3
## 5014 3350 1636
```

(need `replace=TRUE` here: we'll talk about this later).

... or state the vector `1:m` explicitly

```
y <- sample(1:m, size=n, prob=pp, replace=TRUE)
table(y)
```

```
## y
##    1    2    3
## 5010 3371 1619
```

... or directly from the vector of labels ...

```
y <- sample(c("Red","Green","Blue"), size=n, prob=pp, replace=TRUE)
y <- factor(y,levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
##  4980  3325  1695
```

... get the labels from `pp` ...

```
y <- sample(names(pp), size=n, prob=pp, replace=TRUE)
y <- factor(y,levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
##  4994  3389  1617
```

2.3 Sampling from discrete numerical random variables

The ideas above carry straight over to discrete numerical random variables.

The Binomial distribution $Y \sim \text{Bin}(m, p)$ is the number of successes in m trials where the probability of success on each trial is the same p, and all the trials are independent.

Y takes values 0 to m, and those probabilities can be calculated by

$$P(Y = k) = \binom{m}{k} p^k (1-p)^{m-k} = \frac{m!}{k!(m-k)!} p^k (1-p)^{m-k}$$

R provides these probabilities in the function `dbinom()`

e.g. $m = 10$ rolls of a six-sided die, and Y is the number of times we get a 1, so $p = 1/6$

```
m <- 10
p <- 1/6
pp <- dbinom(0:m, m, p)
pp
```

```
##  [1] 1.615056e-01 3.230112e-01 2.907100e-01 1.550454e-01 5.426588e-02
##  [6] 1.302381e-02 2.170635e-03 2.480726e-04 1.860544e-05 8.269086e-07
## [11] 1.653817e-08
```

```
barplot(pp, names=0:m,
        xlab="y", ylab="Pr(Y=y)",
        main=bquote("Bin("*.(m)*","*.(sprintf(" %.4f",p))*")"))
```

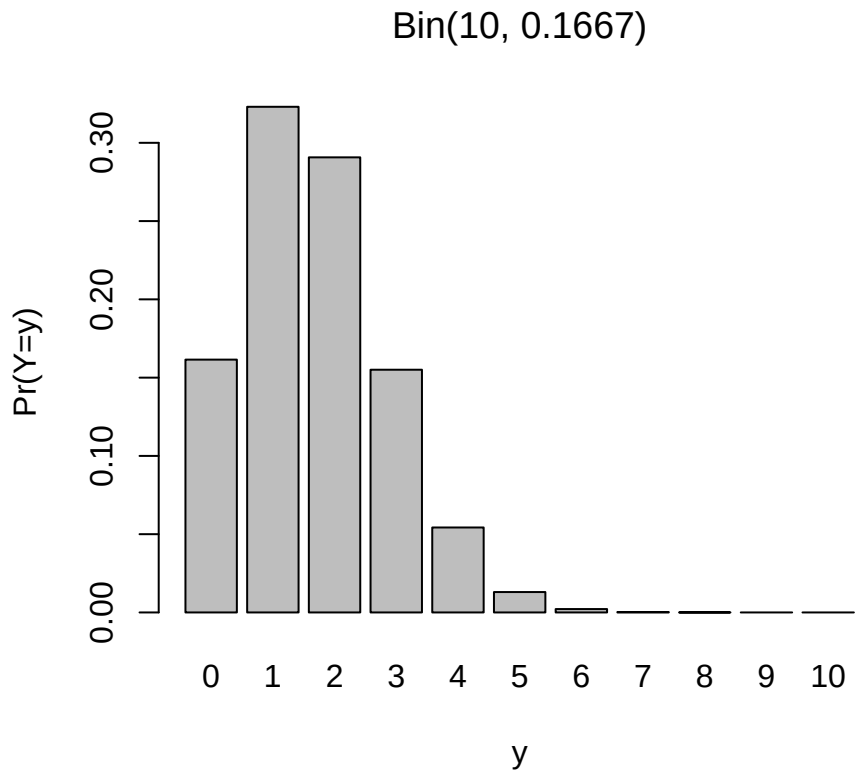


Figure 2.6: Binomial probabilities

The CDF is given by the function `pbinom()`

```
cp <- pbinom(0:m, m, p)
```

```
plot(0:m, cp, type="s",
        xlab="y", ylab="Pr(Y<=y)",
        main=bquote("Bin("*.(m)*","*.(sprintf(" %.4f",p))*")"))
```

Drawing a single value of Y proceeds the same way as for a categorical variable:

So we can draw multiple values of Y from Bin$(10, \frac{1}{6})$ using the `sample()` function:

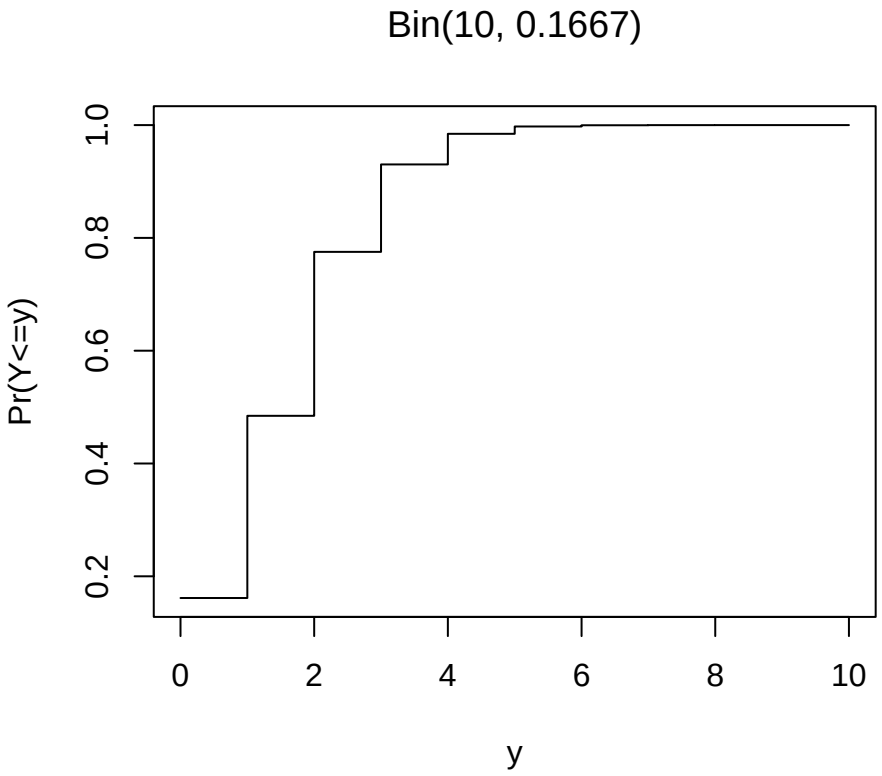


Figure 2.7: Binomial CDF

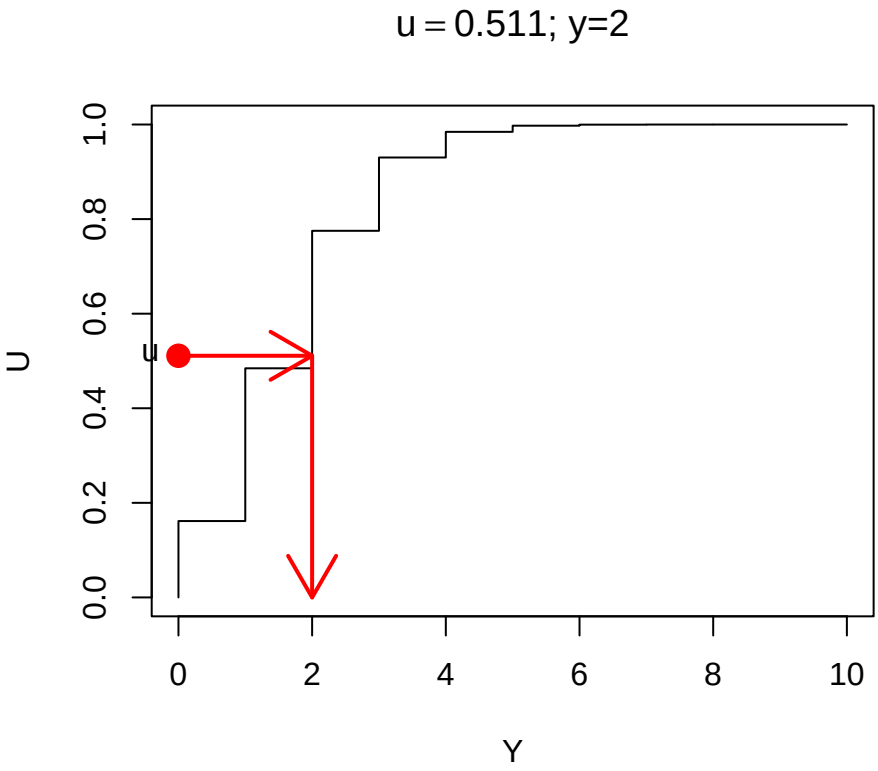


Figure 2.8: Drawing from a Binomial(10,1/6) distribution

```
n <- 10000
pp <- dbinom(0:m, m, p)
y <- sample(0:m, n, prob=pp, replace=TRUE)
barplot(table(y))
```

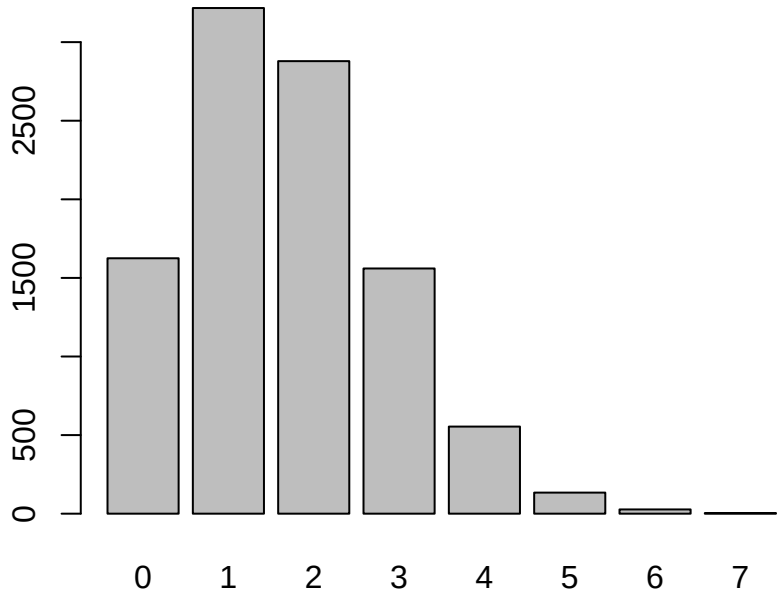


Figure 2.9: 10000 draws from a Binomial(10,1/6)

Of course R provides a specific function for doing the same thing: `rbinom()`

```
n <- 10000
y <- rbinom(n, m, p)
barplot(table(y))
```

But that function is just doing what we've described above using `sample()`.

A particular common example is where we have m objects and want to select one of them at random, with each object having equal probability of being selected: i.e. each has probability $1/m$.

2.4 Sampling from continuous random variables

Continous random variables are simulated using the same principle.

The cumulative distribution function (CDF) of a continuous random variable Y is defined to

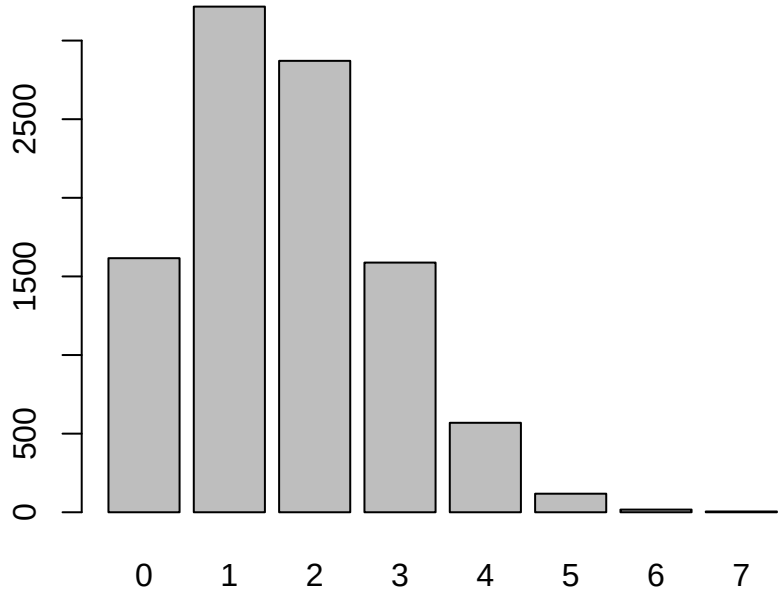


Figure 2.10: 10000 draws from a Binomial(10,1/6)

be

$$F(y) = \Pr(Y \leq y) = \int_{-\infty}^{y} f(y') \, \mathrm{d}y'$$

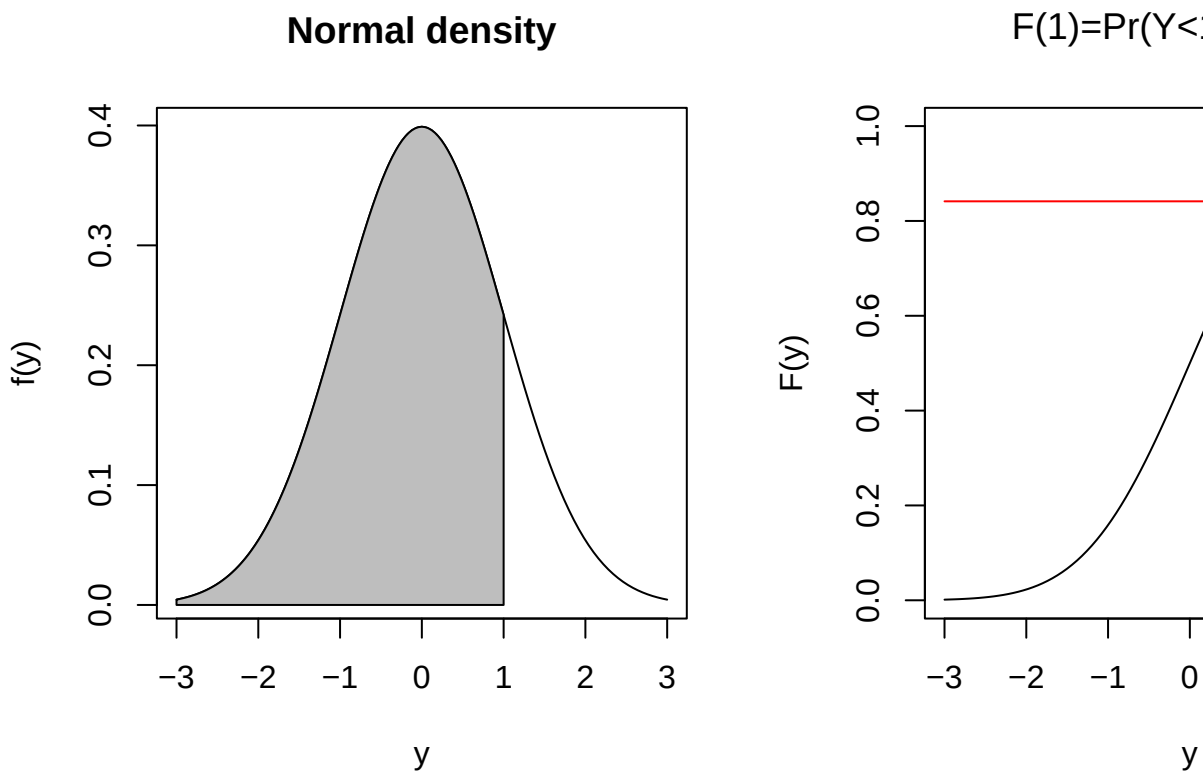


Figure 2.11: Normal distribution density and CDF

We can use $F(y)$ to draw a random value from the distribution of Y by first drawing a random value U from a Uniform(0,1) distribution.

We then solve the equation

$$U = F(y)$$

for y: i.e. find the value of y corresponding to a CDF value of U.

For many standard distributions (including the Normal and Gamma distributions) there is no analytic formula which gives the solution for y given u: but there are numerical approximations which can be used in those settings.

One simple case which does have an analytic solution is the Exponential distribution $Y \sim \mathrm{Exp}(\lambda)$. This is a one parameter distribution where Y is non-negative ($Y \geq 0$), and can be used to model cases such the length of time it takes for an event to occur (e.g. the length of time for a radioactive nucleus to decay). The single parameter λ is the **rate** (e.g. number of radioactive decay events per unit time).

The exponential distribution has probability density function

$$f(y) = \lambda e^{-\lambda y} \qquad \text{where } y \geq 0$$

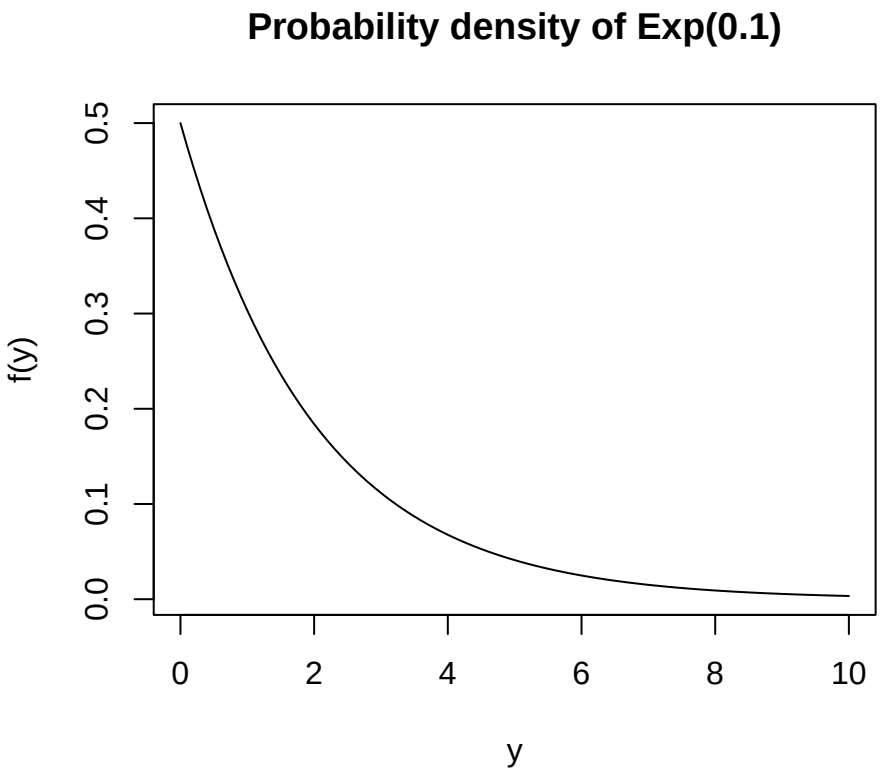


Figure 2.12: Exponential probability distribution

and has mean $E[Y] = 1/\lambda$.

The CDF is the integral of the probability density function

$$\Pr(Y \leq y) = F(y) = \int_0^y f(y)\,\mathrm{d}y$$

which for the Exponential distribution is

$$F(y) = \int_0^y \lambda e^{-\lambda t}\mathrm{d}t = 1 - e^{-\lambda y}$$

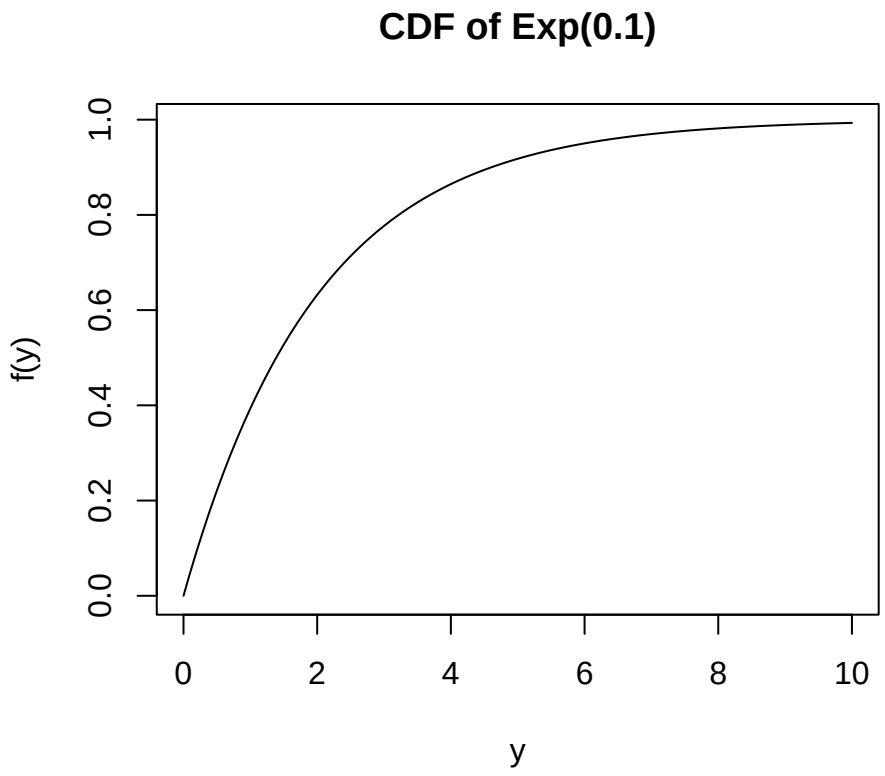


Figure 2.13: Exponential cumulative probability distribution

Solving

$$u = F(y) = 1 - e^{-\lambda y}$$

for y leads to

$$y = -\frac{1}{\lambda}\log(1-u)$$

Using this function we can easily draw directly from an Exponential distribution.

```
n <- 10000
u <- runif(n)
lambda <- 0.5
y <- -(1/lambda)*log(1-u)
mean(y)
```

```
## [1] 1.990456
```

```
hist(y, main="Histogram of draws from Exp(0.5)")
```

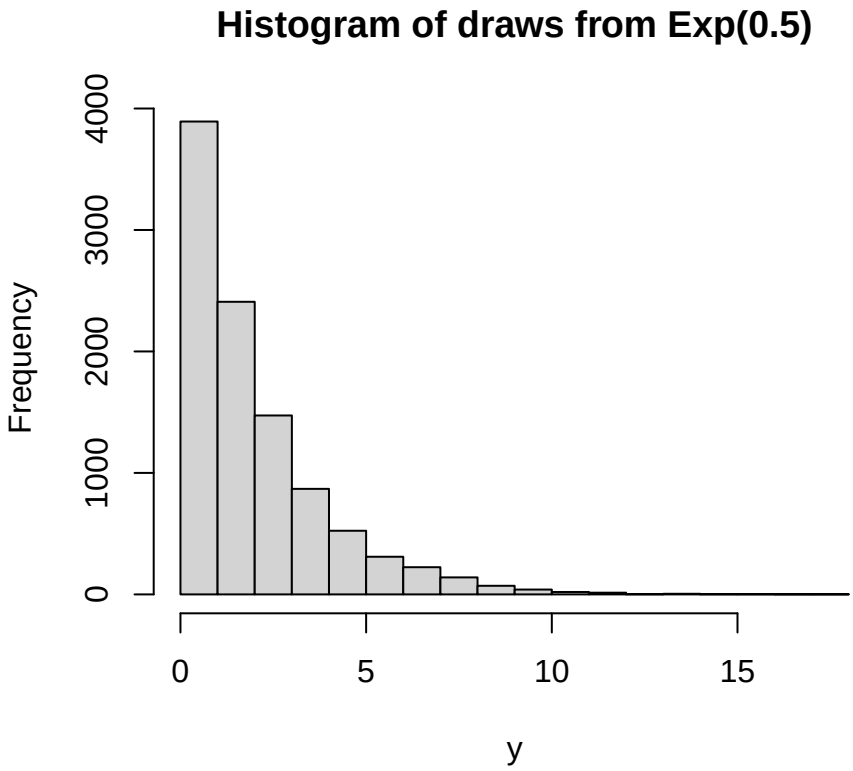


Figure 2.14: 10000 draws from an Exponential distribution

2.5 Sampling from a finite collection

The simulations discussed so far all draw from **infinite** populations: we can set the sample size n to any value. However it often occurs that we have only a finite population - e.g. when drawing a random sample of students from a class, or households from a suburb for interview. Then the population size N is finite.

In the general case we have a set of N **units** (i.e. students or households) in a population of finite size, and we want to draw a sample of n of these units.

We usually start by listing or **enumerating** the units. This list is called a **frame** or a **sample frame**.

Consider the case where we have a small street containing $N = 10$ houses, numbered 1-10. We want to select 5 of these houses with equal probability. There are a number of ways we could do this.

2.5.1 Sampling without replacement

If the houses have equal probability then we could proceed as follows:

- We draw a random number from 1-10, e.g. house number 2, and mark it as selected
- We draw another random number from 1-10, e.g. house number 5, and mark it as selected
- We repeat this process, but if we choose a house that is already selected, we discard that selection;
- The process continues until we have a set of 5 distinct houses in our sample.

At the start we take our frame (i.e. list) of houses, and initially mark them all as unselected. Then at each step where we successfully select a previously unselected house we mark that house as selected.

```
bigN <- 10 # population size
frame <- data.frame(house=1:bigN, selected=FALSE)
n <- 5  # sample size
nselected <- 0
while(nselected < n) {
  i <- sample(bigN, 1)  # sample a single number at random from 1:bigN with equal prob
  if(!frame$selected[i]) {
    frame$selected[i] <- TRUE
    nselected <- nselected + 1
  }
}
frame$house[frame$selected]
```

```
## [1]  1  6  7  8 10
```

It's not guaranteed how many steps this process is going to take, since the number of times we select a house that has already been selected is random. If the sample size n is small and the population size N is large then it will take n steps, or possibly just a few more. If n is close to N and N is large, it could take a very very long time.

A simpler way to solve this is to assign each house a distinct random number between 0 and 1, and then sort the frame in order of the random numbers.

Our sample of n units is just the first n units in the sorted frame. Since the random numbers assigned have no correlation with any property of the units on the frame, then the top n units of of the sorted frame are a simple random sample from the frame.

```
bigN <- 10 # population size
frame <- data.frame(house=1:bigN, selected=FALSE)
n <- 5  # sample size
frame$u <- runif(bigN)
frame <- frame[order(frame$u),]
frame$selected[1:n] <- TRUE
frame
```

```
##    house selected         u
## 9      9     TRUE 0.0812427
## 4      4     TRUE 0.3739315
## 2      2     TRUE 0.4137952
## 6      6     TRUE 0.4341604
## 10    10     TRUE 0.4909159
## 3      3    FALSE 0.5850114
## 8      8    FALSE 0.8708720
## 5      5    FALSE 0.8837804
## 1      1    FALSE 0.9278206
## 7      7    FALSE 0.9475266
```

```
frame <- frame[order(frame$house),]
frame$house[frame$selected]
```

```
## [1]  2  4  6  9 10
```

Of course, the simplest way of all is just to use the `sample()` function

```
bigN <- 10 # population size
n <- 5  # sample size
sample(bigN, n)
```

```
## [1] 10  4  6  3  2
```

It's very often the case that we have a sample frame, i.e. a list of objects with names, and we want to select from this at random.

```
frame <- data.frame(Name=c("Mark","Chloe","Sam","Caitlin","Harry","Nina"),
                    Age=c(20,19,18,19,19,21),
                    Major=c("STAT","STAT","DATA","COMP","MATH","STAT"))
frame
```

```
##      Name Age Major
## 1    Mark  20  STAT
## 2   Chloe  19  STAT
## 3     Sam  18  DATA
## 4 Caitlin  19  COMP
## 5   Harry  19  MATH
## 6    Nina  21  STAT
```

We can select, say, 3 names

```
n <- 3
samplenames <- sample(frame$Name, n)
samplenames
```

```
## [1] "Nina" "Sam"  "Mark"
```

and then subset the frame

```
samplemembers <- frame[frame$Name%in%samplenames,]
samplemembers
```

```
##   Name Age Major
## 1 Mark  20  STAT
## 3  Sam  18  DATA
## 6 Nina  21  STAT
```

If you like dplyr

```
samplemembers <- frame |>
  filter(Name%in%samplenames)
samplemembers
```

```
##   Name Age Major
## 1 Mark  20  STAT
## 2  Sam  18  DATA
## 3 Nina  21  STAT
```

Note we can find out where in the frame particular individuals are located

```
match(c('Mark','Harry'),frame$Name)
```

```
## [1] 1 5
```

If we're sampling without replacement, note (of course) that the sample size can be no bigger than the population size $n \leq N$.

2.5.2 Sampling with replacement

When selecting a sample of 5 houses it makes sense not to allow the possibility of selecting the same house twice. So when we've selected a house we remove it from the list of houses available to select. When using `sample()` this means that we set `replace=FALSE`, meaning after we've selected a member of the population we don't put it back again.

But there are occasions where we select **with replacement** is the sensible thing to do. This means there is a chance where a unit can be selected more than once, and this occurs in situations where we want to assign different probabilities of selection to different units of the population.

Starting with the the case of equal probabilities, we repeatedly draw units from the population under identical circumstances: on every draw each unit is available for selection, and we count how many times each unit is selected.

```
set.seed(111)
bigN <- 10
n <- 5
replicate(n, sample(bigN, 1))
```

```
## [1] 4 3 9 5 3
```

In this sample we've drawn Unit 3 twice.

If n is small and N is large the probability of repeats is small. But if a repeat happens we of course do not make our selected household repeat the questionnaire, instead we count them twice (or as many times as they are selected) in our analysis.

Imagine our sample of 10 houses aren't actually just houses, they're farms, and some of them are very large and some are very small. We want to give the large farms a high chance of selection.

```
set.seed(122)
bigN <- 10
n <- 5
frame <- data.frame(farm=1:10, ncows=c(1000,5000,2000,500,500,
                                       5000,500,5000,30000,100))
frame
```

```
##    farm ncows
## 1     1  1000
## 2     2  5000
## 3     3  2000
## 4     4   500
## 5     5   500
## 6     6  5000
## 7     7   500
## 8     8  5000
## 9     9 30000
```

```
## 10   10   100
```

```
sample(frame$farm, n, prob=frame$ncows, replace=TRUE)
```

```
## [1] 2 2 9 9 9
```

Unsurprisingly, the very large farm number 9 has been selected three times here.

It **is** possible to have unequal probability sampling **without** replacement, estimation of standard errors is complicated in such designs, so we won't be using such designs.

For now, just note that we switch to with replacement designs when we're sampling from sets of units where want to have units with different probabilities of selection.

Chapter 3

Sample Surveys

A sample survey is an exercise in which **data** are collected from a **sample** (a subset) or a **population**. The data collected are used to create **estimates** of the characteristics or **parameters** of the population.

Some examples:

- A sample of voters are telephoned to ask their voting intentions, with the aim of predicting the outcome of the election;
- A sample of sites in a river catchment is tested for the presence of algae to determine the spread of the algae throughout the entire catchment;
- A sample of households is selected, and the residents asked about their employment status, in order to estimate the national unemployment rate.

A **census** is a special case of a sample survey in which every member of the population is surveyed.

3.1 The Survey Process

The different steps in the survey process are shown in Figure 3.1.

- Every survey has a set of **objectives** – the particular populations which are of interest;
- These parameters are properties of the **target population**: the population about which estimates are to be made;
- Not all members of the target population are able to be identified: the ones that can be surveyed make up the **survey population**;
- The **sample frame** is a listing of all members of the survey population;
- The **sample design** is the method of selecting the sample from the frame. The **sample size** is usually decided as a compromise between the required **accuracy of estimates** and the survey **costs** and other constraints (e.g. the time available for the survey);
- **survey instrument** is the means of data collection. It is usually a paper or electronic questionnaire, completed by the respondent or the interviewer/observer. The instrument aims to measure the properties of the sample members which mean that the desired population characteristic can be estimated. The instrument must be **valid** (it must actually measure what it intends to measure) and **reliable** (repeated measurements of the same sample member under identical circumstances should always yield similar results);
- The sample members are **contacted** and **recruited** into the sample. Not all selected members will be able to be contacted (even after strenuous efforts), and some will not

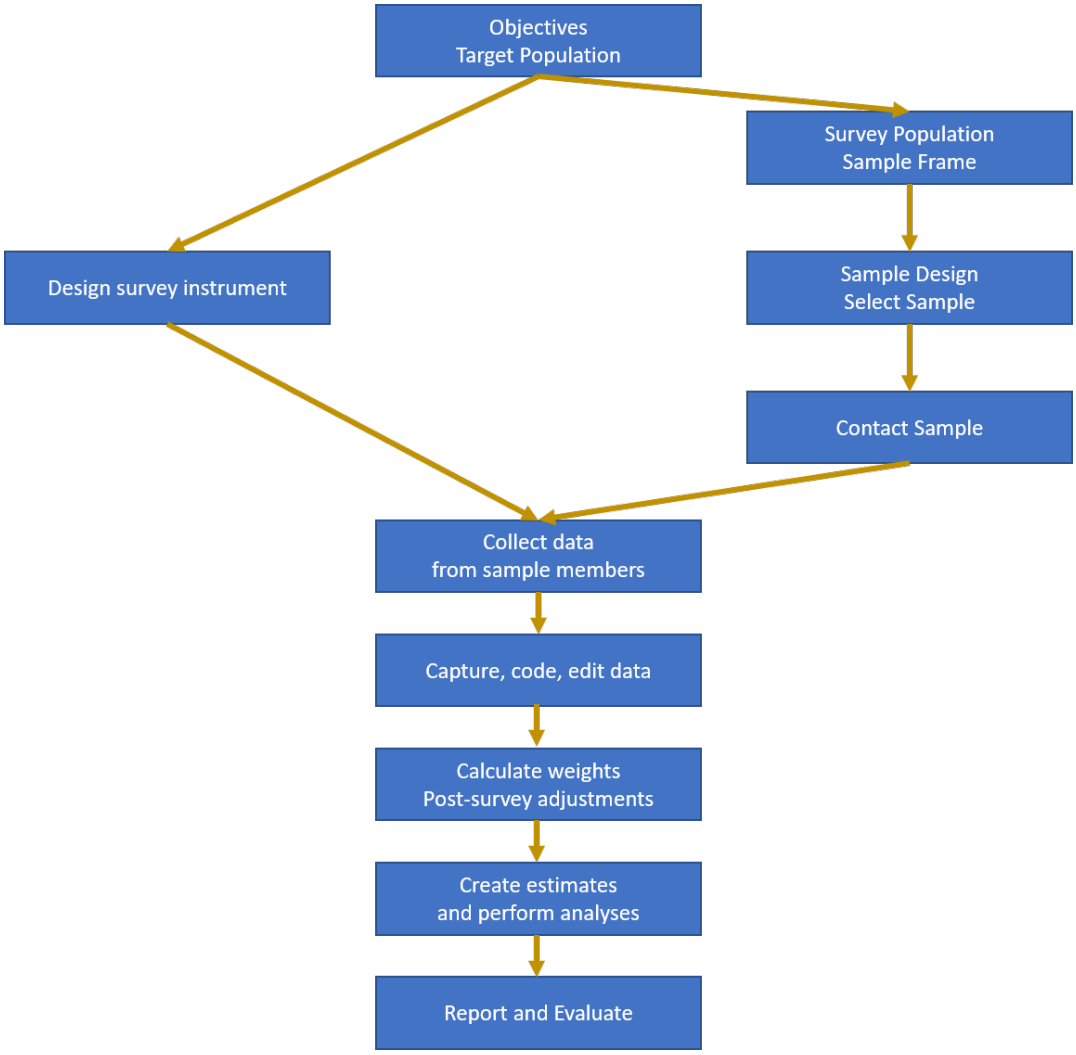


Figure 3.1: The Survey Process

respond even if contacted;
- The data are collected from the sample members in some **mode** (e.g. face-to-face, telephone, web, observational, …);
- The data collected are **captured** (stored on a computer); **coded** (converted into standard classification systems); **edited** (checks for data consistency); and **stored** in a final dataset;
- The data may be **adjusted**, e.g. imputation or weight adjustments for nonresponse may be done;
- **Estimates** of the parameters of interest are constructed, and other **analysis** of the data is carried out (e.g. regression estimation, comparison with other data etc.);
- The results are summarised in a **report**;
- The original data may be **archived** in some appropriate form, or **destroyed**;
- A post-survey **evaluation** may be made to determine how well the survey met its goals.

For example: The Household Labour Force Survey (HLFS).

- One of the main **objectives** of the HLFS is to estimate the unemployment rate every quarter;
- The **target population** is the working age population of New Zealand;
- The **survey population** is the civilian, non-institutionalised usually resident population of adults aged 15+ living in permanent, private dwellings on the main islands of New Zealand (North Island, South Island and Waiheke Island);
- The **sample frame** is a list of dwellings created at the most recent census and regularly updated to reflect changes;
- The **sample design** is a stratified cluster design. Within each regional government area a sample of small areas (PSUs) is selected. A sample of households is selected from within each PSU. Every adult from the selected households is surveyed. 15000 households and 30000 adults are surveyed every three months, in order to create unemployment estimates accurate to within 0.5%.
- The **survey instrument** is an electronic questionnaire.
- The interviewers contact the households in person or by telephone, making up to 10 call backs to ensure contact is made. **Proxy responses** are permitted (i.e. one household member can respond on behalf of another). The interview takes place by computer assisted personal or telephone interviewing (CAPI or CATI).
- The results are **post-stratified** to match the current population estimates in each local government area;
- **Estimates** of the unemployment rate are published within about 6 weeks of the end of the quarter.

3.2 Populations and Frames

The target population defines the **scope** of the survey: the set of units which make up the population in which we are interested.

The sample frame is a method of gaining access to the members of the target population. It is an actual or conceptual list. However it may not be perfect (Figure 3.2).

- Not all members of the target population will appear on the sample frame: the missing units are **out of coverage**;
- Not all units listed on the sample frame are members of the target population: the extra units are **out of scope**.
- The survey population are those units which are both in the target population (in scope) and on the frame (in coverage).

Our aim is to have a sample frame which covers most if not all of the target population, and which has only very few out of scope units.

The reason for this is very important: we can make estimates about units in a population **only** if they are included in the sample frame. This inference is possible **whether or not we select them**. If there is a subpopulation of units that is not on our frame (i.e. they are out of coverage) then we can never select them, we can never learn anything about them, and therefore we cannot make inferences about them.

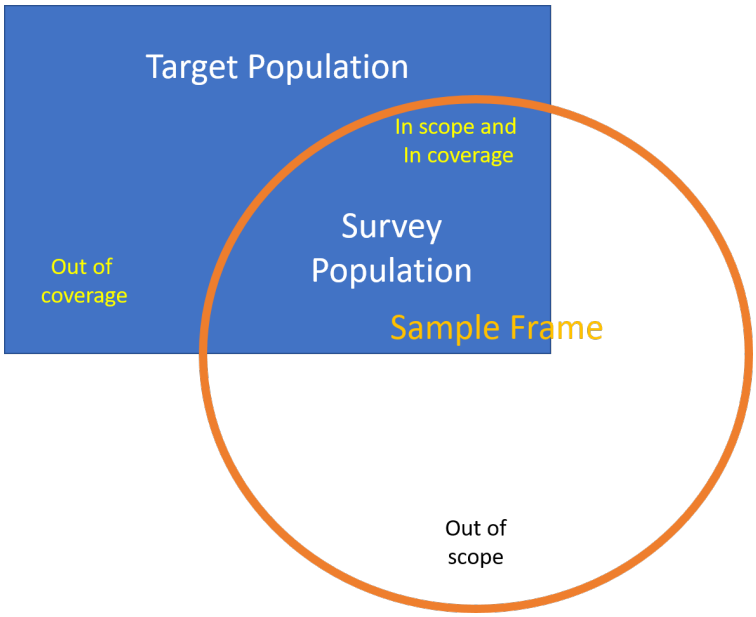


Figure 3.2: Scope and Coverage

3.2.1 What is a sampling frame?

The sampling frame is a key element in the survey process which provides the basis from which a *random* (probability based) sample can be selected. The purpose of having a sampling frame is to enable each member of the target population a nonzero probability of being selected to participate in the survey. The *quality* of the data, from a survey which has as its basis a comprehensive frame, can be *measured and controlled*.

The term sampling frame is sometimes misunderstood, often being thought of as a list containing each member of the population. In fact that is more a description of an 'ideal' sampling frame. The best way to show what a sampling frame can be in practice is to examine some examples:

- **List Frames.** Good up-to-date lists make ideal frames. However lists of units (electoral rolls, telephone listings etc.) are generally formed for purposes other than for use as sampling frames. For this reason they are often not close to 'ideal' sampling frames. Some discussion of list frame problems comes later.
- **Multi-stage Sampling Frames.** These frames consist of a set of lists which are arranged in a hierarchy of stages. For example a typical household survey frame will be organised like so: The first list is a a set of non-overlapping geographic areas from which a sample of areas is taken. For example we may list all the local regional councils and take a sample of them. Within these areas we might then list all the households

and take samples from the household lists. From each household we would then list the residents and interview selections of them. Note that we do not have to list the entire population. The only comprehensive list needed is the list of the areas which will be quite small. By giving each area a chance of being selected we ensure each person living in each area has a chance of selection.

- **Continuous Sampling Frames.** Consider the population of people entering a country from international flights. Suppose we choose a random number between one and ten ($= k$) and then interview the kth, $(k + 10)$th, $(k + 20)$th … persons who file through customs. We will have a sample where every person has had a 1 in 10 chance of being selected. But what is the frame? There is no list. We call such a situation a 'conceptual list', where the people filing into the country effectively formed a long queue of people which we have treated like a listing of the population, although a listing of names was never constructed. \end{description}

One can find many different definitions of sampling frames. The following is a simplified definition that should serve our purposes:

> A *sampling frame* is part of the system of rules and procedures used when determining who is to be surveyed and which ensure that each member of the population get some measurable (non-zero) chance of participating in the survey.

The examples above show that we need to think beyond the concept of a list of units, whose construction is a separate exercise from the other survey stages.

3.2.2 What is a *good* sampling frame?

We will often have several frames available and will have to decide between different options. To help us with this decision we need to be a little more specific about what makes a good sampling frame. The first thing we should consider is the effective coverage of the sampling frame. This can be determined by testing it for the following properties:

- **Each unit should be counted at least once**. If there are missing units (out of coverage) then the survey results may be biased if the excluded units tend to be different to the units which are included.
- **Each unit should be counted only once**. The problem is that we usually can not tell how many times a unit is duplicated. This means we can not tell how many chances a unit has of being sampled. This can cause similar bias if the units which are duplicated are different to the other units. The results will be biased towards the duplicated sub-group of the population.
- **Each unit should be distinguishable from the other units on the frame**. If we select a unit from the frame (say Jane Smith) but then can not tell who this refers to, when there are many Jane Smiths in the population, then the frame has not done its job.
- **Should provide up-to-date information about the units**. For example it if it claims to provide Jane Smith's address it should be the current address.
- **No units should be there if they are not meant to be**. Extra units (out of scope) are not a serious problem unless we fall into the elementary error of replacing them whenever we come across them in the sample. This confuses selection probabilities. The correct procedure is to drop any ineligible unit found in the sample and not replace them.
- **Should allow direct access to each unit**. Ideally the frame will ultimately supply us with a list of the units we are interested in analyzing. Sometimes a frame will only

give samples of groups of units e.g. a list of addresses, each of which may contain many people. Further work is needed before a sample of people is realized.

3.3 Approaches to sampling

We're talking here about random sampling, or **probability sampling**. For the purposes of statistical inference such sampling is the only defensible method - it's the only one that leads us to properly constructed confidence intervals and hypothesis tests. Other approaches do exist though, and even though you'll see them used they all have significant defects.

- **Probability sampling** is the gold standard: every member of the population has a known, non-zero chance of selection;
- **Purposive sampling**: the surveyor selects members of the population that they consider to be representative - dangerously biased towards the views and prejudices of the surveyor;
- **Haphazard sampling**: the surveyor uses any members of the population that they come across. This is what happens on the street corner when you're asked questions by a surveyor;
- **Quota sampling**: resembles stratified sample (see later): the surveyor fills quotas of respondents to balance numbers of respondents in categories (e.g. by age/gender/ethnicity), but does not make an accounting for non-response (see later);
- **Snowball sampling**: used in rare populations that are highly connected (e.g. refugees from a particular country): existing sample members recruit new sample members using their contacts;
- **Self-selected sampling**: this is the **worst** of all: the respondents themselves decide whether to be selected: they see an advertisement on a webpage or phone in with their views. Only profiles motivated population members - the silent majority are never measured.

Note: a **screening sample** is used when we are wanting to sample a population for which there is no suitable frame, but where all members of the target population are to be found on a larger frame, however they cannot be identified on the frame. For example there is no frame which can identify people who go fishing regularly.
The only solution is to sample from a whole-of-population frame, and then ask eligibility questions to determine whether the person selected is a member of the target population. Data is only collected from actual members of the target population. Screening may be done as part of a probability sample from the larger frame.

3.4 Example datasets

We'll use and re-use a number of standard datasets.

3.4.1 API

This is a dataset on Student performance in California schools. The dataset is distributed with the `survey` library in R, and there are a number of unit record datasets which are drawn from a population dataset `apipop` under various sample designs. The population units are the schools, and a number of characteristics are available. Full documentation can be found in the `survey` package. The variables we use mostly are:

- `stype` - School type (Elementary/Middle/High School)
- `snum` - School number
- `dnum` - District number

- `api99` - Academic Performance Indicator for the school in the year 1999
- `api00` - Academic Performance Indicator for the school in the year 2000
- `sch.wide` - Did the school meet its school-wide growth target?
- `awards` - Is the school eligible for an awards program
- `meals` - Percentage of students eligible for subsidised meals
- `ell` - Percentage of students that are English Language Learners
- `grad.sch` - Percentage of students with parents with a postgraduate education
- `enroll` - Number of students enrolled
- `api.stu` - Number of students tested in the API

3.4.2 NZ Schools

A list of NZ schools from the Education Counts website (https://www.educationcounts.govt.nz/directories/list-of-nz-schools)

The dataset includes

- `School_Id` - an identification number
- `Org_Name` - the name of the school
- Contact details (`Telephone`, `Fax`, `Email`, `Contact` email address, postal address)
- `Org_type` - type of school (primary/secondary etc)
- `Total` - number of enrolled students
- `Europenan`, Māori,Pacific,Asian,MLAA,Other‘ - counts of students by ethnicity
- `International` - numbers of international students
- `Urban_Rural_indicator` - categorised population density of the area around the school
- `Territorial_Aithority`, `Regional_Council`, `Education_Region` - administrative regions
- `Latitude`, `Longitude` - location

3.4.3 From the dplyr package

- `starwars` - a list of characteristics of 87 characters from the Star Wars movie series;

Chapter 4

Statistical Inference

4.1 Inference in infinite populations

Let's briefly review a few familiar results from earlier statistics courses.

Most of what we've done in earlier courses has assumed that we have a **simple random sample** from an **infinite** population.

We'll use data from the `starwars` dataset (in the `dplyr` package). There are data on 87 characters, including their heights in centimetres.
Six heights are missing, but let's just guess them so we have no missing data (this process of inventing data has a respectable name: **'imputation'**, and we'll come back to it later).

```
library(dplyr)
data(starwars)
sum(is.na(starwars$height))
```

```
## [1] 6
```

```
starwars <- starwars |>
  mutate(height=case_when(name=="Arvel Crynyd" ~ 172, # same as Luke
                          name=="Finn" ~ 172, # same as Luke
                          name=="Rey" ~ 150,  # same as Leia
                          name=="Poe Dameron" ~ 172, # same as Luke
                          name=="BB8" ~ 96, # same as R2D2
                          name=="Captain Phasma" ~ 167, # same as C3PO
                          .default=height))
```

The mean height $\bar{Y}$ of characters in the Star Wars universe is a **population parameter**. We can **estimate** it from a sample of characters.

> **Note:** we're using capital letters for **population parameters** ($\bar{Y}$ is the population mean), and lower case letters for **sample statistics** ($\bar{y}$ is the sample mean).
>
> It's also common to see Greek letters for **population parameters** (e.g. μ for the population mean), upper case latin letters for random variables, and lower case latin letters for sample statistics.

For the purposes of this exercise let's imagine there are only $N = 87$ individuals in the entire Star Wars universe. Let's draw a random sample of $n = 20$ from this population of $N = 87$:

name	height	mass	hair_color	homeworld
Tarfful	234	136	brown	Kashyyyk
Jek Tono Porkins	180	110	brown	Bestine IV
Bossk	190	113	none	Trandosha
Lando Calrissian	177	79	black	Socorro
IG-88	200	140	none	NA
Kit Fisto	196	87	none	Glee Anselm
Biggs Darklighter	183	84	black	Tatooine
Obi-Wan Kenobi	182	77	auburn, white	Stewjon
Quarsh Panaka	183	NA	black	Naboo
Mace Windu	188	84	none	Haruun Kal
Ratts Tyerel	79	15	none	Aleen Minor
Beru Whitesun Lars	165	75	brown	Tatooine
Chewbacca	228	112	brown	Kashyyyk
Jabba Desilijic Tiure	175	1358	NA	Nal Hutta
Rugor Nass	206	NA	none	Naboo
Sly Moore	178	48	none	Umbara
Dormé	165	NA	brown	Naboo
Leia Organa	150	49	brown	Alderaan
Finn	172	NA	black	NA
Darth Maul	175	80	none	Dathomir

```
set.seed(222)
bigN <- nrow(starwars)
n <- 20
samplemembers <- starwars[sample(bigN, n, replace=FALSE),]
samplemembers |>
  dplyr::select(name,height,mass,hair_color,homeworld) |>
  kbl() |>
  kable_styling()
```

Compute the sample mean $\bar{y}$ and standard deviation s_y:

```
ybar <- mean(samplemembers$height)
ybar
```

```
## [1] 180.3
```

```
sdy <- sd(samplemembers$height)
sdy
```

```
## [1] 31.13147
```

We know how to make a point estimate $\widehat{\bar{Y}}$ for the population mean $\bar{Y}$ (remember that a hat $\hat{\cdot}$ over a symbol indicates an estimate of that parameter): the population mean is estimated by the sample mean $\bar{y}$:

$$\widehat{\bar{Y}} = \bar{y} = 180.3$$

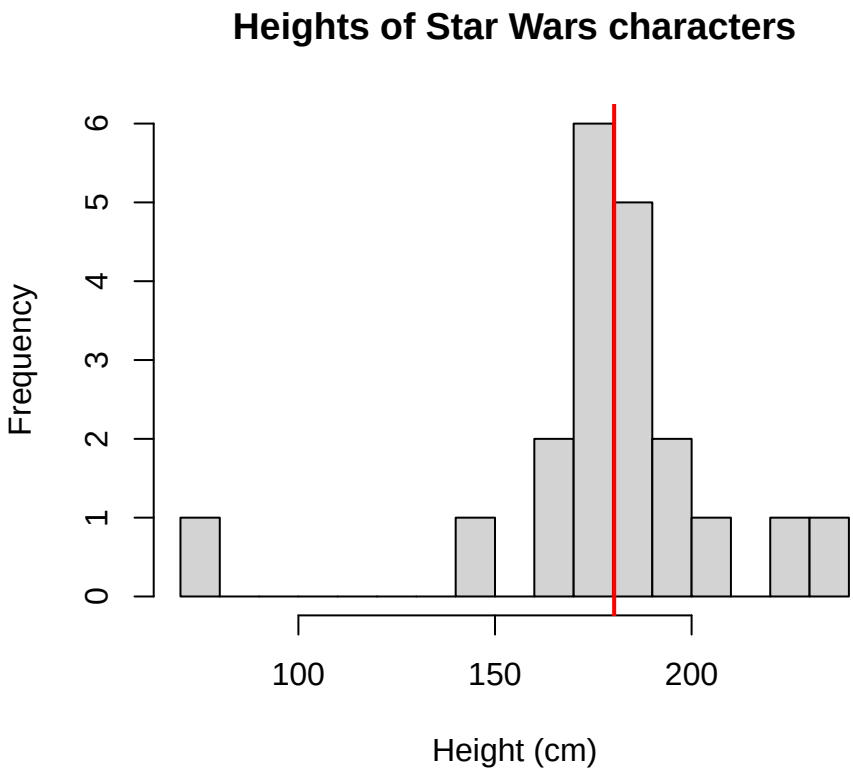


Figure 4.1: Histogram of heights of a sample of Star Wars characters (vertical line at the sample mean)

and a 95% confidence interval is given by

$$\begin{aligned}
\widehat{\bar{Y}} \pm \mathbf{MOE}[\widehat{\bar{Y}}] &= \widehat{\bar{Y}} \pm T^*_{n-1} \times \mathbf{SE}[\widehat{\bar{Y}}] && (4.1)\\
&= \bar{y} \pm T^*_{n-1} \times \frac{s_y}{\sqrt{n}} && (4.2)\\
&= 180.3 \pm 2.09 \times \frac{31.1}{\sqrt{20}} && (4.3)\\
&= 180.3 \pm 2.09 \times 6.96 && (4.4)\\
&= 180.3 \pm 14.6 && (4.5)\\
&= (165.7, 194.9) && (4.6)
\end{aligned}$$

Here:

- $\mathbf{SE}[\widehat{\bar{Y}}]$ is the **standard error** or **sampling error** of the estimator $\widehat{\bar{Y}}$ of the population mean $\bar{Y}$;
- T^*_{n-1} is the quantile of the $t-$distribution with $n-1$ degrees of freedom so that the interval $[-T^*_{n-1}, +T^*_{n-1}]$ encloses 95% of the probability of the $t-$distribution;
- $\mathbf{MOE}[\widehat{\bar{Y}}]$ is the **margin of error**, the half-width of the confidence interval, and is the standard error scaled by T^*_{n-1}.

This is what is known as a **model based** estimate, the statistical model being

$$y_i = \mu + \varepsilon_i \qquad \text{where } \varepsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2) \tag{4.7}$$

There are just two parameters here: the population mean μ, and the population standard deviation σ.

We can fit this simplest of all linear regression models using the `lm()` function and get the same results:

```
lm1 <- lm(height ~ 1, data=samplemembers)
summary(lm1)
```

```
##
## Call:
## lm(formula = height ~ 1, data = samplemembers)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -101.30   -6.05    0.70   11.20   53.70
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  180.300      6.961    25.9 2.77e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 31.13 on 19 degrees of freedom
```

```
confint(lm1)
```

```
##               2.5 % 97.5 %
## (Intercept) 165.73 194.87
```

Here `(Intercept)` is the estimate for μ, the population mean.
The 'residual standard error' in the R output is actually what we'd rather call $\hat{\sigma}$, our estimate of the standard deivation σ.

We can get hold of this value from a fitted `lm()` object using the `sigma()` function:

```
sigma(lm1)
```

```
## [1] 31.13147
```

The confidence interval anove is only valid if the model in equation (4.7) holds: i.e. if the data are normally distributed about the mean. The histogram in Figure 4.1 suggests that this is not the case however, since there is a clear outlier at low height.

> The Central Limit Theorem (CLT) tells us that if n is large enough then our estimate for μ is useable, and its confidence interval realistic, even if the model in equation (4.7) doesn't actually hold.
>
> The CLT enables much of statistcal practice to yield meaningful results even without our knowing the true data generation model.
>
> In this course most of the examples will have sample sizes large enough that the CLT will mean our results are valid. In these cases when constructing a confidence interval it will be sufficient to replace T^*_{n-1} with the equivalent $Z^* = 1.96$ from the Normal distribution.

4.2 Inference in finite populations

Recall that the standard error of an estimate is the variability of estimates between different samples: in fact it is defined as the standard deviation of all possible estimates from all possible estimates drawn using the same sample design.

In an infinite population there are an infinite number of possible samples.

However in a finite population that isn't the case. In a population of size N if we draw a simple random sample without replacement of size n there are

$$\binom{N}{n} = {}^{N}C_{n} \tag{4.8}$$

$$= \frac{N!}{n!(N-n)!} \tag{4.9}$$

$$= \frac{N \times (N-1) \times (N-2) \times ... \times (N-n+1)}{n \times (n-1) \times (n-2) \times ... \times 1} \tag{4.10}$$

possible samples: i.e. the number of ways it is possible to draw n objects from a collection of N without regard to the order they are selected.

> Recall that $n!$ is the factorial of n: the product of all integers from n down to 1:
>
> $$n! = n \times (n-1) \times (n-2) \times ... \times 2 \times 1$$
>
> So $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

To calculate a factorial in R use the `factorial()` function

```
factorial(5)
```

```
## [1] 120
```

And to evaluate $\binom{N}{n}$ use the `choose()` function: e.g. the number of ways to choose 4 objects from a collection of 10:

```
choose(10,4)
```

```
## [1] 210
```

In our example of selecting $n = 20$ from $N = 87$ there is a truly astronomical number of possible samples:

$$\binom{87}{20} = \frac{87!}{20!67!} = 2.38 \times 10^{19}$$

There is of course only one way to select a census though:

$$\binom{87}{87} = \frac{87!}{87!0!} = 1$$

(using the at first face slightly annoying convention that 0!=1). Since there's only one possible sample when we're drawing a census the variation among sample estimates from different samples is zero: i.e. the standard error is zero.

It's also true that if n is close to N then sample estimates hardly differ from one another.

We clearly need a modification of our standard error formula

$$\mathbf{SE}[\widehat{\bar{Y}}] = \frac{s_y}{\sqrt{n}} \qquad \text{(Infinite population)}$$

to allow for sample sizes n being close to the population size.

That modification is the **finite population correction (fpc)**:

$$\mathbf{SE}[\widehat{\bar{Y}}] = \sqrt{1 - \frac{n}{N}} \times \frac{s_y}{\sqrt{n}} \qquad \text{(Finite population)}$$

The factor $\sqrt{1 - n/N}$ squashes the standard error to zero in the case that $n = N$, but for very small n has a value close to 1, and makes no difference at all.

In our example we had a standard error of

$$\begin{aligned} \mathbf{SE}[\widehat{\bar{Y}}] &= \frac{s_y}{\sqrt{n}} && (4.11)\\ &= \frac{31.1}{\sqrt{20}} && (4.12)\\ &= 6.96 && (4.13) \end{aligned}$$

If we incorporate the fpc this becomes

$$\begin{aligned} \mathbf{SE}[\widehat{\bar{Y}}] &= \sqrt{1 - \frac{n}{N}} \times \frac{s_y}{\sqrt{n}} && (4.14)\\ &= \sqrt{1 - \frac{20}{87}} \frac{31.1}{\sqrt{20}} && (4.15)\\ &= 0.8776 \times \frac{31.1}{\sqrt{20}} && (4.16)\\ &= 6.11 && (4.17) \end{aligned}$$

This narrows the confidence interval for the estimate of the population mean height from (165.7,194.9) to (167.5,193.1).

This is a **design based** confidence interval, in which it is the sample design which foremost in our derivation of standard errors.

There is quite a lot of theory behind derivation of standard errors in the complex designs which arise in sample surveys.

The `survey` package in R implements the consequences of sample designs which will mean we can use the theory without having to derive it.

For example here's a call to some `survey` package functions which gives us the effect of the fpc in our example:

```
samplemembers$bigN <- bigN
ss <- svymean(height~1, design=svydesign(ids=~1, fpc=~bigN, data=samplemembers))
ss
```

```
##         mean     SE
## height 180.3 6.1089
```

```
confint(ss)
```

```
##           2.5 %   97.5 %
## height 168.3268 192.2732
```

If you're interested in the theory take a look at a text like Cochran (1977) or Särndal et al. (1992).

4.3 The survey inference process

Sampling begins with the sample frame - usually this is a **list** of the members, or **units**, which make up the population.

The list contains at minimum some **identifier** of each unit, which enables it to be found/contacted/measured once it has been selected, and may contain a lot of other information too.

Data that we have on the frame is called **auxiliary information**: it is data that is available about a unit whether or not it has been sampled. Some sample designs rely on this auxiliary data during the process of sample selection.

In probabilistic (random) sampling each unit i has a known nonzero chance of selection π_i, which may depend on the auxiliary data in some way. These probabilities don't have to be equal, just known and nonzero.

In random sampling we generate a random variable I_i for each population member, based on the sample design, to decide whether it is selected into the sample or not:

$$I_i = \begin{cases} 1, & \text{if unit } i \text{ is selected} \\ 0, & \text{otherwise} \end{cases} \tag{4.18}$$

The sample size is just the sum of these random variables

$$n = \sum_{i=1}^{N} I_i$$

So we start with the N rows of the sample frame (Table 4.1) generate the sample indicators I_i, and reduce to the n rows of data in Table 4.2. For each member k of the sample we know

Table 4.1: Sample Frame (N rows)

Unit ID	Auxiliary Data	Selection Probability	Selection Indicator
1	$\mathbf{X}_1$	π_1	I_1
2	$\mathbf{X}_2$	π_2	I_2
3	$\mathbf{X}_3$	π_3	I_3
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$\mathbf{X}_i$	π_i	I_i
$\vdots$	$\vdots$	$\vdots$	$\vdots$
N	$\mathbf{X}_N$	π_N	I_N

Table 4.2: Sample Data (n rows)

Sample Unit	Unit ID	Auxiliary Data	Selection Probability	Sample Weight	Sample Data
1	i_1	$\mathbf{x}_1$	π_1	w_1	$\mathbf{y}_1$
2	i_2	$\mathbf{x}_2$	π_2	w_2	$\mathbf{y}_2$
3	i_3	$\mathbf{x}_3$	π_3	w_3	$\mathbf{y}_3$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
k	i_k	$\mathbf{x}_k$	π_k	w_k	$\mathbf{y}_k$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	i_n	$\mathbf{x}_n$	π_n	w_n	$\mathbf{y}_n$

its population identifier i_k, its auxiliary information $\mathbf{x}_k$, selection probability π_k, and we also collect and record the survey data $\mathbf{y}_k$.

The **expected value** of each indicator variable is π_i

$$\begin{aligned} E[I_i] &= 1 \times \Pr(I_i = 1) + 0 \times \Pr(I_i = 0) && (4.19)\\ &= 1 \times \pi_i + 0 \times (1 - \pi_i) && (4.20)\\ &= \pi_i && (4.21) \end{aligned}$$

Each selected sample member carries a **sample weight** which is the number of units in the population that that unit represents. These weights are defined as the **inverses of the selection probabilities**.

$$W_i = \frac{1}{\pi_i}$$

Consider some characteristic Y_i which each population member has. e.g. in a survey of households we might be measuring the amount of money Y_i that household i spends on internet streaming services. The total amount of money spent by all households in the population combined is

$$Y = \sum_{i=1}^{N} Y_i$$

However we only know Y_i for some households: those which are selected, i.e. those which have $I_i = 1$.

The **Horwitz-Thompson** (HT) estimator of the population total is

$$\widehat{Y} = \sum_{i=1}^{N} I_i W_i Y_i \tag{4.22}$$

$$= \sum_{k=1}^{n} w_k y_k \tag{4.23}$$

i.e. a weighted sum of the observed household spending values y_k just for the n selected households. ($I_i = 0$ for any unselected households, so the sum of N terms simplifies to a sum of n terms, just over the sample members.)

We can see that the HT estimator is **unbiased**:

$$E[\widehat{Y}] = \sum_{i=1}^{N} E[I_i] W_i Y_i \tag{4.24}$$

$$= \sum_{i=1}^{N} \pi_i W_i Y_i \tag{4.25}$$

$$= \sum_{i=1}^{N} \pi_i \frac{1}{\pi_i} Y_i \tag{4.26}$$

$$= \sum_{i=1}^{N} Y_i \tag{4.27}$$

$$= Y \tag{4.28}$$

i.e. although the observed value $\widehat{Y}$ from a given sample may not equal the true value Y, when averaged over all possible samples we can see that on average it does equal the true population value.

We also want the standard error $\mathbf{SE}[\widehat{Y}]$ to be small, so that we have precise estimates.

We're all familiar with the idea that the larger the sample size n the smaller the standard error. We now know that the standard error is going to crush down to zero in finite populations the closer we get to a census.

There's a further advantage we can get by making a good choice of **sample design**.
Two different methods of selecting a sample of the **same sample size** n may lead to very different values of the standard error, and thus confidence intervals of very different widths.

Incidentally, the expected value of the sum of the sample weights among sample members is

equal to the population size:

$$\sum_{k=1}^{n} w_k = \sum_{i=1}^{N} I_i W_i \tag{4.29}$$

$$E\left[\sum_{k=1}^{n} w_k\right] = \sum_{i=1}^{N} E[I_i] W_i \tag{4.30}$$

$$= \sum_{i=1}^{N} \pi_i W_i \tag{4.31}$$

$$= \sum_{i=1}^{N} \pi_i \frac{1}{\pi_i} \tag{4.32}$$

$$= \sum_{i=1}^{N} 1 \tag{4.33}$$

$$= N \tag{4.34}$$

Here's one way to think about sample weights. We have a population of size N we want to make estimates about: what we'd really like is a dataset of all N individuals. Instead we only have $n < N$ individuals. So we (conceptually) replicate each of the n sampled indidividuals by their weight: this makes a new synthetic dataset which does have the N entries we're interested in. Each row is identical to a number of others, however in the real population there are many individuals who sometimes very closely resemble each other. So the properties of our synthetic population may be good enough for estimating the properties of the true population.

The weight of a sample member indicates how many members of the population that sample member represents. If we had a population of size $N = 100$ and a simple random sample of size $n = 20$ then each sample member represents $w = 100/20 = 5$ people in the population.

In many designs the weights differ between individuals: high weights mean that the sample member represents many other individuals. A weight of 1 means that sample member represents themself alone.

There's no problem with weights being fractions - since we only conceptually but don't **actually** replicate the data.

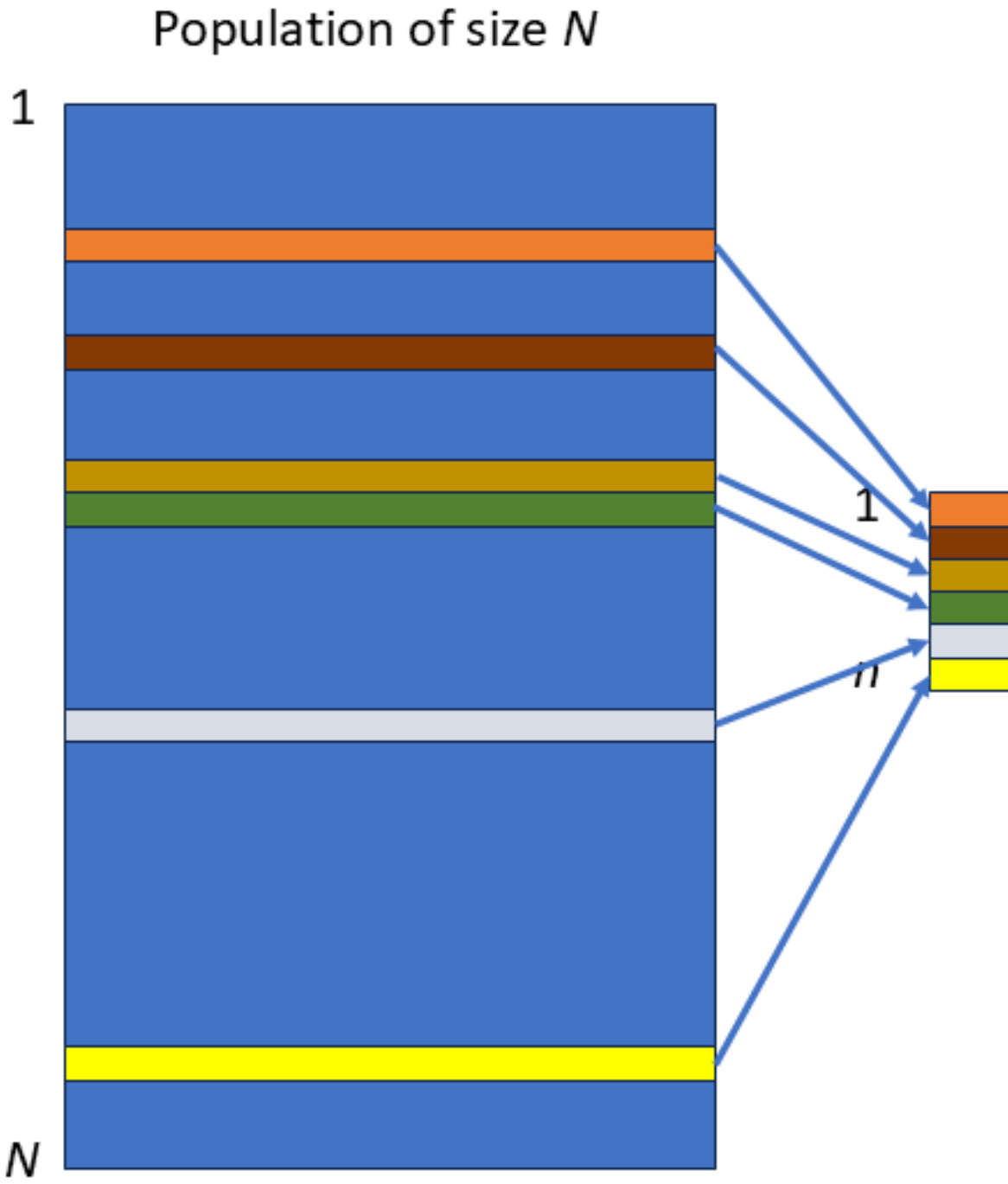
4.4 Example

We might have a small street of five houses. Our frame is just the list of addresses. We might know the household rateable value from the City Council list of dwellings.

```
hframe <- data.frame(
  ID=c(6451,6452,8773,7164,6125),
  Address=c("1 Chestnut Place","2 Chestnut Place","2A Chestnut Place",
            "3 Chestnut Place","4 Chestnut Place"),
  Value=c(1.201, 0.780, 0.520, 2.530, 0.900)
)
```

We might then select a simple random sample of $n = 2$ houses from the $N = 5$ in the small street. The probability of selection is $\pi_i = 2/5$ for each house. We draw our random sample and select house number 2A and house number 5 into our sample.

We visit each selected house and ask how much they spend y_k on all their internet streaming services each month.


Population of size N
1
N
1
n

ID	Address	Value ($M)
6451	1 Chestnut Place	1.201
6452	2 Chestnut Place	0.780
8773	2A Chestnut Place	0.520
7164	3 Chestnut Place	2.530
6125	4 Chestnut Place	0.900

ID	Address	Value ($M)	Prob. π_i	Selected, I_i
6451	1 Chestnut Place	1.201	0.4	0
6452	2 Chestnut Place	0.780	0.4	0
8773	2A Chestnut Place	0.520	0.4	1
7164	3 Chestnut Place	2.530	0.4	0
6125	4 Chestnut Place	0.900	0.4	1

Each house in the sample has a weight of $2.5 = 5/2$: i.e. it represents itself and 1.5 other houses in the street. The sum of the weights of the two selected houses is equal to the population size $N = 5$.

Our sample is really too small to do any sensible inference on: but let's do it anyway to show how things work. We calculate the mean and standard deviation of the monthly internet streaming values: $\bar{y} = \$47.00$, and $s_y = \$39.60$.

Our estimate of the mean monthly spend is $\bar{y} = \$47.0$, with a margin of error of

$$\begin{aligned} \mathbf{MOE}[\widehat{\bar{Y}}] &= Z^* \sqrt{1 - \frac{n}{N}} \times \frac{s_y}{\sqrt{n}} && (4.35) \\ &= 1.96 \sqrt{1 - \frac{2}{5}} \times \frac{\$39.60}{\sqrt{2}} && (4.36) \\ &= \$42.51 && (4.37) \end{aligned}$$

which gives a 95% confidence interval of ($4.49,$89.51) (although the use of the large sample value $Z^* = 1.96$ is very much **not** justified here.)

In finite populations we are sometimes interested in population totals - which don't make any sense in infinite populations. Now we have an estimate of the mean spend per house, we can scale it up by the population size to see how much per month is being spent by all of the houses combined:

$$\begin{aligned} \widehat{Y} &= N\widehat{\bar{Y}} && (4.38) \\ &= 5 \times \$47.00 && (4.39) \\ &= \$235.00 && (4.40) \end{aligned}$$

and we can scale the confidence interval up in the same way

$$\begin{aligned} \mathbf{CI}[\widehat{Y}] &= N\mathbf{CI}[\widehat{\bar{Y}}] && (4.41) \\ &= 5 \times (\$4.49, \$89.51) && (4.42) \\ &= (\$22.45, \$447.55) && (4.43) \end{aligned}$$

	ID	Address	Value ($M)	Weight w_k	Internet spend, y_k
3	8773	2A Chestnut Place	0.52	2.5	75
5	6125	4 Chestnut Place	0.90	2.5	19

Chapter 5

Sample Designs

In this Chapter we consider the various sample designs in common use, the reasons for using each one, and methods for selecting from suitable frames.

We'll use a variety of datasets.

5.1 Census

The simplest of all! We select every unit in the population, so $n = N$.

The probability of selection is

$$\pi = \frac{n}{N} = \frac{N}{N} = 1$$

i.e. every unit is selected with certainty, and the sample weights are all

$$w_i = \frac{1}{\pi_i} = \frac{1}{1} = 1$$

so each unit in the sample represents itself alone.

Design considerations

None: we're sampling all N members of the population.

5.2 Simple Random Sampling Without Replacement (SRSWOR)

The Simple Random Sample WithOut Replacement is the most fundamental of all sample designs, and it appears nested inside many more complex designs as we'll see later.

We have a population of size N which is fully enumerated (i.e. fully listed), and we want to select a sample of size n.

The defining property of a SRSWOR is that **all samples of size** n have an equal chance of selection. A **consequence** of this definition is that **all population units** have an equal chance of selection.

There are

$$\binom{N}{n} = {}^{N}C_{n} = \frac{N!}{n!(N-n)!}$$

possible samples. The R function `choose(n,k)` can be used for evaluating the number of possible samples. e.g. the number of ways to select $n = 20$ units from $N = 87$ is

```
choose(87,20)
```

```
## [1] 2.375456e+19
```

The probability of selection of each unit is

$$\pi_i = \frac{n}{N}$$

leading to sample weights

$$w_i = \frac{N}{n}$$

The SRSWOR is used when the **costs** of different samples are not known to differ (e.g. there are no units that are more costly to visit than others), and where there is no informative auxiliary information.

If we have a sample frame stored as an R data frame `sframe` with a column `ID` uniquely identifying the units, then we can select an SRSWOR as follows:

```
bigN <- nrow(sframe)
samp <- sframe[sample(bigN,n),]
samp$weight <- bigN/n
```

or

```
bigN <- nrow(sframe)
IDset <- sample(sframe$ID,n)
samp <- sframe[sframe$ID %in% IDset,]
samp$weight <- bigN/n
```

or as a function

```
select.srswor <- function(df, n) {
  # SRSWOR
  bigN <- nrow(df)
  if(n<=bigN) {
    samp <- df[sample(bigN,n,replace=FALSE),]
    samp$bigN <- bigN
    samp$weight <- bigN/n
  } else {
    stop("Cannot have a SRSWOR with n>N")
  }
  return(samp)
}
```

Design considerations

- We only need to decide on the sample size n (see later).

Example

Select $n = 5$ units from a frame listing of airports with IATA codes sourced from Kaggle https://www.kaggle.com/datasets/syedasimalishah/airports/data

The top ten rows of the dataset of $N = 6072$ are shown here:


Sir

ID	IATA	Name	City	Country	Latitude	Longitude
1	AAA	Anaa Airport	Anaa	French Polynesia	-17.352600	-145.509995
2	AAC	El Arish International Airport	El Arish	Egypt	31.073299	33.835800
3	AAE	Rabah Bitat Airport	Annaba	Algeria	36.822201	7.809174
4	AAF	Apalachicola Regional Airport	Apalachicola	United States	29.727501	-85.027496
5	AAH	Aachen-Merzbrück Airport	Aachen	Germany	50.823055	6.186389
6	AAK	Buariki Airport	Buariki	Kiribati	0.185278	173.636993
7	AAL	Aalborg Airport	Aalborg	Denmark	57.092759	9.849243
8	AAM	Malamala Airport	Malamala	South Africa	-24.818100	31.544600
9	AAN	Al Ain International Airport	Al Ain	United Arab Emirates	24.261700	55.609200
10	AAO	Anaco Airport	Anaco	Venezuela	9.430225	-64.470726

Table 5.1: Sample of airports (SRSWOR)

ID	IATA	Name	City	Country	Latitude
3593	NTD	Point Mugu Naval Air Station (Naval Base Ventura Co)	Point Mugu	United States	34.12030
1472	EUM	Neumünster Airport	Neumuenster	Germany	54.07944
2512	KHZ	Kauehi Airport	Kauehi	French Polynesia	-15.78080
197	ANC	Ted Stevens Anchorage International Airport	Anchorage	United States	61.17440
4710	SRN	Strahan Airport	Strahan	Australia	-42.15500

And here is a SRSWOR of $n = 5$ airports

```
samp <- airports |> select.srswor(n=5)
```

5.3 Simple Random Sampling With Replacement (SRSWR)

This design is rarely used, but is the simplest form of Probability Proportional to Size With Replacement (PPSWR) sampling, but with the sizes all equal.

Sampling proceeds in principle by selecting one unit at a time where each unit has a $1/N$ chance of being selected each time. After n draws the probability a unit has been selected at least once is

$$p_i = 1 - \left(1 - \frac{1}{N}\right)^n$$

(This is 1 minus the chance that a unit is not selected on any of the n sampling occasions.)

If n is tiny compared to N, p_i is approximately n/N which is the SRSWOR selection probability. In practice we assign a weight of N/n to each unit as it is selected, so if unit i is selected m_i times then it ends up with a weight of

$$w_i = m\frac{N}{n}$$

If we use the approximate weights calculated in this way we have what is known as the Hansen-Hurwitz estimator, rather than the Horwitz-Thompson estimator. However we think of the weights in the same way and this distinction need not concern us.

Design considerations

As in a SRSWOR, we only need to decide on the sample size n.

Table 5.2: Sample of airports (SRSWOR)

ID	IATA	Name	City	Country	Latitude	Longitude	nsampled
805	CCB	Cable Airport	Upland	United States	34.11160	-117.6880	1
1063	CUD	Caloundra Airport	Caloundra	Australia	-26.80000	153.1000	1
1077	CVE	Coveñas Airport	Coveñas	Colombia	9.40092	-75.6913	1
2738	LAY	Ladysmith Airport	Ladysmith	South Africa	-28.58170	29.7497	1
4804	SXN	Sua Pan Airport	Sowa	Botswana	-20.55340	26.1158	1

Example

Here is a SRSWR of $n = 5$ airports: note that we have aggregated across any instances of multiple selections, so that in the output sample list each unit appears only once no matter how often it was selected.

```
bigN <- nrow(airports)
n <- 5
samp <- airports[sample(bigN,n,replace=TRUE),]
samp$weight <- bigN/n
samp <- samp |>
  group_by(ID) |>
  summarise(across(-weight, first), nsampled=n(), weight=sum(weight))
```

or as a function

```
select.srswr <- function(df, n) {
  # SRSWR
  bigN <- nrow(df)
  df$rownames <- 1:bigN
  samp <- df[sample(bigN,n,replace=TRUE),]
  samp$bigN <- bigN
  samp$weight <- bigN/n
  samp <- samp |>
    group_by(rownames) |>
    summarise(across(-weight, first), nsampled=n(), weight=sum(weight)) |>
    as.data.frame() |>
    ungroup() |>
    select(-rownames)
  return(samp)
}
```

```
samp <- airports |> select.srswr(n=5)
```

5.4 Probability proportion to size Sampling (PPSWR)

This sample design generalises the SRSWR design to unequal probabilities of selection. This design recognises that some units may have more influence on our estimates because of some **size** property that is known (it must be on the frame). We therefore give these units higher probabilities of being selected, and therefore lower weight.

We draw n units, one at a time, but with probabilities which are proportional to the size measure X_i: thus the probability of selection of unit i on each of the n draws from the

Country	Code	Year	Population	Prob
Afghanistan	AFG	2023	41454762	0.0051249
Albania	ALB	2023	2811660	0.0003476
Algeria	DZA	2023	46164222	0.0057071
American Samoa	ASM	2023	47544	0.0000059
Andorra	AND	2023	80869	0.0000100
Angola	AGO	2023	36749909	0.0045432
Anguilla	AIA	2023	14435	0.0000018
Antigua and Barbuda	ATG	2023	93331	0.0000115
Argentina	ARG	2023	45538402	0.0056297
Armenia	ARM	2023	2943398	0.0003639

population is

$$p_i = \frac{X_i}{\sum_{j=1}^{N} X_j}$$

Approximating the probability of selection π after n draws as np_i leads to the sample weights

$$w_i = \frac{1}{np_i}$$

In a PPSWR sample these weights are not guaranteed to add up to the population size.

Design considerations

We need to decide on:

- the size variable X_i (which must be known for every unit in the population), and
- the sample size n.

The size variable X_i should correlate in some way with the population characteristics we want to estimate. If we're interested in estimating the total economic output of businesses, then the amount of tax each business paid last year would be a good size variable. But it wouldn't be helpful if we were interested in estimating the proportion of businesses with family friendly employment practices.

With replacement designs lose some efficiency because their estimators do not incorporate a finite population correction in their standard errors. For small sampling fractions (i.e. small n/N) this is not a significant drawback.

Example

We have a list of $N = 236$ countries and their 2023 populations: the first 10 are listed below

Select a PPSWR sample of $n = 10$ countries, with probability proportional to their population sizes.

```
set.seed(111)
bigN <- nrow(pop)
n <- 10
samp <- pop[sample(bigN,n,prob=pop$Prob,replace=TRUE),]
samp$weight <- 1/(n*samp$Prob)
samp <- samp |>
  group_by(Country) |>
  summarise(across(-weight, first), nsampled=n(), weight=sum(weight))
```

Country	Code	Year	Population	Prob	bigN	nsampled	weight
Bangladesh	BGD	2023	171466986	0.0211977	236	1	23.6
Brazil	BRA	2023	211140731	0.0261023	236	1	23.6
India	IND	2023	1438069597	0.1777819	236	2	47.2
Indonesia	IDN	2023	281190068	0.0347622	236	1	23.6
Japan	JPN	2023	124370947	0.0153754	236	1	23.6
Pakistan	PAK	2023	247504505	0.0305978	236	1	23.6
South Africa	ZAF	2023	63212386	0.0078147	236	1	23.6
United States	USA	2023	343477330	0.0424625	236	2	47.2

or as a function

```
select.ppswr <- function(df, n, sizevar) {
  # PPSWR
  bigN <- nrow(df)
  df$rownames <- 1:bigN
  samp <- df[sample(bigN,n,prob=df[,sizevar],replace=TRUE),]
  samp$bigN <- bigN
  samp$weight <- bigN/n
  samp <- samp |>
    group_by(rownames) |>
    summarise(across(-weight, first), nsampled=n(), weight=sum(weight)) |>
    as.data.frame() |>
    ungroup() |>
    select(-rownames)
  return(samp)
}
```

```
samp <- pop |> select.ppswr(n=10, sizevar="Population")
```

5.5 Linear Systematic Random Sample (LSRS)

This design is often used in cases where the population is not fully enumerated, but is presented to us in the form of an ordered list or a queue. For example we might want to select one in ten passengers cars checking in for a ferry crossing. Another example is where there is information in the ordering of a list: for example a list might be sorted from youngest to oldest, or sorted from north to south. We can space out our sample down the list, avoiding the chance that our sample doesn't have a good spread of ages, or a good spread of latitudes.

We select n units from N by first calculating the **skip** k, which is the nearest integer at or below N/n. We then choose at random one unit from the first k of our list or queue, say that is unit number j. We then select units j, $j+k$, $j+2k$, $j+3k$, etc.

Selecting one unit at random in the first k (rather than just selecting the **first** unit, as we might naively think of doing) means that every unit in the list has a chance of selection.

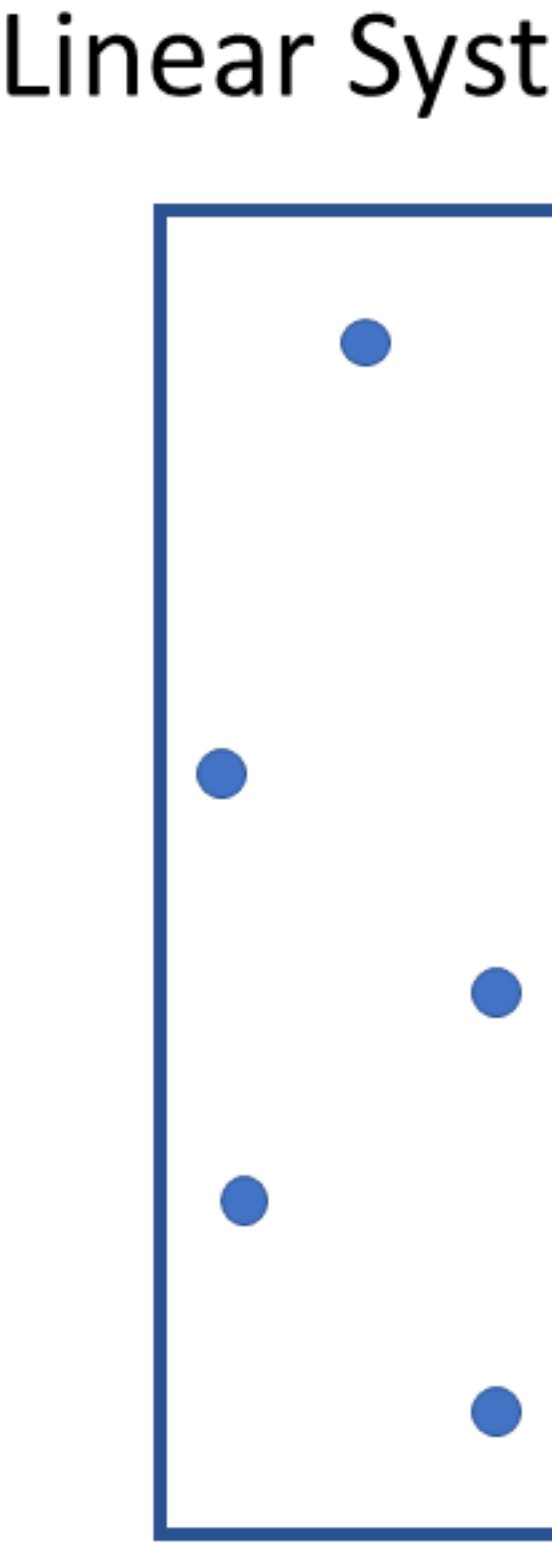
The selection probabilities are

$$\pi_i = \frac{n}{N}$$

and weights are

$$w_i = \frac{N}{n}$$

Data from this design are often analysed as if they were drawn as a SRSWOR, but in fact this is a special case of a one-stage cluster sample (1SC, see below).


Linear Syst

Note that if there are N units but N is not an exact multiple of the skip k, then the sample size will differ depending on which of the first k units we select to initiate sampling. For example if we wanted to select $n = 3$ units by LSRS from a list of $N = 11$ units, then the skip size is $k = \text{floor}(11/3) = \text{floor}(3.67) = 3$. If we start the process by selecting the first unit, we'd end up with the sample of 4 units, $\{1, 4, 7, 10\}$, but if we start by selecting the 4th unit we'd end up with only 3 units, $\{4, 7, 10\}$.

Another common reason to take an LSRS is in household surveys: we're selecting dwellings from a small area, and for reasons of confidentiality (we don't want members of different selected households to be aware that they are both in the same survey), or to prevent collusion between neighbouring selected households, we space the sample out along the street using a LSRS to separate the selected dwellings as much as possible.

An LSRS is also a convenient way to sample if you have a printed list. After choosing the random start location among the first k units you can skip down the list at fixed intervals - something that is conventient and simple to do on a printed list.

Design considerations

- Choose a sample size n
- Consider the ordering of the list and whether units differ systematically through the list

An LSRS has one risk, and that is a periodicity in the frame that you are unaware of. For example if we are selecting apartments from a building to inspect for structural weaknesses, and select every fourth one (after a random start), we had better be sure that there aren't exactly four apartments per floor: otherwise we'll have inadvertently selected a column of apartments stacked one above the other, e.g. all in the north-east corner. We'll miss any systematic issues that occur with the north-west apartments in that case.

Such unknown periodicities are unlikely in most cases, but we should be aware that they can occur.

More importantly if the list is ordered in some way, e.g. by size of location, then a LSRS guarantees a spread across the variable that defines the ordering. This can be highly desirable since we are sure to be sampling across the full diversity of the ordering variable. A good example is sampling passengers seated in an aircraft - if we take an LSRS using a list of passengers ordered by row and seat number we'll be guaranteed to get a spread of business class, premium economy and economy class passengers in our sample.

Example

To select a LSRS of airports, let's order the airports by latitude, and then take a LSRS of $n = 5$ units.

```
bigN <- nrow(airports)
airports <- airports[order(airports$Latitude),]
n <- 5
k <- floor(bigN/n)
cat("Skip size is k =",k,"\n")
```

```
## Skip size is k = 1214
```

```
i0 <- sample(k,1) # first selected unit
cat("Starting sampling at unit",i0,"\n")
```

```
## Starting sampling at unit 305
```

Table 5.3: Sample of airports (SRSWOR)

ID	IATA	Name	City	Country	Latitude	Longitude	weight
5304	VAO	Suavanao Airport	Suavanao	Solomon Islands	-7.58556	158.73100	1214.4
4746	STV	Surat Airport	Surat	India	21.11410	72.74180	1214.4
1714	GIB	Gibraltar Airport	Gibraltar	Gibraltar	36.15120	-5.34966	1214.4
1638	FYN	Fuyun Koktokay Airport	Fuyun	China	46.80417	89.51201	1214.4
5277	UTO	Indian Mountain LRRS Airport	Indian Mountains	United States	65.99280	-153.70399	1214.4

```
n <- (bigN-i0)%/%k + 1 # recalculate the sample size
cat("Actual sample size is",n,"\n")
```

```
## Actual sample size is 5
```

```
samp <- airports[i0 + (0:(n-1))*k,]
samp$weight <- bigN/n
```

```
select.lsrs <- function(df, n) {
  # LSRS
  bigN <- nrow(df)
  k <- floor(bigN/n)
  i0 <- sample(k,1) # first selected unit
  n <- (bigN-i0)%/%k + 1 # recalculate the sample size
  samp <- df[i0 + (0:(n-1))*k,]
  samp$bigN <- bigN
  samp$weight <- bigN/n
  return(samp)
}

samp <- airports |> select.lsrs(5)
```

5.6 Stratified Simple Random Sample (STSRS)

In this design we take our population of N units and break it into H subgroups using our auxiliary data from the frame. We then draw separate samples from **each of the subgroups**.

To draw a stratified sample each population unit needs to be assigned in advance to one, and only one, of these subgroups - which are known as **strata** (singular: **stratum**).

There are N_h units in stratum h, and all of these stratum-level population sizes add to the total population size:

$$N = \sum_{h=1}^{H} N_h$$

Dividing both sides of this equation by N we have the stratum fractions $F_h = N_h/N$: these are the proportions of the **population** that belong to the various strata, and they add up to 1:

$$1 = \sum_{h=1}^{H} \frac{N_h}{N} = \sum_{h=1}^{H} F_h$$

For example, we can assign all residential addresses in New Zealand to exactly one of a defined set of non-overlapping regions, such as regional council areas. Or we might just have

two strata: North Island, South Island (neglecting any addresses on offshore islands). There are $N = 2.4$ million addresses in New Zealand, of which $N_1 = 1.72$ million in the North Island ($F_1 = 0.74 = 74$) and $N_2 = 0.62$ million are in the South Island ($F_2 = 0.26 = 26$).

We then divide or **allocate** our total sample size n to the strata. Stratum h is allocated n_h units, and the total sample size is then the sum of these:

$$n = \sum_{h=1}^{H} n_h$$

We then carry out independent sampling in each of the strata.

The way we choose to define the strata and the way we then choose to allocate the sample across the strata depend on our goals.

Stratified sampling is done for a number of reasons

1. **Estimation Efficiency** - efficiency is the word statisticians use when considering how we can get standard errors which are as **small as possible**.

   In business surveys we often stratify because businesses differ from each other dramatically, particularly in size measures such as turnover or size of workforce. That means a small number of businesses (e.g. Spark, Air New Zealand, Fonterra) have a very large effect on estimates, and they differ dramatically from each other as well. If they happen to be included or happen to be missed from our samples that means we'd get estimates which differ very much between samples. To protect ourselves against this we stratify by business size (using tax data). The largest businesses are in the top stratum, which is small, but a census is always done of those businesses (the top stratum is called a **full coverage stratum**). The smallest businesses are in the lowest stratum, and they are selected with low probability: since they resemble each other more, and one can stand in for another.

   The best stratified designs have strata designed so that the units within each stratum resemble each other strongly (one can stand in for another easily), but where units in different strata differ strongly from each other. For the same sample size as a SRSWOR stratified designs can sometimes achieve much much smaller standard errors.

2. **Subgroup estimates** - we may stratify by region because we want to make estimates for each region separately. In that case we need to ensure that our sample is large enough in each stratum to get reliable estimates. A national SRSWOR runs the risk that one region might happen to miss out on getting much sample.

3. **Operational considerations** - we may stratify by region because that is the easiest way to manage an interview workforce - we may have interviewers based in a number of regional cities, and having them interview in their local areas is the best way to organise that workforce.

Two common methods for allocating the n sample units across the H strata are:

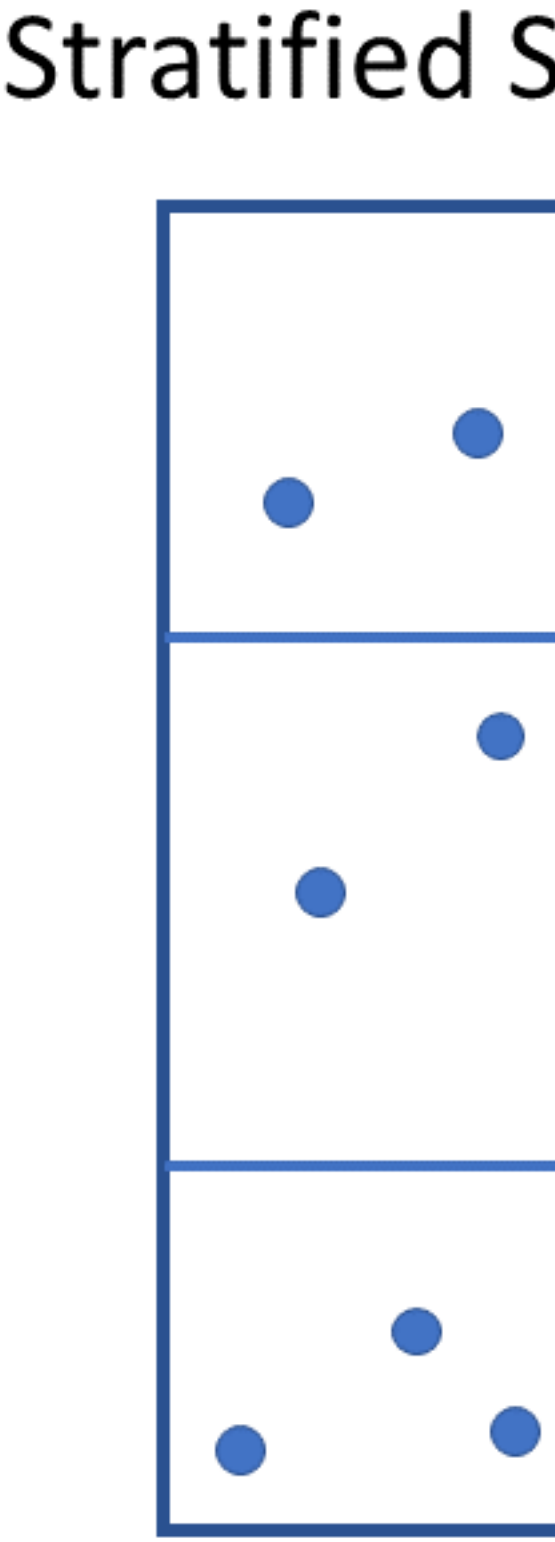
1. **Equal allocation**

   $$n_h = \frac{n}{H}$$

   Every stratum gets the same number of units allocated - independent of the stratum size. That means we get different weights in each stratum:

   $$w_h = \frac{N_h}{n_h} = H\frac{N_h}{n}$$

   This is sensible when we want to have high precision estimates in each stratum separately.


Stratified S

h	Education Region	N_h	F_h
1	Bay of Plenty, Waiariki	185	0.0739
2	Canterbury, Chatham Islands	283	0.1131
3	Hawke's Bay, Tairāwhiti	174	0.0695
4	Nelson, Marlborough, West Coast	127	0.0507
5	Otago, Southland	228	0.0911
6	Tai Tokerau	151	0.0603
7	Taranaki, Whanganui, Manawatū	234	0.0935
8	Tāmaki Herenga Manawa	187	0.0747
9	Tāmaki Herenga Tāngata	197	0.0787
10	Tāmaki Herenga Waka	179	0.0715
11	Waikato	277	0.1107
12	Wellington	281	0.1123

2. **Proportional allocation**

$$n_h = nF_h = n\frac{N_h}{N}$$

Here large strata get a lot of units allocated to them, and small strata get fewer units. The consequence of this design is that the weights of all units are equal, no matter which stratum they come from:

$$w_h = \frac{N_h}{n_h} = \frac{N}{n}$$

This allocation is also called **self weighting**, and the weights are the same as in SRSWOR. (Though note: even though the weights are the same, the standard errors of estimates can differ **very significantly** from those from a SRSWOR.)

Design considerations

- Choose the stratification variable(s) X_i, decide the number of strata H, define the strata using the stratification variable(s), determine the stratum sizes N_h, and stratum fractions $F_h = N_h/N$.
- Choose the overall sample size n and **allocate** it across the strata.

Example

The 2026 directory of NZ schools available from https://www.educationcounts.govt.nz/directories/list-of-nz-schools lists 2503 schools of various types (after removing Schools with zero enrolments or with incomplete data).
Each School is associated with one of $H = 12$ Education Regions, with counts N_h and stratum fractions F_h listed below.

We can select a stratified sample using the following function which implements an independent simple random sample within each stratum with either equal or proportional allocation.

It implements PPSWR sampling within strata as well.

```
select.stsrs <- function(df, n, stratumvar,
                         allocation="equal",
                         nmin=2, # minimum number of units to select per stratum
                         method="SRSWOR", sizevar=NULL) {
  bigN <- nrow(df)
  if(!stratumvar%in%names(df)) {
```

Education Region, h	N_h	n_h	Weight, w_h	Selected Schools
Bay of Plenty, Waiariki	10	185	18.5	482;1170;1711;1758;1787;1796;1808;1904;1929;1984
Canterbury, Chatham Islands	10	283	28.3	354;3304;3329;3348;3357;3389;3450;3553;3572;3586
Hawke's Bay, Tairāwhiti	10	174	17.4	209;1668;1678;2566;2575;2578;2593;2657;2698;2705
Nelson, Marlborough, West Coast	10	127	12.7	292;294;301;3062;3071;3180;3233;3343;3690;6975
Otago, Southland	10	228	22.8	376;399;3737;3753;3824;3827;3943;3982;4017;4050
Tai Tokerau	10	151	15.1	1000;1014;1020;1039;1059;1060;1095;1104;1130;1667
Taranaki, Whanganui, Manawatū	10	234	23.4	2165;2167;2341;2365;2382;2416;2440;2451;2462;4123
Tāmaki Herenga Manawa	10	187	18.7	48;69;460;1210;1370;1376;1382;1385;1484;4133
Tāmaki Herenga Tāngata	10	197	19.7	32;571;767;1106;1223;1309;1394;1432;1492;6977
Tāmaki Herenga Waka	10	179	17.9	96;452;1201;1271;1275;1329;1339;1442;1455;4229
Waikato	10	277	27.7	1744;1753;1777;1790;1835;1844;2001;2008;2033;6941
Wellington	10	281	28.1	133;546;2742;2874;2941;2963;2967;3032;3045;3048

```
    stop("Stratum variable is not present in the data frame")
  }
  stratumproperties <- as.data.frame(table(df[,stratumvar]))
  names(stratumproperties) <- c(stratumvar,"Nh")
  if(allocation=="equal") {
    stratumproperties$nh <- pmin(stratumproperties$Nh,pmax(nmin,
                                 round(n/nrow(stratumproperties))))
  } else if(allocation=="proportional") {
    stratumproperties$nh <- pmin(stratumproperties$Nh,pmax(nmin,
  } else {
    stop("Invalid allocation")
  }
  df <- merge(df,stratumproperties,by=stratumvar)

  if(method=="SRSWOR") {
    samp <- do.call(rbind, by(df, df[,stratumvar],
                              function(dfx) select.srswor(dfx,first(dfx$nh))))
  } else if(method=="PPSWR") {
    samp <- do.call(rbind, by(df, df[,stratumvar],
                              function(dfx) select.ppswr(dfx,first(dfx$nh),sizevar=sizevar)))
  } else {
    stop("Sampling method invalid")
  }
  return(samp)
}
```

SRSWOR with Equal allocation

```
samp <- schools |> select.stsrs(n=120,
                                stratumvar="Education_Region", allocation="equal")
```

SRSWOR with proportional allocation

```
samp <- schools |> select.stsrs(n=120,
                                stratumvar="Education_Region",
                                allocation="proportional", sizevar="Total")
```

PPSWR with Equal Allocation (Schools are selected within strata with probability proportional to the number of enrolled students `Total`.)

Education Region, h	N_h	n_h	Weight, w_h	Selected Schools
Bay of Plenty, Waiariki	9	185	20.55556	145;545;558;707;1645;1745;1879;1921;19
Canterbury, Chatham Islands	14	283	20.21429	328;346;531;3275;3276;3285;3412;3462;3
Hawke's Bay, Tairāwhiti	8	174	21.75000	1669;2547;2574;2588;2590;2685;2702;272
Nelson, Marlborough, West Coast	6	127	21.16667	300;2811;3181;3207;3231;3530
Otago, Southland	11	228	20.72727	384;390;3716;3719;3731;3772;3812;3826;
Tai Tokerau	7	151	21.57143	1029;1030;1075;1085;1116;1666;2104
Taranaki, Whanganui, Manawatū	11	234	21.27273	454;2157;2160;2175;2265;2336;2366;244
Tāmaki Herenga Manawa	9	187	20.77778	78;1190;1224;1240;1282;1411;1416;1515;
Tāmaki Herenga Tāngata	9	197	21.88889	45;1107;1200;1254;1311;1316;1407;1500;
Tāmaki Herenga Waka	9	179	19.88889	748;1248;1277;1332;1369;1418;1472;155
Waikato	13	277	21.30769	130;135;146;158;159;671;1344;1771;1891
Wellington	13	281	21.61538	237;258;1189;1651;2669;2818;2876;2911;

Education Region, h	N_h	n_h	Selected Schools
Bay of Plenty, Waiariki	9	185	120;121;143;152;532;1699*;1730;1761;1766
Canterbury, Chatham Islands	10	283	310;312;327;333;335;348;3289;3407;3517;3570
Hawke's Bay, Tairāwhiti	10	174	216;217;227;230;2445;2564;2581;2628;2677;2688
Nelson, Marlborough, West Coast	8	127	289;296*;1148;1587;3189;3208;3229;3472
Otago, Southland	9	228	369;374;376;400;533;548;586;3967*;4054
Tai Tokerau	9	151	5;8;13*;15;21;1032;1050;1113;1648
Taranaki, Whanganui, Manawatū	10	234	202;203;961;2164;2205;2346;2389;2397;2465;2477
Tāmaki Herenga Manawa	10	187	65;1190;1319;1370;1383;1402;1462;2085;3630;6760
Tāmaki Herenga Tāngata	9	197	46;725;767;1223;1304;1341;1396*;1406;1572
Tāmaki Herenga Waka	10	179	90;91;631;949;1203;1247;1423;1435;1460;1547
Waikato	9	277	126;132*;451;577;615;1693;1946;2021;3612
Wellington	10	281	240;253;255;275;276;478;498;2832;2878;3036

```
samp <- schools |> select.stsrs(n=120,
                                 stratumvar="Education_Region",
                                 allocation="equal",
                                 method="PPSWR", sizevar="Total")
```

(Schools selected multiple times are indicated with a *)

5.7 One stage cluster sampling (1SC)

One stage cluster sampling superficially resembles stratified sampling. As in stratified sampling we divide the population into groups, in this case called **clusters**, assign each member of the population to one and only one group. However rather than sample members from each cluster, we instead draw a sample of clusters (by SRSWOR, or sometimes by PPSWR). We then take a **census** of each selected cluster.

In the Schools example this would could define small regions across the country, select a random sample of those regions, and then carry out a census of schools in each selected region.

Unlike stratified sampling, which is often **more efficient** than simple random sampling (i.e. it has smaller standard errors for the same sample size), cluster sampling is **less efficient**: i.e. its standard errors are worse (larger) than we'd get with a SRS with the same size.

We can see why the standard errors are worse when we consider **why** we take cluster samples:

the reason is predominantly to controlling costs. If we want to do a household survey in a city like Wellington we could list all of the residential addresses, and then draw a simple random sample of those dwellings. That would mean having a sample that is scattered all over the city - and if we have to visit each dwelling to collect the data, then that means a lot of travelling for the interviewers: especially if they have to visit some houses multiple times to make contact with the residents. So it's more **convenient** to select a set of houses on the same street and survey them - we can go from one to the next without much travel time.

The trouble is that people who live near each other tend to resemble each other: they have similar incomes, jobs, family sizes etc. That means that for each additional house you survey in a street you're not really getting as much new information as if you were to survey an additional house somewhere else in the city.

For that reason cluster samples have to have larger sample sizes than SRS to achieve the same accuracy of their estimates.

Note that it makes no sense to cluster if there is no saving to be made regarding cost or some other equivalent operational consideration. If we are doing a phone survey, where no travel at all is involved, then clustering doesn't make any sense.

Our notation becomes slightly more complex here: we have N clusters, labelled $i = 1, \dots, N$. Then in cluster i we have M_i units, labelled $j = 1, \dots, M_i$. There is a total of

$$M = \sum_{i=1}^{N} M_i$$

units in the population.

The selection probability of a unit is simply the probability that the cluster they are a member of is selected. So if we select clusters by SRSWOR then unit j in cluster i has selection probability

$$\pi_{ij} = \frac{n}{N}$$

leading to a weight of

$$w_{ij} = \frac{N}{n}$$

where again we note that n is the number of **clusters** selected, and N is the number of **clusters** in the population.

Design considerations

- We need to decide how to break our populations into clusters, and also the sizes of those clusters
- We need to choose an overall sample size.

Clusters can vary in size, so we even though we can control the number of clusters we select, we can't control the overall sample size: we might select by chance a set of very large clusters, or a set of very small clusters.

In stratified sampling we want to make the strata as different from one another as possible, and to be as internally homogeneous.

In cluster sampling it's the reverse situation: we want our clusters to be internally diverse, and for that diversity to be present as far as possible within every cluster. This is because we're only selecting a few clusters, and they have to stand in for all of the unselected clusters. So they'd better reflect the full population diversity.


One

Territorial Authority, i	M_i	Selected Schools
Hamilton City	59	129;130;131;132;135;136;138;139 ...
Kawerau District	4	651;655;661;1770
Manawatu District	26	197;199;2333;2338;2341;2354;2356;2360 ...
Ruapehu District	22	169;176;183;1617;1845;1847;1961;1982 ...
Timaru District	28	352;354;357;358;359;360;361;1611 ...
Wairoa District	14	214;1616;1668;1669;1675;1676;1677;1678 ...
Waitematā Local Board Area	21	48;50;51;53;62;921;1210;1220 ...
Western Bay of Plenty District	25	117;123;1185;1717;1758;1765;1794;1811 ...
Whau Local Board Area	24	42;78;83;84;781;941;1206;1212 ...
Ōtorohanga District	12	157;1688;1736;1771;1779;1783;1793;1853 ...

Example

There are $N = 87$ Territorial Authorities (TAs) in New Zealand (these are local government districts). Each TA has M_i schools, so the total number of schools in the population is

$$M = \sum_{i=1}^{N} M_i$$

We could select a SRS of $n = 10$ TAs and then do a census of the Schools in each selected TA. Here n is the sample size of clusters, not the sample size of schools. If cluster i contains M_i schools then the total number of schools selected is

$$m = \sum_{i \in \text{sample}} M_i$$

A function to do this selection in R is:

```
select.1sc <- function(df, n, clustervar,
                       method="SRSWOR", sizevar=NULL) {
  bigM <- nrow(df)
  if(!clustervar%in%names(df)) {
    stop("Cluster ID variable is not present in the data frame")
  }
  clusterproperties <- as.data.frame(table(df[,clustervar]))
  names(clusterproperties) <- c(clustervar,"Mi")
  if(method=="SRSWOR") {
     sampcluster <- clusterproperties |>
                       select.srswor(n=n)
  } else if(method=="PPSWR") {
     sampcluster <- clusterproperties |>
                       select.ppswr(n=n, sizevar="Mi")
  } else {
    stop("Sampling method invalid")
  }
  samp <- merge(df,sampcluster,by=clustervar)

  return(samp)
}
```

```
samp <- schools |> select.1sc(n=10, clustervar="Territorial_Authority")
```

5.8 Two stage cluster sampling (2SC)

The obvious drawback of the One Stage cluster design is that we are getting a lot of information from clusters which may be very internally homogeneous. This drives the inefficiency of the design: we're wasting a lot of our sampling effort.

An obvious solution is to select a set of clusters, and then only take a sample of units within those clusters, rather than sampling all of them.

This breaks the sample design into two stages

- **Stage 1** - select a set of clusters, which we refer to as **Primary Sampling Units** (PSUs);
- **Stage 2** - select a set of units within the selected clusters. These units are called **Secondary Sampling Units** (SSUs).

Sampling at both stages can be by any method, though SRSWOR or PPSWR are the most common.

We use the same notation as in one stage clustering: there are N PSUS of which select n, and in PSU i there are M_i SSUs, of which we select m_i. The total number of SSUs is in all of the PSUs in the population is

$$M = \sum_{i=1}^{N} M_i$$

whereas the total sample size of SSUs is

$$m = \sum_{i \in \text{Sample}} m_i$$

where we just sum over the selected PSUs.

If we use SRSWOR and select n PSUs from the N available then the probability of selection of a PSU is

$$\pi_i^{(1)} = \Pr(\text{PSU } i \text{ is selected}) = \frac{n}{N}$$

leading to the first stage weight of

$$w_i^{(1)} = \frac{1}{\pi_i^{(1)}} = \frac{N}{n}$$

If PSU i is selected we then select m_i units (SSUs) out of the M_i available. The probability that SSU j is selected given that PSU i has been selected is

$$\pi_{ij}^{(2)} = \Pr(\text{SSU } j \text{ is selected given that PSU } i \text{ was selected}) = \frac{m_i}{M_i}$$

leading to the second stage weight of

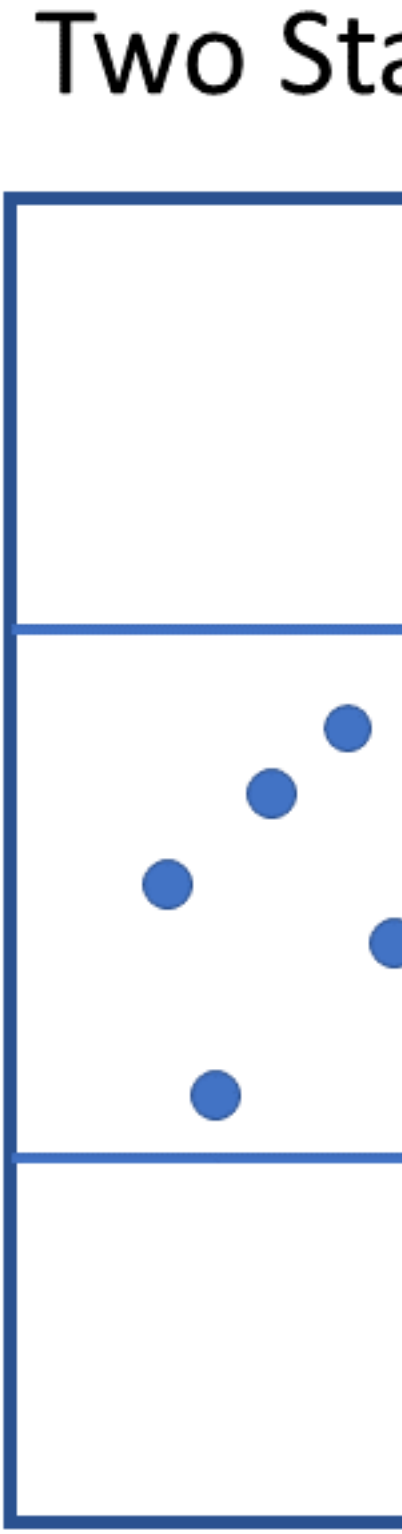
$$w_{ij}^{(2)} = \frac{1}{\pi_{ij}^{(2)}} = \frac{M_i}{m_i}$$

The overall probability that SSU j in PSU i is selected is the product of the first and second stage selection probabilities:

$$\pi_{ij} = \pi_i^{(1)} \pi_{ij}^{(2)} = \frac{n}{N} \times \frac{m_i}{M_i}$$

and its overall weight is the product of the first and second stage weights

$$w_{ij} = w_i^{(1)} w_{ij}^{(2)} = \frac{N}{n} \times \frac{M_i}{m_i}$$


Two Sta

For example I might have $N = 100$ small areas covering Wellington, and I select $n = 10$ of them. Then in one of those selected areas there might be $M_i = 120$ houses, of which I select $m_i = 15$. The first and second stage probabilities are

$$\pi_i^{(1)} = \frac{10}{100} = 0.100 \qquad \text{and} \qquad \pi_{ij}^{(2)} = \frac{15}{120} = 0.125$$

So the chances that a house in PSU i is selected is

$$\pi_{ij} = (0.100)(0.125) = 0.0125$$

with total weight

$$w_{ij} = \frac{100}{10} \times \frac{120}{15} = 10 \times 8 = 80$$

There are two special cases of two stage clustering which reduce to simpler designs

- A first stage census: $n = N$: we sample from every PSU. This is equivalent to **stratified sampling**, where we sample from every stratum.
- A second stage census: $m_i = M_i$: we take a census of all SSUs in each selected cluster. This is equivalent to **one stage clustering**.

Design considerations

We have a lot of decisions to make:

- How many PSUs (clusters) N, and how we define them
- The number of PSUs to select n
- The number of SSUs to select within each selected PSU. Rather than state the sample size m_i for each cluster, this is often specified as the **proportion** of the cluster size we'll select. e.g. we might select 1 in 10 of the SSUs.

Example

Let's return to the schools example. We'll select $n = 10$ TAs, but then select 20% of the schools in each selected TA - although we'll also ensure we have at least 2 schools in each selected cluster.

A function to do this selection in R is:

```
select.2sc <- function(df, n,  # number of first stage clusters to select
                       cfraction, clustervar,
                       mmin=2, # minimum number of 2nd stage clusters to select
                       method1="SRSWOR", sizevar1=NULL,
                       method2="SRSWOR", sizevar2=NULL) {
  bigM <- nrow(df)
  if(!clustervar%in%names(df)) {
    stop("Cluster ID variable is not present in the data frame")
  }
  clusterproperties <- as.data.frame(table(df[,clustervar]))
  bigN <- nrow(clusterproperties)
  names(clusterproperties) <- c(clustervar,"Mi")
  if(method1=="SRSWOR") {
     sampcluster <- clusterproperties |>
                       select.srswor(n=n)
  } else if(method1=="PPSWR") {
     sampcluster <- clusterproperties |>
```

Territorial Authority, i	N	n	M_i	m_i	$w_i^{(1)}$	$w_{ij}^{(2)}$	Weight, w_{ij}	Selected Schools
Masterton District	87	10	20	4	8.70	5.00	43.50	244;1660;2843;2909
Nelson City	87	10	22	4	8.70	5.50	47.85	293;294;3221;4121
Queenstown-Lakes District	87	10	16	3	8.70	5.33	46.40	746;747;958
Tasman District	87	10	35	7	8.70	5.00	43.50	292;296;3180;3203 ...
Timaru District	87	10	28	6	8.70	4.67	40.60	2109;3498;3528;3533 ...
Waimate District	87	10	9	2	8.70	4.50	39.15	3519;3574
Waitematā Local Board Area	87	10	21	4	8.70	5.25	45.67	48;921;1392;2085
Waitomo District	87	10	17	3	8.70	5.67	49.30	1778;2073;2267
Whanganui District	87	10	40	8	8.70	5.00	43.50	422;1585;2361;2412 ...
Whangarei District	87	10	54	11	8.70	4.91	42.71	13;620;1015;1017 ...

```
                        select.ppswr(n=n, sizevar="Mi")
  } else {
    stop("First stage sampling method invalid")
  }
  sampcluster$mi <- pmin(sampcluster$Mi,pmax(mmin,round(cfraction*sampcluster$Mi)))
  samp1 <- merge(df,sampcluster,by=clustervar)
  samp1 <- samp1 |> rename(bigN1=bigN,weight1=weight)
  samp1$n1 <- n

  # Second stage selection
  if(method2=="SRSWOR") {
    samp2 <- do.call(rbind,
                     by(samp1, samp1[,clustervar],
                        function(ds) select.srswor(ds,n=first(ds$mi))
                               ))
  } else if(method2=="PPSWR") {
    samp2 <- do.call(rbind,
                     by(samp2, samp2[,clustervar],
                        function(ds) select.ppswr(ds,n=first(ds$mi),sizevar=sizevar2)
                               ))
  } else {
    stop("Second stage sampling method invalid")
  }
  samp2 <- samp2 |> rename(bigN2=bigN,weight2=weight)
  samp2$n2 <- samp2$mi
  samp2$weight <- samp2$weight1*samp2$weight2

  return(samp2)
}
```

```
samp <- schools |> select.2sc(n=10, cfraction=0.20, mmin=2,
                              clustervar="Territorial_Authority")
```

5.9 Complex multistage designs

The ideas in two stage cluster sampling can be extended to a third stage of sampling (i.e. we may wish to sample within SSUs), and we can also combine cluster sampling with stratified sampling.

For example, the Statistics NZ Household sampling frame is a multistage cluster design, where the sample is selected as follows:

1. NZ is stratified into $H = 12$ regions (e.g. Northland, Auckland, Hawke's Bay etc.);
2. Each stratum (region) is broken into N_h PSUs - small areas that contain (on average) about 100 households.
   A PPSWR of n_h PSUs is taken within each stratum h; (First stage sample.)
3. Within the k^{th} selected PSU, which is of size M_{hk}, a linear systematic random sample is taken of m_{hk} households; (Second stage sample.)
4. Depending on the survey, **all** $A_{hk\ell}$ eligible members of the ℓ^{th} selected household will be surveyed, or a SRSWOR of $a_{hk\ell}$ members may be taken. (Third stage sample.)

The sample size is therefore

$$\text{Sample Size} = \sum_{h=1}^{H} \sum_{k \in s_I} \sum_{\ell \in s_k} a_{hk\ell}$$

and the sample weights of individual j in household ℓ of PSU k in stratum h are

$$w_{hk\ell j} = \frac{N_h}{n_h} \frac{M_{hk}}{m_{hk}} \frac{A_{hk\ell}}{a_{hk\ell}}$$

i.e. these are just the product of the weights at each stage of selection.

Function to draw a stratified two stage cluster sample

```
select.st2sc <- function(df,
                         n,      # total number of 1st stage clusters to select over a
                         stratumvar, allocation="equal",
                         nmin=2, # minimum number of 1st stage clusters to select per
                         cfraction, clustervar,
                         mmin=2, # minimum number of 2nd stage clusters to select
                         method1="SRSWOR", sizevar1=NULL,
                         method2="SRSWOR", sizevar2=NULL) {

  # stratum selection
  bigN <- nrow(df)
  if(!stratumvar%in%names(df)) {
    stop("Stratum variable is not present in the data frame")
  }
  if(!clustervar%in%names(df)) {
    stop("Clustring variable is not present in the data frame")
  }
  xx <- tapply(df[,clustervar], df[,stratumvar], function(x) length(unique(x)))
  stratumproperties <- data.frame(Var1=names(xx),Freq=xx)
  names(stratumproperties) <- c(stratumvar,"Nh")
  rownames(stratumproperties) <- NULL
  if(allocation=="equal") {
    stratumproperties$nh <- pmin(stratumproperties$Nh,pmax(nmin,
                                 round(n/nrow(stratumproperties))))
  } else if(allocation=="proportional") {
    stratumproperties$nh <- pmin(stratumproperties$Nh,pmax(nmin,
                                 round(n*stratumproperties$Nh/sum(stratumproperties$Nh
  } else {
    stop("Invalid allocation")
```

Region, h	Territorial Authority, i	N_h	n_h	M_{hi}	m_{hi}	$w_i^{(1)}$	$w_{ij}^{(2)}$	Weight, w_{ij}
Auckland Region	Aotea/Great Barrier Local Board Area	21	7	3	2	3.00	1.50	4.50
Auckland Region	Devonport-Takapuna Local Board Area	21	7	24	5	3.00	4.80	14.40
Auckland Region	Kaipātiki Local Board Area	21	7	25	5	3.00	5.00	15.00
Auckland Region	Maungakiekie-Tāmaki Local Board Area	21	7	29	6	3.00	4.83	14.50
Auckland Region	Māngere-Ōtāhuhu Local Board Area	21	7	35	7	3.00	5.00	15.00
Auckland Region	Puketāpapa Local Board Area	21	7	21	4	3.00	5.25	15.75
Auckland Region	Waitākere Ranges Local Board Area	21	7	14	3	3.00	4.67	14.00
Bay of Plenty Region	Whakatane District	6	2	32	6	3.00	5.33	16.00
Bay of Plenty Region	Ōpōtiki District	6	2	13	3	3.00	4.33	13.00
Canterbury Region	Ashburton District	9	3	23	5	3.00	4.60	13.80

```
  }
  df <- merge(df,stratumproperties,by=stratumvar)

  samp <- do.call(rbind, by(df, df[,stratumvar],
                            function(dfx) {
                               select.2sc(dfx, n=first(dfx$nh),
                                          cfraction=cfraction,
                                          clustervar=clustervar,
                                          mmin=mmin,
                                          method1=method1, sizevar1=sizevar1,
                                          method2=method2, sizevar2=sizevar2)
                            }))
  return(samp)
}
```

```
samp <- schools |> select.st2sc(n=30, stratumvar="Region", allocation="proportional",
                                cfraction=0.20, clustervar="Territorial_Authority")
```

First 10 of the selected first stage clusters:

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Region and Territorial_Authority.
## i Output is grouped by Region.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Region, Territorial_Authority))` for per-operation
##   grouping (`?dplyr::dplyr_by`) instead.
```

Chapter 6

Sample Designs in R

The method of anlalysis of survey data depends crucially on its sample design. Simple methods of analysis which neglect the sample design can lead to estimates which are severely biased, and standard errors that are incorrect.

In the this and the following Chapter we'll use the `survey` package in R to carry out analyses. We'll use the `schools` data set as our primary example, but also use data on populations and Gross Domestic Product for each country in 2023 from the 'Our World in Data' website.

```
# Census
schools.census <- schools |> select.srswor(n=nrow(schools))
# SRSWOR
schools.srswor <- schools |> select.srswor(n=20)
# PPSWR
schools.ppswr <- schools |> select.ppswr(n=20, sizevar="Total")
# STSRS
schools.stsrs <- schools |> select.stsrs(n=60, stratumvar="Education_Region")
# 1SC
schools.1sc <- schools |> select.1sc(n=10, clustervar="Territorial_Authority")
# 2SC
schools.2sc <- schools |> select.2sc(n=10, cfraction=0.20, mmin=2,
                                     clustervar="Territorial_Authority")
# ST2SC
schools.st2sc <- schools |> select.st2sc(n=30, stratumvar="Region", allocation="proportional",
                                 cfraction=0.20, clustervar="Territorial_Authority")
```

In order to make estimates from sampled data using the `survey` package we need to specify the sample design we're using. We'll do this for the various sample designs covered in the previous chapter.

> Note that in the `survey` package we identify variables with their names without quotes and with a ~ symbol in front. (If you're interested you can use the `survyr` package which is a `dplyr`-like wrapper for `survey` with functions defined so that the ~ isn't necessary.)

We'll define each design, and then use it to estimate the mean size of school rolls in the population (stored in the variable `Total` in the `schools` data frame. (One exception will be the PPSWR example where we'll select a sample of countries using their population size as the size variable.)

The true mean school size is

```
ybar <- mean(schools$Total)
ybar
```

```
## [1] 334.481
```

6.1 Census

The analysis of a census is trivial (at least, if there is no non-response). We can create a census dataset by taking an SRSWOR of size $n = N$.

```
schools.census <- schools |> select.srswor(n=nrow(schools))
```

This function adds the column `bigN` to the dataset, which contains population size N repeated on every row.

Now we can specify the design.

```
schools.census.des <- svydesign(id=~1, data=schools.census, fpc=~bigN)
```

Note that we tell R which column contains the population size using the `fpc` argument. Our data aren't clustered, so the `id` variable (which identifies clusters) is set to 1.

```
svymean(~Total, design=schools.census.des, deff=TRUE)
```

```
##          mean     SE DEff
## Total 334.48   0.00  NaN
```

The estimated mean is 334.48, which is of course exactly equal to the population mean - and note that the standard error is zero: **censuses have no sampling error**. We've measured everything there is to measure so we have no uncertainty.

We've requested `Deff=TRUE`, and have got a Deff value of NaN, which means the Deff was not able to be calculated. We'll discuss what this means later on.

If we had left out the `fpc` command

```
schools.census.des <- svydesign(id=~1, data=schools.census)
```

```
## Warning in svydesign.default(id = ~1, data = schools.census): No weights or
## probabilities supplied, assuming equal probability
```

we get a warning that we can interpret as R assuming an SRSWR from an infinite population.

The estimated mean is the same, but the standard error are now what'd we'd have learned from first year statistics.

```
svymean(~Total, design=schools.census.des, deff=TRUE)
```

```
##           mean       SE DEff
## Total 334.4810   8.4582  Inf
```

```
sy <- sd(schools.census$Total)
n <- nrow(schools.census)
se <- sy/sqrt(n)
```

i.e. the standard error is

$$\mathbf{SE}[\widehat{\bar{Y}}] = \frac{s_y}{\sqrt{n}} = \frac{423.162}{\sqrt{2503}} = 8.458$$

School_Id	Org_Name	Add2_City	Regional_Council	bigN	weig
3193	Hampden Street School	Nelson	Nelson Region	2503	41.716
1519	Sunnyvale School	Waitakere	Auckland Region	2503	41.716
202	Palmerston North Boys' High School	Palmerston North	Manawatū-Whanganui Region	2503	41.716
1234	Bombay School	Bombay	Auckland Region	2503	41.716
3396	Kahikatea Kirkwood Intermediate	Christchurch	Canterbury Region	2503	41.716

Also notice that the `Deff` value has changed to `Inf` (meaning infinity) - again we'll come back to what this means this shortly.

6.2 Simple Random Sample WithOut Replacement (SRSWOR)

Let's draw a sample of $n = 60$ schools from the population by SRSWOR.

```
schools.srswor <- schools |> select.srswor(n=60)
```

Here are the top 5 rows of the 60 in the sample

To specify the design we use the same syntax as above:

```
schools.srswor.des <- svydesign(id=~1, data=schools.srswor, fpc=~bigN)
```

Estimating the mean school size is also just the same:

```
svymean(~Total, design=schools.srswor.des, deff=TRUE)
```

```
##           mean      SE DEff
## Total 347.367  49.582    1
```

This is the mean of a genuine sample now, so we have an estimated mean value that differs from the truth, and the standard error is the gauge of its accuracy.

The standard error comes from the finite population corrected formula:

```
bigN <- first(schools.srswor$bigN) # take the entry in the first row: they're all the same
sy <- sd(schools.srswor$Total)
n <- nrow(schools.srswor)
se <- sqrt(1-n/bigN)*sy/sqrt(n)
```

$$\mathbf{SE}[\widehat{\bar{Y}}] = \sqrt{1-\frac{n}{N}}\frac{s_y}{\sqrt{n}} = \sqrt{1-\frac{60}{2503}}\frac{388.745}{\sqrt{60}} = 49.582$$

Compare this to the simple SE, which is just slightly bigger

```
se <- sy/sqrt(n)
```

$$\mathbf{SE}[\widehat{\bar{Y}}] = \frac{s_y}{\sqrt{n}} = \frac{388.745}{\sqrt{60}} = 50.187$$

This is what we get if we omit the `fpc`

```
schools.srswor.des <- svydesign(id=~1, data=schools.srswor)
```

```
## Warning in svydesign.default(id = ~1, data = schools.srswor): No weights or
## probabilities supplied, assuming equal probability
```

Country	Code	GovtSpend	GDP	Region	Population	GDPPerCapita
Albania	ALB	1863822443	1.496298e+10	Europe	2811660	5321.759
Algeria	DZA	44321604142	2.151440e+11	Africa	46164222	4660.405
Angola	AGO	15741572839	1.032363e+11	Africa	36749909	2809.157
Argentina	ARG	111098899281	5.916847e+11	South America	45538402	12993.093
Armenia	ARM	1598415044	1.540616e+10	Asia	2943398	5234.142
Australia	AUS	385802798739	1.649425e+12	Oceania	26451126	62357.456
Austria	AUT	84758230499	4.246078e+11	Europe	9130434	46504.669
Azerbaijan	AZE	NA	5.755883e+10	Asia	10318209	5578.374
Bahrain	BHR	6908975351	3.996681e+10	Asia	1569674	25461.855
Bangladesh	BGD	18377804014	3.232800e+11	Asia	171466986	1885.377

```
svymean(~Total, design=schools.srswor.des, deff=TRUE)
```

```
##          mean      SE DEff
## Total 347.367  50.187  Inf
```

6.3 Probability Proportional to Size With Replacement (PPSWR)

We have a dataset from 'Our World in Data' giving population sizes, Gross Domestic Product and military expenditure in 2023 (all in US dollars). The first 10 rows are below:

We can select a PPSWR using population as the size variable

```
mil.ppswr <- mil |> select.ppswr(n=30, sizevar="Population")
```

The `weight` column here combines the weight over the number of times a unit is selected

```
mil.ppswr.des <- svydesign(id=~1, data=mil.ppswr, weight=~weight)
```

The mean military expenditure per country is

```
svymean(~MilSpend, design=mil.ppswr.des, deff=TRUE)
```

```
##                mean         SE   DEff
## MilSpend 7.4657e+10 3.4650e+10 1.0362
```

though more interesting is the **total** of this variable, to see how much the world is spending.

```
svytotal(~MilSpend, design=mil.ppswr.des, deff=TRUE)
```

```
##               total         SE  DEff
## MilSpend 1.1199e+13 5.3452e+12 1.096
```

> The estimation of the **total** of a variable is only something that makes sense in a finite population, and is a type of estimate that is rarely found anywhere outside sample survey analysis.
>
> It is frequently of interest in Official (i.e. government) statistics that we want to estimate the total number of people with a certain characteristic, or the total amount of a commodity being exported, or total export earnings.

Note again that in a with replacement design we can't apply the finite population correction.

6.4 Stratified Sample

Recall that in this design we specify a set of groups called strata, and each population unit belongs to one of these strata.

Selecting a sample of $n = 60$ schools by equal allocation across the 12 Education Regions:

```
schools.stsrs <- schools |> select.stsrs(n=60, stratumvar="Education_Region")
```

Specify this design

```
schools.stsrs.des <- svydesign(id=~1, strata=~Education_Region, fpc=~Nh,
                               data=schools.stsrs,
                               survey.lonely.psu="remove")
```

Note that we've included the option `survey.lonely.psu="remove"` to mean that the estimation of standard errors doesn't fail if a stratum only has a single PSU.

We can now return to estimating the mean school size

```
svymean(~Total, design=schools.stsrs.des, deff=TRUE)
```

```
##          mean      SE   DEff
## Total 337.235  39.972 0.6816
```

6.4.1 Design Effect

Now it's finally time to talk about the **Design Effect**, or the Deff for short.

The Deff of an estimator $\widehat{T}$ of a population characteristic T in a particular sample design is defined to be

$$\textbf{Deff}[\widehat{T}_{\text{design}}] = \frac{\textbf{Var}[\widehat{T}_{\text{design}}]}{\textbf{Var}[\widehat{T}_{\text{SRSWOR}}]} = \left(\frac{\textbf{SE}[\widehat{T}_{\text{design}}]}{\textbf{SE}[\widehat{T}_{\text{SRSWOR}}]}\right)^2$$

where both estimators are based on samples of the same size.

It tells us how the standard error differs between our design of interest and a SRSWOR of the same sample size.

Note that

- We have a different Deff for every estimate we make from a survey.
- A Deff<1 means that the Standard Errors are smaller than what we'd get in SRSWOR. A small Deff indicates a **more efficient** design.
- A Deff of 1 means the design is no better than SRSWOR - and of course (look back above) the Deffs of all estimates in SRSWOR are 1 by definition.
- If we're dealing with a **census**, then the Standard Error is zero, and in any case there's no difference between designs, so the Deff isn't defined. (That's why it was `NaN` or `Inf` in our census examples.)
- **Stratified designs** can achieve Deffs much much less than 1 if their strata are well defined to be informative about the estimates of interest. e.g. in business surveys where the strata are defined by business size we can get very good Deffs.
- **Cluster designs** often have Deffs greater than 1, which show them to be significantly less efficient than SRSWOR.

The Deff values we get from R are approximations, and aren't very reliable when sample sizes are small. But do notice that we have a Deff<1 here for the stratified sample of $n = 60$ units, and the SE is lower than that achieved in the SRSWOR of $n = 60$ units above.

6.5 One stage cluster design (1SC)

A one stage cluster design takes $n = 10$ Territorial Authority areas (TAs) at random and selects all schools in each selected TA.

```
schools.1sc <- schools |> select.1sc(n=10, clustervar="Territorial_Authority")
```

We need a column `bigN` which gives the number of TAs in the population.

```
schools.1sc.des <- svydesign(id=~Territorial_Authority, data=schools.1sc,
                             fpc=~bigN)
```

```
svymean(~Total, design=schools.1sc.des, deff=TRUE)
```

```
##          mean      SE   DEff
## Total 287.197  43.829 5.7634
```

The Deff of this one stage cluster sample is bad (much larger than 1) - this is due to strong differences between TAs in the mean school size. The more the clusters differ from each other on the characteristic of interest, the worse the Deff gets.

6.6 Two stage cluster design (2SC)

A two stage cluster design takes $n = 10$ Territorial Authority areas (TAs) at random and selects 20% of the schools in each selected TA.

```
schools.2sc <- schools |> select.2sc(n=10, cfraction=0.20, mmin=2,
                                     clustervar="Territorial_Authority")
```

We need a column `bigN1` which gives the number of TAs in the population, and `bigN2` which is the number of schools in each TA. We also need to specify names of unique identifiers of the first stage units (the PSUs: these are Territorial Authorities) and the second stage units (the SSUs: these are Schools).

```
schools.2sc.des <- svydesign(id=~Territorial_Authority+School_Id, data=schools.2sc,
                             fpc=~bigN1+bigN2,
                             survey.lonely.psu="remove")
```

```
svymean(~Total, design=schools.2sc.des, deff=TRUE)
```

```
##          mean      SE   DEff
## Total 486.981  84.261 2.1739
```

This design specification has the fpc specified at both stages of selection, and assumes SRSWOR at both stages. The `survey` package currently only allows for the analysis of two stage cluster designs with PPSWR at the first stage of selection. We'll stick to considering designs with SRSWOR at both stages.

For the record, this is how to select a two stage cluster sample with PPSWR at the first stage, and how to specify its sample design in R:

```
schools.2sc.ppswr <- schools |> select.2sc(n=10,
                                           method1="PPSWR", sizevar1="Total",
                                           cfraction=0.20, mmin=2,
                                     clustervar="Territorial_Authority")
schools.2sc.ppswr.des <- svydesign(id=~Territorial_Authority+School_Id, data=schools.2s
                                   weight=~weight,
```

```
                                    survey.lonely.psu="remove")
svymean(~Total, design=schools.2sc.ppswr.des, deff=TRUE)
```

```
##          mean      SE   DEff
## Total 437.854  80.806 1.6635
```

6.7 Stratified two stage cluster design

Finally here is a stratified two stage sample of schools within TA's with Regions. The Regions are strata, the TAs are clusters within the Regions, and then 20% of schools are sampled in the selected TAs.

```
schools.st2sc <- schools |> select.st2sc(n=30, stratumvar="Region", allocation="proportional",
                                         cfraction=0.20, clustervar="Territorial_Authority")
```

The design is the same as the two stage cluster sample, except we add the stratum specification.

```
schools.st2sc.des <- svydesign(id=~Territorial_Authority+School_Id, strata=~Region,
                               data=schools.st2sc,
                               fpc=~bigN1+bigN2,
                               survey.lonely.psu="remove")
```

```
svymean(~Total, design=schools.st2sc.des, deff=TRUE)
```

```
##          mean      SE  DEff
## Total 295.326  25.898 1.248
```

Chapter 7

Survey Analysis

With the ability to load a survey dataset and specify its survey design we're now ready to do some analysis.

We'll use the `api` dataset which comes with the survey package: it's a set of observations of the Academic Performance Index (API) on all California Schools with at least 100 students in the years 1999 and 2000.

```
data(api)
```

In particular we'll use the `apiclus2` dataset, which is a two stage cluster sample of 126 schools drawn from the population of 6194. The top 10 rows of the data set and some of the key variables are shown below.

```
apiclus2$stype <- factor(apiclus2$stype, levels=c("E","M","H"))
levels(apiclus2$stype) <- c("Elementary","Middle","High")

apiclus2 |>
  slice_head(n=10) |>
  select(cds, stype, name, dnum, cnum, api99, api00) |>
  kbl() |>
  kable_styling()
```

The full set of variables in the dataset is:

The `apiclus2` dataset was drawn by taking a sample of districts (labelled by district number

cds	stype	name	dnum	cnum	api99	api00
31667796031017	Elementary	Alta-Dutch Flat	15	30	785	821
55751846054837	Elementary	Tenaya Elementa	63	54	718	773
41688746043517	Elementary	Panorama Elemen	83	40	632	600
41688746043509	Middle	Lipman Middle	83	40	740	740
41688746043491	Elementary	Brisbane Elemen	83	40	711	716
40687266042998	Elementary	Cayucos Element	117	39	779	811
20651936023881	Elementary	Fuller (Merle L	132	19	432	472
20651936023931	Elementary	Fairmead Elemen	132	19	494	520
20651936023907	Middle	Wilson Elementa	132	19	589	568
06615980631259	High	Colusa High	152	5	585	591

Variable	Description
cds	Unique identifier
stype	Elementary/Middle/High School
name	School name (15 characters)
sname	School name (40 characters)
snum	School number
dname	District name
dnum	District number
cname	County name
cnum	County number
flag	reason for missing data
pcttest	percentage of students tested
api00	API in 2000
api99	API in 1999
target	target for change in API
growth	Change in API
sch.wide	Met school-wide growth target?
comp.imp	Met Comparable Improvement target
both	Met both targets
awards	Eligible for awards program
meals	Percentage of students eligible for subsidized meals
ell	'English Language Learners' (percent)
yr.rnd	Year-round school
mobility	percentage of students for whom this is the first year at the school
acs.k3	average class size years K-3
acs.46	average class size years 4-6
acs.core	Number of core academic courses
pct.resp	percent where parental education level is known
not.hsg	percent parents not high-school graduates
hsg	percent parents who are high-school graduates
some.col	percent parents with some college
col.grad	percent parents with college degree
grad.sch	percent parents with postgraduate education
avg.ed	average parental education level
full	percent fully qualified teachers
emer	percent teachers with emergency qualifications
enroll	number of students enrolled
api.stu	number of students tested.

dnum), and then a sample of schools within districts (schools are labelled by their school number snum).
The column fpc1 contains the number of districts (757), and the column fpc2 contains the number of schools in each district (which varies by district).

We can specify this two stage cluster design using

```
apiclus2.des <- svydesign(ids=~dnum+snum, fpc=~fpc1+fpc2, data=apiclus2)
```

The summary() function tells us the basics of the design we've just specified:

```
summary(apiclus2.des)
```

```
## 2 - level Cluster Sampling design
## With (40, 126) clusters.
## svydesign(ids = ~dnum + snum, fpc = ~fpc1 + fpc2, data = apiclus2)
## Probabilities:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## 0.003669 0.037743 0.052840 0.042390 0.052840 0.052840
## Population size (PSUs): 757
## Data variables:
##  [1] "cds"      "stype"    "name"     "sname"    "snum"     "dname"
##  [7] "dnum"     "cname"    "cnum"     "flag"     "pcttest"  "api00"
## [13] "api99"    "target"   "growth"   "sch.wide" "comp.imp" "both"
## [19] "awards"   "meals"    "ell"      "yr.rnd"   "mobility" "acs.k3"
## [25] "acs.46"   "acs.core" "pct.resp" "not.hsg"  "hsg"      "some.col"
## [31] "col.grad" "grad.sch" "avg.ed"   "full"     "emer"     "enroll"
## [37] "api.stu"  "pw"       "fpc1"     "fpc2"
```

We can see that from the summary that $n = 40$ districts were sampled (out of $N = 757$). Let's count the number of schools sampled per district. We'll just display the top 15 largest districts:

```
apiclus2 |>
  group_by(dnum,dname) |>
  summarise(Mi=first(fpc2),mi=n()) |>
  ungroup() |>
  arrange(desc(Mi)) |>
  slice_head(n=15) |>
  kbl(col.names=c("dnum","District Name","$M_i$","$m_i$"),escape=FALSE) |>
  kable_styling(full_width=FALSE)
```

After the 10 largest districts a census of schools is taken in each selected district $m_i = M_i$

7.1 Continous variables

Consider the key outcome variable 'api00", the API score in the year 2000 for each school.

Display

The basic workhorse for the display of numerical variables is the histogram

```
svyhist(~api00, design=apiclus2.des, prob=FALSE,
        xlab="API Score (Year 2000)",
        main="Weighted distribution of API (2000) scores")
```

dnum	District Name	M_i	m_i
620	Sacramento City Unified	72	5
570	Pomona Unified	36	5
575	Poway Unified	28	5
638	San Lorenzo Unified	14	5
200	El Centro Elementary	11	5
731	Sylvan Union Elementary	9	5
596	Rincon Valley Union Elem	7	5
639	San Lorenzo Valley Unif	7	5
781	Walnut Creek Elementary	6	5
295	Healdsburg Unified	5	5
403	Los Gatos Union Elem	5	5
480	Natomas Unified	5	5
534	Palo Verde Unified	5	5
549	Piedmont City Unified	5	5
173	Del Mar Union Elementary	4	4

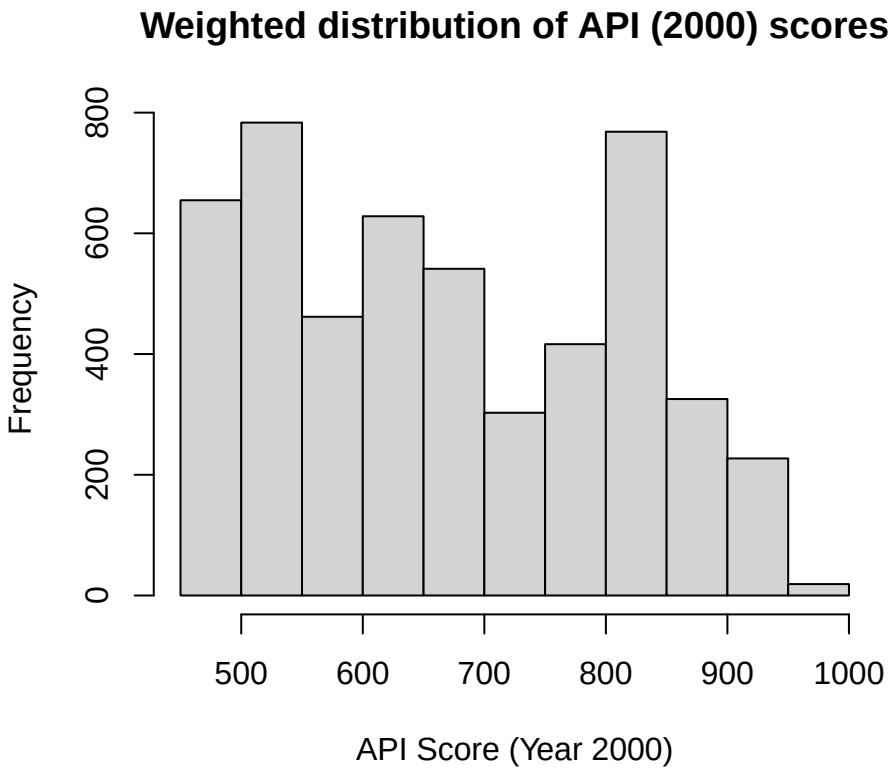


Figure 7.1: Weighted histogram of API (2000) scores

Note that this histogram differs strongly from the simple histogram of the sample values:

```
hist(apiclus2$api00,
        xlab="API Score (Year 2000)",
        main="Unweighted distribution of API (2000) scores")
```

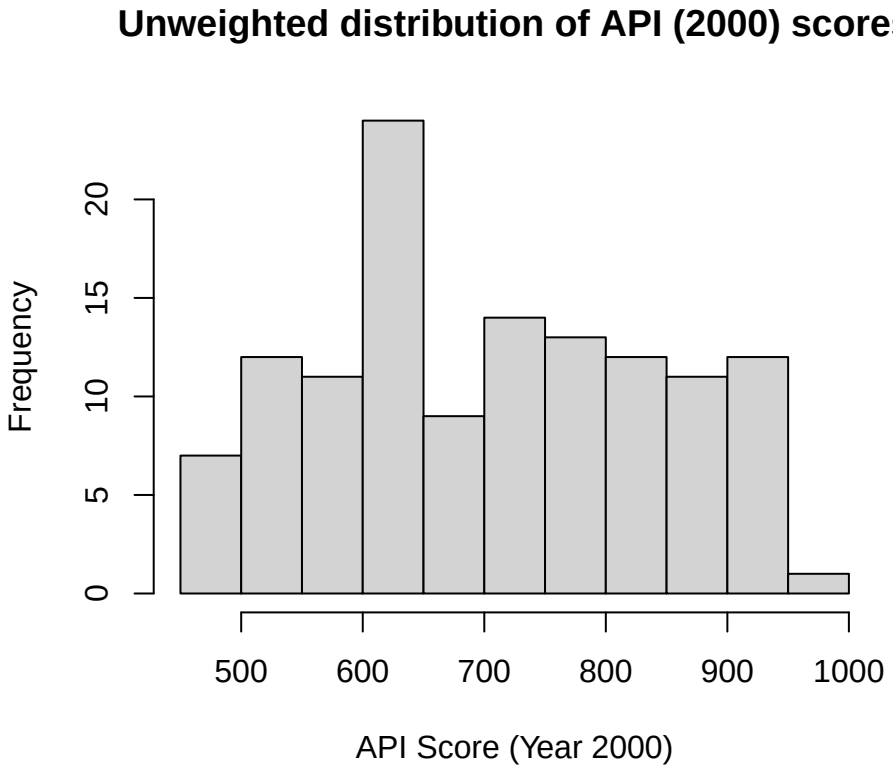


Figure 7.2: Unweighted histogram of API (2000) sample data

a warning to us that with survey data we can go badly astray if we ignore the sample design.

Boxplots are an alternative to histograms:

```
svyboxplot(api00~1, design=apiclus2.des,
           ylab="API (2000)", main="API (2000) Scores")
```

... especially if we want to display variables divided by a category:

```
svyboxplot(api00~stype, design=apiclus2.des,
           xlab="School Type",
           ylab="API (2000)", main="API (2000) Scores")
```

If we have two continuous variables we can plot them as a scatterplot, showing the weights of each observation:

```
svyplot(api00~api99, design=apiclus2.des, style="bubble",
        xlab="API (1999)", ylab="API (2000)")
```

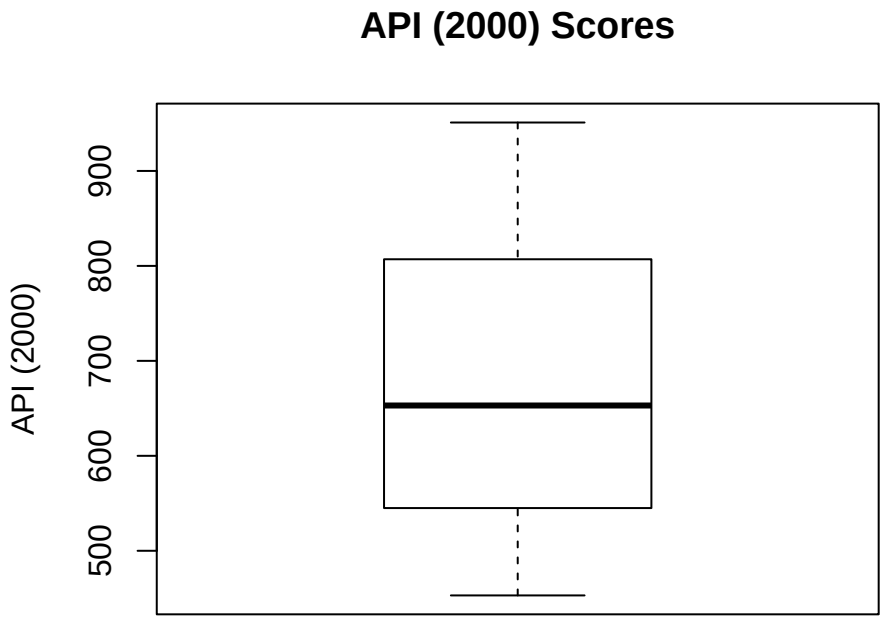


Figure 7.3: Distribution of API (2000) sample data

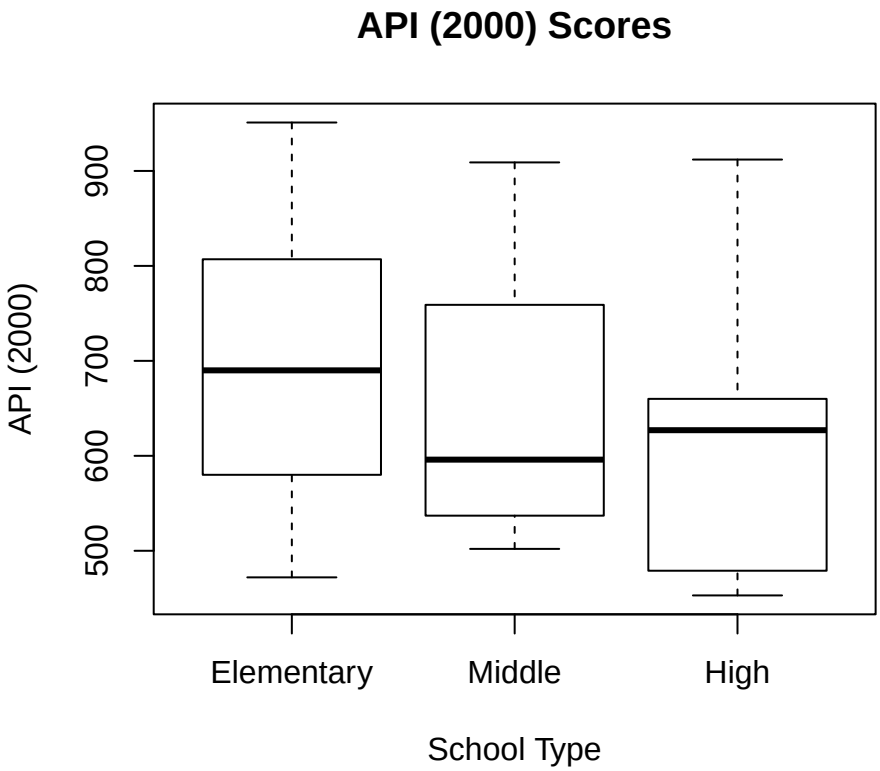


Figure 7.4: Distribution of API (2000) sample data

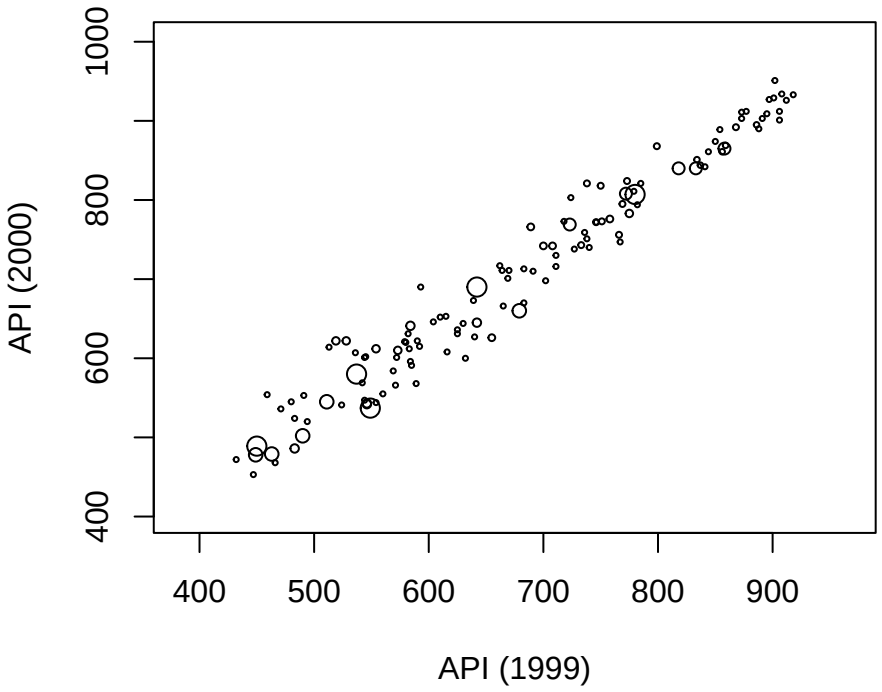


Figure 7.5: Distribution of API scores in 1999 and 2000

Estimation

We've already seen that we can estimate means using the `svymean()` function.

```
svymean(~api00, design=apiclus2.des)
```

```
##         mean     SE
## api00 670.81 30.099
```

The `jtools` package allows us to estimate the population standard deviation too

```
library(jtools)
svysd(~api00, design=apiclus2.des)
```

```
##       std. dev.
## api00    136.83
```

(**Pause a minute** and remind yourself of the difference between a standard deviation and a standard error.)

We can estimates means for a set of separate variables, using a perhaps uninintuitive notation:

```
svymean(~api99+api00, design=apiclus2.des)
```

```
##         mean     SE
## api99 645.03 29.711
## api00 670.81 30.099
```

or for a variable split by categories by wrapping `svymean()` up inside a call to `svyby()`

```
svyby(~api00, by=~stype, svymean, design=apiclus2.des)
```

```
##                 stype    api00       se
## Elementary Elementary 692.8104 29.92660
## Middle         Middle 642.3520 45.09132
## High             High 598.3407 17.69417
```

A **dotchart** is a nice way of showing the estimated means and their confidence intervals:

```
ss <- svyby(~api00, by=~stype, svymean, design=apiclus2.des)
sc <- confint(ss)
ns <- nrow(sc)
dotchart(ss, xlim=range(sc), xlab="Mean API (2000) Score")
arrows(sc[,1], 1:ns, sc[,2], 1:ns, angle=90, code=3, length=0.1)
```

If we want to test statistical hypotheses about differences between groups one way is to use the regression framework

```
summary(svyglm(api00~stype, design=apiclus2.des))
```

```
##
## Call:
## svyglm(formula = api00 ~ stype, design = apiclus2.des)
##
## Survey design:
## svydesign(ids = ~dnum + snum, fpc = ~fpc1 + fpc2, data = apiclus2)
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   692.81      29.93  23.150  < 2e-16 ***
```

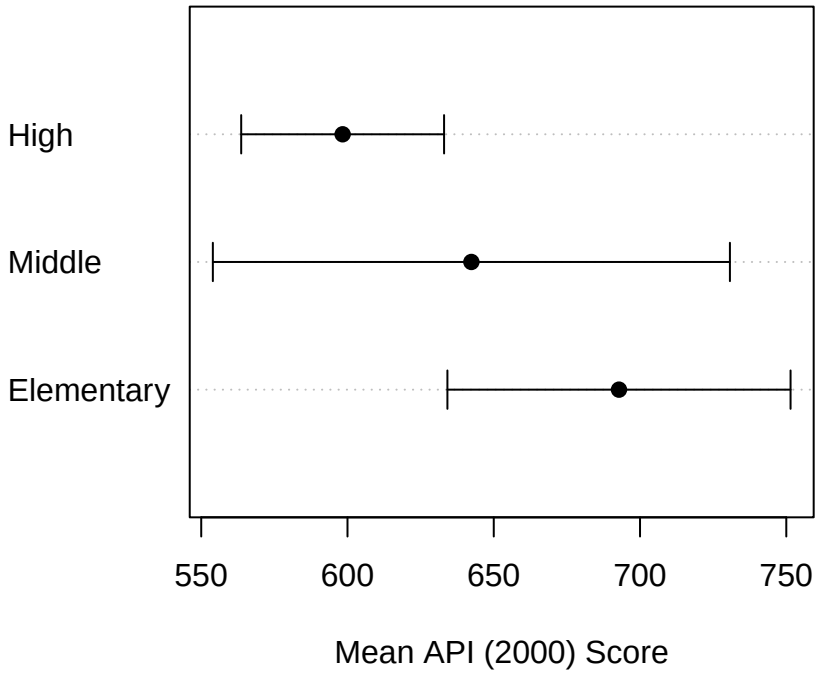


Figure 7.6: Mean API (2000) scores by School Type

```
## stypeMiddle   -50.46      25.30  -1.994  0.05354 .
## stypeHigh     -94.47      28.21  -3.348  0.00188 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 17528.44)
##
## Number of Fisher Scoring iterations: 2
```

Here we can see that with Elementary as the reference level we get the same estimate of the mean API reported as the `(Intercept)`. We further see that there is no significant difference between Middle and Elementary Schools in mean API, but the mean API is significantly lower at High Schools compared to Elementary Schools.

Regressions take of course take lots of variables into account, and we can apply familiar regression techniques to investigating conditional associations between predictors and a continuous outcome.

For example we can test whether the percentage of children at a school who have free school meals (i.e. the `meals` variable) associates with the API score:

```
summary(svyglm(api00~stype+meals, design=apiclus2.des))
```

```
##
## Call:
## svyglm(formula = api00 ~ stype + meals, design = apiclus2.des)
##
## Survey design:
## svydesign(ids = ~dnum + snum, fpc = ~fpc1 + fpc2, data = apiclus2)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  874.8638    16.4677  53.126  < 2e-16 ***
## stypeMiddle  -60.9517    20.6964  -2.945  0.00563 **
## stypeHigh   -169.2128    20.2618  -8.351 6.06e-10 ***
## meals         -3.2367     0.2888 -11.209 2.70e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 5560.106)
##
## Number of Fisher Scoring iterations: 2
```

… and there is indeed a strong negative association: the higher the `meals` value the lower the API score.

We can estimate **totals** using the `svytotal()` function - for example we can estimate the total number of students enrolled at all schools in California:

```
svytotal(~enroll, design=apiclus2.des, na.rm=TRUE)
```

```
##          total     SE
## enroll 2639273 799638
```

Although note that we have had to remove some schools with unknown `enroll` values by stating `na.rm=TRUE`, and these forced removals likely mean that our estimate of the total is an underestimate.

Confidence intervals can be calculated using the `confint()` function

```
confint(svymean(~api99+api00, design=apiclus2.des), level=0.95)
```

```
##          2.5 %   97.5 %
## api99 586.8009 703.2670
## api00 611.8188 729.8048
```

```
confint(svyby(~api00, by=~stype, svymean, design=apiclus2.des))
```

```
##                2.5 %   97.5 %
## Elementary 634.1553 751.4655
## Middle     553.9746 730.7294
## High       563.6607 633.0206
```

The standard errors can be directly extracted using `SE()`

```
SE(svyby(~api00, by=~stype, svymean, design=apiclus2.des))
```

```
## [1] 29.92660 45.09132 17.69417
```

7.1.1 Relative Sampling Errors (RSEs)

The **relative standard errors**, also known as **relative sampling errors** (RSEs) can be extracted by `cv()`:

```
cv(svyby(~api00, by=~stype, svymean, design=apiclus2.des))
```

```
## Elementary     Middle       High
## 0.04319595 0.07019721 0.02957206
```

RSEs are a useful measure of quality of estimates of quantities that are strictly positive, such as the API score.

If we learned that the SE of an estimate (e.g. the mean of `api00`) was 30.1 we wouldn't be able to assess whether it was a good estimate or not without knowing that the value of the estimate was 670.81. A good way to assess the quality of an estimate is the SE of an estimate $\widehat{T}$ as a proportion of the estimate $\widehat{T}$:

$$\mathbf{RSE}[\widehat{T}] = \frac{\mathbf{SE}[\widehat{T}]}{\widehat{T}}$$

For overall mean `api00` the RSE is

$$\mathbf{RSE}[\widehat{\widehat{Y}}] = \frac{30.1}{670.81} = 0.045$$

This is a highly precise estimate: the SE is only 4.5% of the value of the estimate.

Good RSEs are only a few percent at most. Poor RSEs are larger than 10%, and anything bigger than 20% isn't really publishable.

In general a standard deviation divided by a mean goes by the term **coefficient of variation** (CV), which is why the R function is called `cv()`.

```
cv(svymean(~api00, design=apiclus2.des))
```

```
##            api00
## api00 0.04486956
```

7.2 Categorical Variables

The workhorse display for categorical variables is the barplot, showing **proportions** in categories. Proportions are first calculated using the `svymean()` function:

```
barplot(svymean(~stype, design=apiclus2.des),
        names=levels(apiclus2$stype),
        xlab="School Type", ylab="Proportion")
```

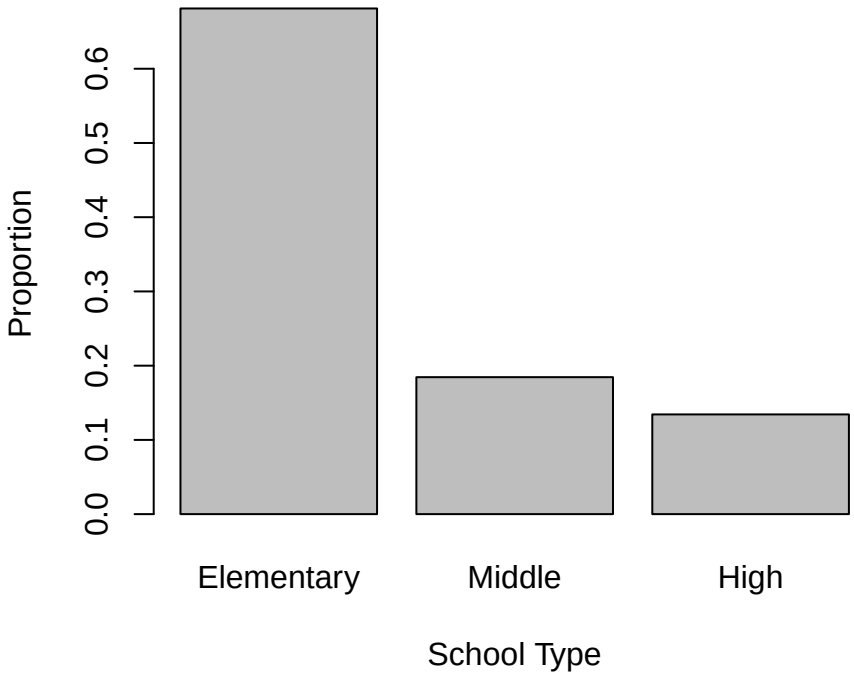


Figure 7.7: Estimates of the proportions of Schools of each type

```
svymean(~stype, design=apiclus2.des)
```

```
##                    mean     SE
## stypeElementary 0.68118 0.0556
## stypeMiddle     0.18450 0.0222
## stypeHigh       0.13432 0.0567
```

It's worth pausing to recognise that a proportion is a mean of a special type of variable called an **indicator variable**: it takes the value 1 if the observation is a member of specific category and 0 if not. Add up all the ones and zeroes and you get the number of observations in the category, and divide by the number of observations you get a proportion.

$$p = \frac{1+0+1+1+0+0+....}{n} = \frac{1}{n}\sum_{i=1}^{n} Y_i$$

where Y_i is an indicator variable.

Table 7.1: (#tab:tabci.symmetric)Symmetric confidence interval estimates

	Estimate	2.5%	97.5%
stypeElementary	0.681	0.572	0.790
stypeMiddle	0.185	0.141	0.228
stypeHigh	0.134	0.023	0.246

With sample data we calculate **weighted sums** though:

$$\hat{p} = \frac{\sum_{i=1}^{n} w_i Y_i}{\sum_{i=1}^{n} w_i} = \frac{\widehat{Y}}{\widehat{N}}$$

where the top line of this fraction is $\widehat{Y}$, our estimate of the number of individuals (in a finite population) who belong to the category of interest. The bottom line is $\widehat{N}$, the estimated size of the population. $\widehat{N}$ is exact in some designs (e.g. SRSWOR, or Stratified SRSWOR), but is approximate in other designs (especially cluster designs).

These total estimates within categories can be estimated using the `svytotal()` function.

```
svytotal(~stype, design=apiclus2.des)
```

```
##                    total      SE
## stypeElementary 3493.56 1119.75
## stypeMiddle      946.25  311.81
## stypeHigh        688.87  289.35
```

When creating barplots of estimated proportions and/or totals we really should indicate the uncertainties in our estimates, and show confidence intervals on the bars. We can use the standard errors provided by `svymean()` and `svytotal()`: the function `confint()` will provide these:

```
confint(svytotal(~stype, design=apiclus2.des))
```

```
##                       2.5 %   97.5 %
## stypeElementary 1298.8892 5688.221
## stypeMiddle      335.1077 1557.392
## stypeHigh        121.7472 1255.993
```

These confidence intervals are symmetric about the point estimates:

```
ss <- svymean(~stype, design=apiclus2.des)
ci <- confint(ss)
bp <- barplot(ss, xlab="School Type", ylab="Estimated Size",
              names=levels(apiclus2$stype),
              ylim=c(0,max(ci)))
arrows(bp, ci[,1], bp, ci[,2], angle=90, length=0.2, code=3)
```

```
names(ss) <- levels(apiclus2$stype)
cbind(as.numeric(ss),ci) |>
  kbl(col.names=c("Estimate","2.5%","97.5%"),
      digits=3,
      caption="Symmetric confidence interval estimates") |>
  kable_styling(full_width=FALSE)
```

These symmetric intervals are actually somewhat unrealistic, because `confint()` doesn't

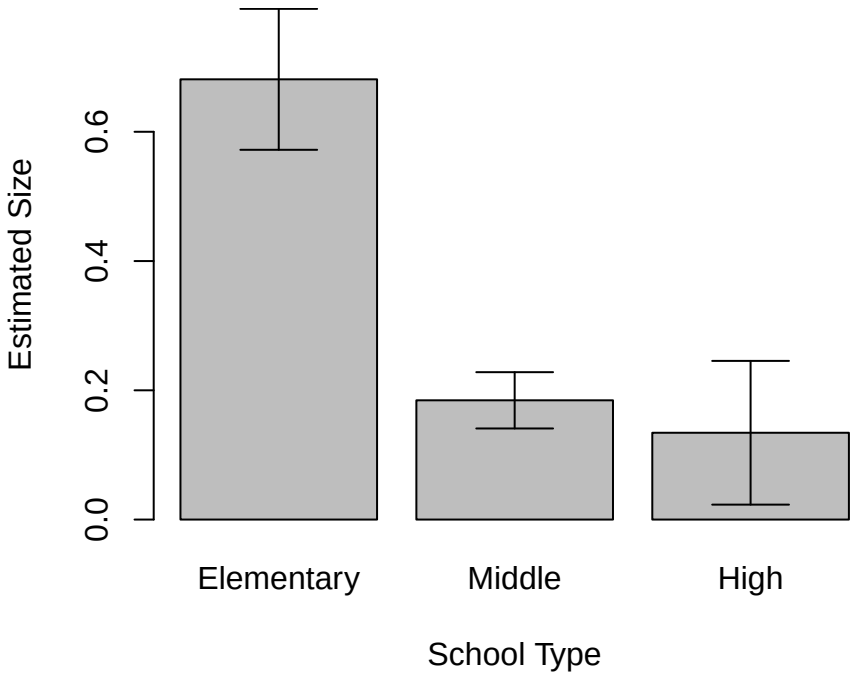


Figure 7.8: Estimated proportions of schools by school type - with symmetric confidence intervals

Table 7.2: (#tab:tabci.asymmetric)Asymmetric confidence interval estimates

	Estimate	2.5%	97.5%
Elementary	0.681	0.560	0.782
Middle	0.185	0.144	0.234
High	0.134	0.055	0.294

know it's dealing with indicator variables, and doesn't take into account the fact that proportions and totals can't go negative.

A better method of constructing confidence intervals is the special function `svyciprop()`:

```
svyciprop(~I(stype=="Elementary"), design=apiclus2.des)
```

```
##                                    2.5% 97.5%
## I(stype == "Elementary") 0.681 0.560 0.782
```

Note that we've had to create an indicator variable here using the `I()` function for just one level in order to calculate its confidence interval.

To get a confidence interval proportion for every level of a factor it's somewhat fiddly with the raw `survey` package:

```
ss <- svymean(~stype, design=apiclus2.des)
ci <- t(sapply(levels(apiclus2$stype),
       function(stypelevel) {
          form <- eval(parse(text=paste0("formula(~I(stype==\"",stypelevel,"\"))")))
          confint(svyciprop(form, design=apiclus2.des))
       }))
bp <- barplot(ss, xlab="School Type", ylab="Estimated Size",
              names=levels(apiclus2$stype),
              ylim=c(0,max(ci)))
arrows(bp, ci[,1], bp, ci[,2], angle=90, length=0.2, code=3)
```

```
names(ss) <- levels(apiclus2$stype)
cbind(as.numeric(ss),ci) |>
  kbl(col.names=c("Estimate","2.5%","97.5%"),
      digits=3,
      caption="Asymmetric confidence interval estimates") |>
  kable_styling(full_width=FALSE)
```

The biggest thing to notice is that the confidence interval for "High" is now asymmetric: it's longer on the high side that the low side, and more correctly represents our uncertainty. The estimates for "Elementary" and "Middle" are essentially unchanged.

For **crosstabulations** we can use `svytable()`, which gives weighted counts in categories - e.g. of whether School Wide growth targest had been met

```
svytable(~stype+sch.wide, design=apiclus2.des)
```

```
##             sch.wide
## stype              No      Yes
##   Elementary  242.240 3251.315
##   Middle      446.630  499.620
##   High        586.675  102.195
```

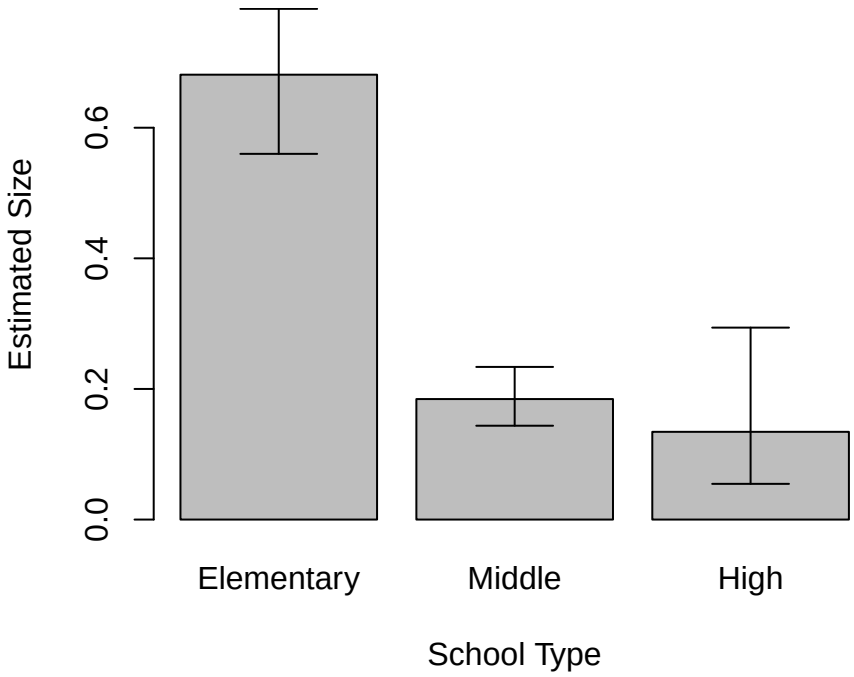


Figure 7.9: Estimated proportions of schools by school type - with asymmetric confidence intervals

Which can be displayed with a side-by-side barplot

```
st <- svytable(~sch.wide+stype, design=apiclus2.des)
bp <- barplot(st, beside=TRUE, col=c("light blue","dark red"),
              legend=TRUE, args.legend=list(title="Met targets"),
              xlab="School Type", ylab="Frequency")
```

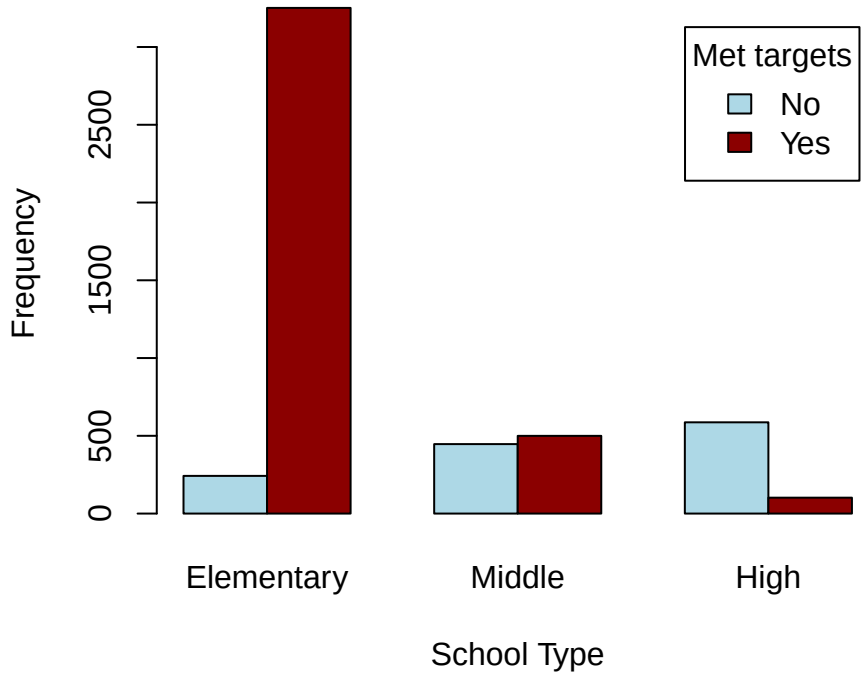


Figure 7.10: Counts of schools meeting targets, by school type

A test of association is given by the weighted Chi-Square test

```
svychisq(~stype+sch.wide, design=apiclus2.des)
```

```
##
##  Pearson's X^2: Rao & Scott adjustment
##
## data:  svychisq(~stype + sch.wide, design = apiclus2.des)
## F = 11.873, ndf = 1.3825, ddf = 53.9175, p-value = 0.0003221
```

We can carry out a logistic regression to make a similar test of association.
We have `sch.wide` as the binary outcome variable and `stype` as the predictor:

```
summary(svyglm(I(sch.wide=="Yes")~stype, family=quasibinomial(link="logit"),
               design=apiclus2.des))
```

```
##
## Call:
```

```
## svyglm(formula = I(sch.wide == "Yes") ~ stype, design = apiclus2.des,
##     family = quasibinomial(link = "logit"))
##
## Survey design:
## svydesign(ids = ~dnum + snum, fpc = ~fpc1 + fpc2, data = apiclus2)
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.5969     0.5997   4.331 0.000109 ***
## stypeMiddle  -2.4848     1.1565  -2.148 0.038292 *
## stypeHigh    -4.3445     0.8757  -4.961 1.59e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasibinomial family taken to be 1.008)
##
## Number of Fisher Scoring iterations: 5
```

With regular `lm()` and `glm()` regression fits we can use F tests to determine whether specific terms in the model should be included. A similar test can be done to test for the inclusion of specific terms using `regTermTest`

```
sr <- svyglm(I(sch.wide=="Yes")~stype, family=quasibinomial(link="logit"),
             design=apiclus2.des)
regTermTest(sr, ~stype)
```

```
## Wald test for stype
##  in svyglm(formula = I(sch.wide == "Yes") ~ stype, design = apiclus2.des,
##     family = quasibinomial(link = "logit"))
## F =  13.61363  on  2  and  37  df: p= 3.7061e-05
```

Here we can see that `stype` is contributing significantly to the fit of the model.

7.3 Ratio estimation

There often arise circumstances in survey analysis where we can expect one variable to be proportional to another: for example population and GDP are likely to move together.

The are three clear outliers here are the US, China and India.

We have full data on $N = 150$ countries, and the total GDP of the world in 2023 was $Y = 9.1046 \times 10^4$ in billions of US dollars.

Let's imagine though that we only had a sample of $n = 40$ countries out of the $N = 150$ and wanted to estimate the total GDP Y. The first 10 rows of our sample are shown below.

Our sample happens to miss out on all three of the big countries here. Note that there are some mid-size countries with low GDP visible in the plot, so that the relationship between population and GDP isn't simple.

Our design specification is:

```
gdp.sample.des <- svydesign(id=~1, fpc=~bigN, data=gdp.sample)
```

and our total GDP estimate is

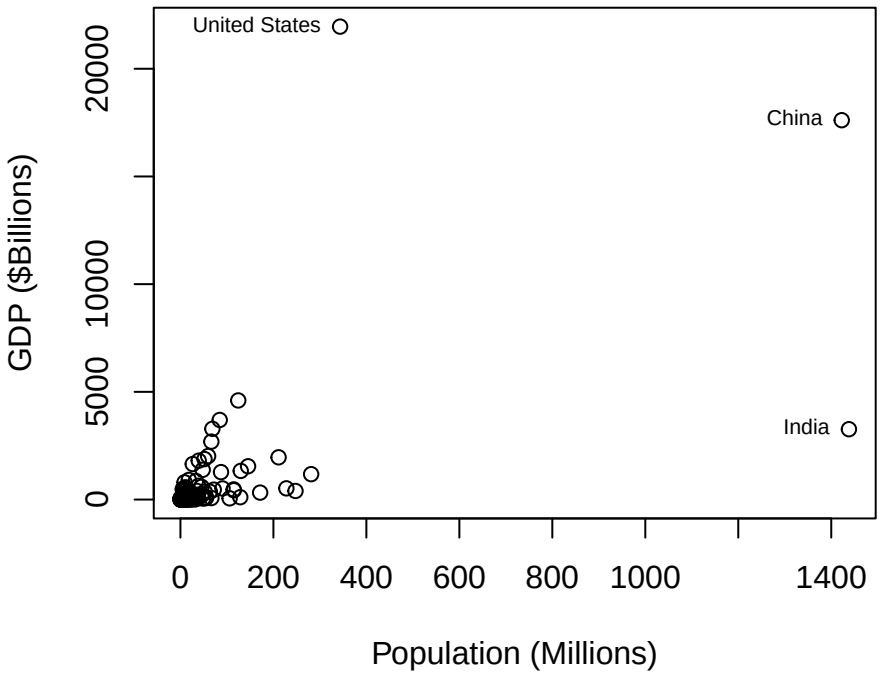


Figure 7.11: Population and GDP (2023) (Source: Our World in Data)

Country	Population	GDP	bigN	weight
Zambia	20.723967	27.580696	150	3.75
Saudi Arabia	33.264289	863.963569	150	3.75
Mauritius	1.273591	14.170188	150	3.75
Bahrain	1.569674	39.966811	150	3.75
Benin	14.111035	17.831007	150	3.75
Myanmar	54.133800	63.756974	150	3.75
Mozambique	33.635163	20.532020	150	3.75
Bosnia and Herzegovina	3.185072	20.681711	150	3.75
Montenegro	0.633553	5.161591	150	3.75
Japan	124.370947	4601.203386	150	3.75

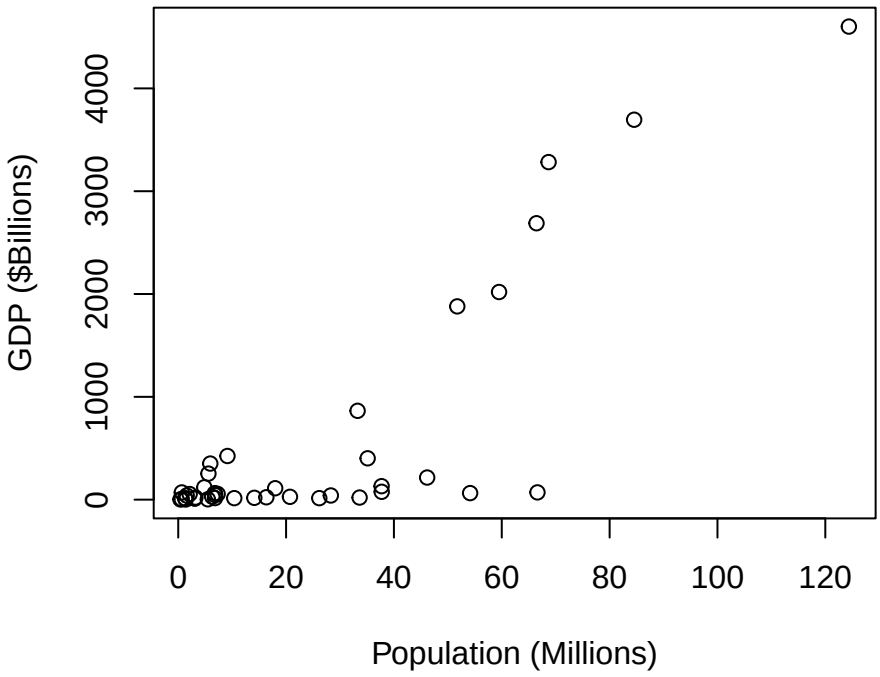


Figure 7.12: Sample of 40 countries: Population and GDP (2023) (Source: Our World in Data)

```
ss1 <- svytotal(~GDP, gdp.sample.des)
ss1
```

```
##     total    SE
## GDP 81890 22980
```

This estimate has a very high SE due to the high variability among countries in GDP, and leads to a very wide confidence interval

```
ci1 <- confint(ss1)
ci1
```

```
##        2.5 %   97.5 %
## GDP 36850.06 126930.8
```

with a very high RSE

```
cv(ss1)
```

```
##           GDP
## GDP 0.2806213
```

indicating an estimate of low quality.

We could use the correlation between population x_i and GDP y_i and fit a standard regression model

$$y_i = \beta x_i + \varepsilon_i \qquad \varepsilon_i \sim N(0, \sigma^2)$$

omitting the intercept since we want GDP to be proportional to population.

```
lmfit <- svyglm(GDP ~ -1 + Population, design=gdp.sample.des)
summary(lmfit)
```

```
##
## Call:
## svyglm(formula = GDP ~ -1 + Population, design = gdp.sample.des)
##
## Survey design:
## svydesign(id = ~1, fpc = ~bigN, data = gdp.sample)
##
## Coefficients:
##            Estimate Std. Error t value Pr(>|t|)
## Population   27.870      4.117    6.77 4.44e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 423229.9)
##
## Number of Fisher Scoring iterations: 2
```

The coefficient β is the GDP ($Billions) per million people in the population. A 95% confidence interval for β is

```
confint(lmfit)
```

```
##                2.5 %   97.5 %
## Population 19.54262 36.19642
```

It does have a better RSE than our earlier estimate

```
cv(lmfit)
```

```
## Population
##  0.1477149
```

Scaling the estimated GDP per million by the total population of the world ($X = 7646.716928$ million) we still get a rather wide confidence interval

```
ci2 <- confint(lmfit)*poptotal
ci2
```

```
##                2.5 %   97.5 %
## Population 149436.9 276783.8
```

The reason we're still not doing very well is that the regression model isn't a good representation of the situation. Take a look at the residual plot:

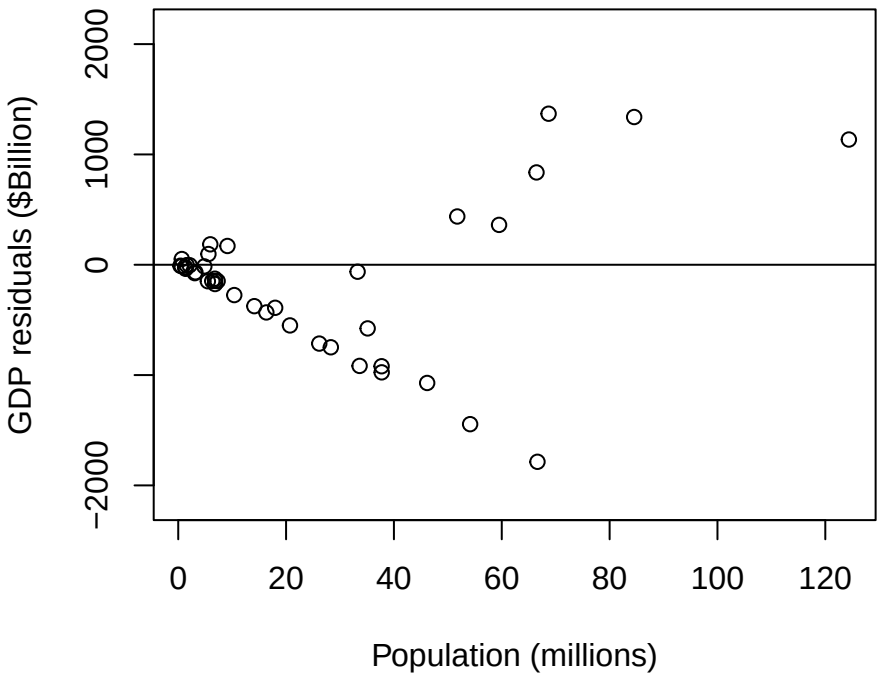


Figure 7.13: Residual plot

This is a terrible looking residual plot, not least because the variance so clearly isn't constant: there is clear **funnelling** of the residuals. The variance is increasing with population. That isn't surprising in that there is enormous variety in GDP per capita among the larger countries.

An improvement on the model is to have a residual variance which scales with population x_i:

$$y_i = \beta x_i + \varepsilon_i \qquad \varepsilon_i \sim N(0, \sigma^2 x_i)$$

Method	Estimate	RSE	CI
Simple	81890	0.28	(36850,126931)
Regression	213110	0.15	(149437,276784)
Ratio	164829	0.18	(107733,221924)

We can estimate this ratio of the total of Y to the total of X directly using the `svyratio()` function:

```
ss3 <- svyratio(~GDP, ~Population, design=gdp.sample.des)
ss3
```

```
## Ratio estimator: svyratio.survey.design2(~GDP, ~Population, design = gdp.sample.des
## Ratios=
##     Population
## GDP   21.55548
## SEs=
##     Population
## GDP   3.809595
```

The SE is slightly larger than in our first regression model, but we have more confidence in this model due to its better treatment of the variance. The confidence interval for the GDP per capita is now

```
confint(ss3)
```

```
##                    2.5 %   97.5 %
## GDP/Population 14.08881 29.02215
```

leading to the following confidence interval for the total GDP of the world in 2023:

```
ci3 <- confint(ss3)*poptotal
ci3
```

```
##                    2.5 %   97.5 %
## GDP/Population 107733.2 221924.2
```

Collecting all of these results together we see the improvement in our estimate by using the additional information of population size alongside each country's GDP.

Chapter 8

Weight adjustments

8.1 Weight Calibration

At the end of a survey we may find ourselves in the undesirable situation that

- **Purely by chance**, the sample is somewhat unbalanced: e.g. by chance we may have selected many more males than females, or more large households than small ones, or
- We may have had **unit nonresponse** - i.e. there may be selected units from which we have collected no data. In the case of human subjects this may have been from a failure to contact the selected person, or the refusal of the person to answer our questions.

In both cases we are in the situation that our sample, even when weighted, is not a good representation of the population from which it is drawn.

For example consider the SURF dataset, which is a synthetic sample of 200 individuals between the ages of 15 and 45. The first 10 rows are shown below.

The dataset contains a mixture of numerical and categorical variables, all with very self explanatory names (except note that `Income` and `Hours` are weekly quantities, and also that the income values are somewhat old!). We'll treat it as a SRSWOR from the New Zealand population between the ages of 15 and 45.

The total population in this age range is $N = 1.972719 \times 10^6$, the sample size is $n = 200$, and so the weight of each sampled individual is

$$w = \frac{N}{n} = 1.972719 \times 10^6 200 = 9863.6$$

Personid	Gender	Qualification	Age	Hours	Income	Marital	Ethnicity	AgeGrp	bigN	weight
1	female	school	15	4	87	never	European	15-24	1972719	9863.595
2	female	vocational	40	42	596	married	European	35-45	1972719	9863.595
3	male	none	38	40	497	married	Maori	35-45	1972719	9863.595
4	female	vocational	34	8	299	never	European	25-34	1972719	9863.595
5	female	school	45	16	301	married	European	35-45	1972719	9863.595
6	male	degree	45	50	1614	married	European	35-45	1972719	9863.595
7	female	none	36	12	201	other	European	35-45	1972719	9863.595
8	male	degree	35	45	934	previously	European	35-45	1972719	9863.595
9	female	vocational	38	26	624	married	European	35-45	1972719	9863.595
10	male	school	37	50	533	previously	European	35-45	1972719	9863.595

AgeGrp	male	female	Total
15-24	324888	310032	634920
25-34	337845	340668	678513
35-45	321358	337928	659286
Total	984091	988628	1972719

Let's specify this simple random sampling design to R:

```
surf.des <- svydesign(id=~1, fpc=~bigN, data=surf)
```

and then estimate the mean income:

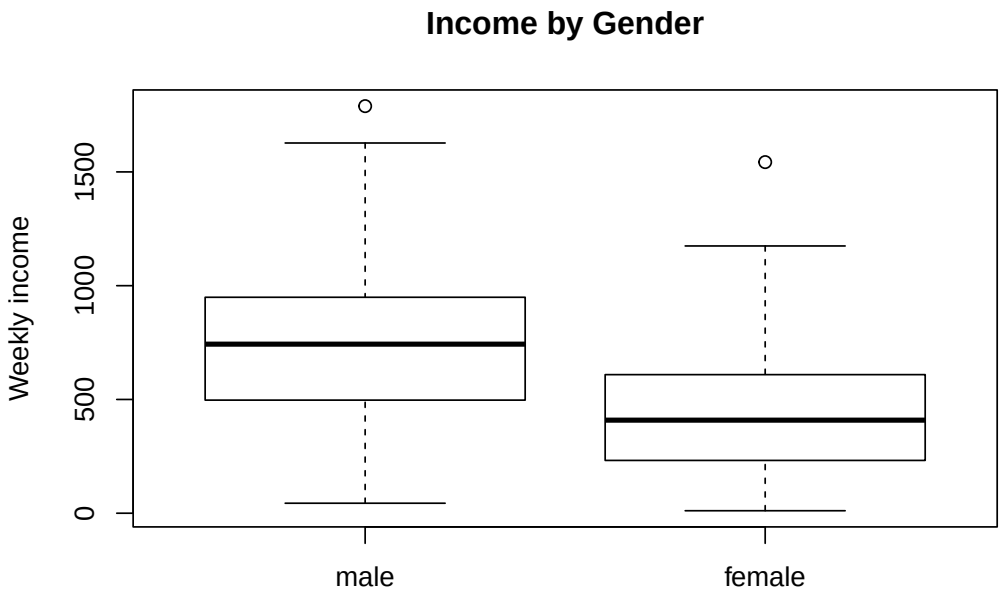
```
svymean(~Income, design=surf.des)
```

```
##          mean     SE
## Income 575.36 24.508
```

We should note that the mean income differs considerably between males and females.

```
svyby(~Income, by=~Gender, design=surf.des, svymean)
```

```
##        Gender   Income       se
## male     male 732.8172 37.02642
## female female 438.5047 26.16272
```



Let's also note the distribution of the population by age and sex, as measured by the most recent census:

Or we can express these counts as percentages:

The proportions of males and females in the population are roughly equal in the population, but in the sample they're not quite equal:

AgeGrp	male	female	Total
15-24	16.5%	15.7%	32.2%
25-34	17.1%	17.3%	34.4%
35-45	16.3%	17.1%	33.4%
Total	49.9%	50.1%	100.0%

```
svymean(~Gender, design=surf.des)
```

```
##                mean     SE
## Gendermale    0.465 0.0354
## Genderfemale 0.535 0.0354
```

This has been caused by our having, by chance, sampled slightly more females than males.

```
table(surf$Gender)
```

```
##
##   male female
##     93    107
```

The consequence of this is that our estimate of the population mean income is and underestimate, due to females earning less than males, and their being slightly overrepresented in the sample.

One obvious solution to this situation is to slightly upweight the males in the sample, and downweight the females, to take care of the imbalance. Of course we only **know** about the imbalance because we have the census counts which tell us that the proportions of males and females are roughly equal: otherwise we'd have no idea that there might be a problem.

These external counts (external to the survey that is) are called **population benchmarks**, and the weight adjustment that makes the weights match the benchmarks is called **post-stratification**. It's 'post-' here because we're making the stratification (breaking into groups) **after** the survey has taken place. Breaking the sample into groups **beforehand**, and taking separate samples within each group is called stratified sampling. But that is only possible if we know the group membership of sample members before we sample them - i.e. if the group membership information is available on the frame.

We have $n = 200$ individuals in a SRSWOR drawn from a population of size $N = 1.972719 \times 10^6$, with weights $w = 1.972719 \times 10^6/200 = 9863.6$. These correctly add up to the population total

$$n \times w = n \times \frac{N}{n} = 200 \times 9863.6 = 1.972719 \times 10^6$$

What we want is different weights for the males and females, so that they separately add up to the total numbers of males $N_1 = 9.84091 \times 10^5$ and the total numbers of females $N_2 = 9.88628 \times 10^5$ in the population. We can achieve this by computing new post-stratified weights using the known benchmarks N_h and sample sizes n_h in each of the post-strata, and computing new weights

$$w_h = \frac{N_h}{n_h}$$

i.e. these are the weights we'd have used if we had actually carried out a stratified sample in the first place.

These new weights are now appropriately higher for males and lower for females.

In R we can modify the weights in the design object using the `postStratify()` function. We first need to create a data frame storing the benchmarks:

Gender	SampWeight	nh	Nh	PostStratWeight
male	9863.595	93	984091	10581.624
female	9863.595	107	988628	9239.514

Gender	Nh
male	984091
female	988628

```
surf.des.ps <- postStratify(surf.des, strata=~Gender, population=Gender.benchmarks)
```

The orignal estimate of the proportions of males and females

```
svymean(~Gender, design=surf.des)
```

```
##                mean     SE
## Gendermale   0.465 0.0354
## Genderfemale 0.535 0.0354
```

has been modified to

```
svymean(~Gender, design=surf.des.ps)
```

```
##                  mean SE
## Gendermale   0.49885  0
## Genderfemale 0.50115  0
```

Notice that the Standard Errors here are zero: these proportions are no longer estimates, they have been **imposed** by our benchmarks.

Then we can compare the orginal estimate of mean income

```
svymean(~Income, surf.des)
```

```
##          mean     SE
## Income 575.36 24.508
```

with the post-stratified estimate

```
svymean(~Income, surf.des.ps)
```

```
##          mean     SE
## Income 585.32 22.651
```

The estimate has been increased slightly as expected. Also note that the Standard Error has been reduced: this is often the case with post-stratified estimates: post-stratification makes the properties of different samples differ less than they otherwise would, and thereby reduces our overall uncertainty.

Poststratfication as a treatment for non-response

Non-response can make the small chance differences in the previous section even worse.

In population surveys males and young people tend to have the lowest response rates. So let's see what happens to our mean income estimate if we take our sample of 200 individuals and filter out 20% of the male respondents:

```
set.seed(234)
surf.nonresp <- surf
```

AgeGrp	Gender	count
15-24	female	310032
15-24	male	324888
25-34	female	340668
25-34	male	337845
35-45	female	337928
35-45	male	321358

```
surf.nonresp$Responds <- rbinom(nrow(surf),1,prob=ifelse(surf$Gender=="male",0.8,1.0))
surf.nonresp <- surf.nonresp[surf.nonresp$Responds==1,]
surf.nonresp$weight <- bigN/nrow(surf.nonresp)
table(surf.nonresp$Gender)
```

```
##
##   male female
##     75    107
```

We've now got a lot fewer males than females. Let's see what this does to the simple estimate of the population mean income:

```
surf.nonresp.des <- svydesign(id=~1, fpc=~bigN, data=surf.nonresp)
svymean(~Income, design=surf.nonresp.des)
```

```
##          mean     SE
## Income 555.99 24.501
```

The introduction of the non-response has led to a reduction in our original estimate from 575.36 to 555.99.

Post-stratifying however

```
surf.nonresp.des.ps <- postStratify(surf.nonresp.des, strata=~Gender,
                                    population=Gender.benchmarks)
```

leads to a dramatically improved estimate

```
svymean(~Income, design=surf.nonresp.des.ps)
```

```
##          mean     SE
## Income 580.73 23.354
```

We know more than just the distribution of the population over Gender, we also know it over the three age groups. So we can post-stratify by these variables too.

We can store the AgeGrp/Gender benchmarks in a data frame `AgeGrp.Gender.benchmarks`:

and then post-stratify

```
surf.nonresp.des.ps2 <- postStratify(surf.nonresp.des, strata=~AgeGrp*Gender,
                                     population=AgeGrp.Gender.benchmarks)
svymean(~Income, design=surf.nonresp.des.ps2)
```

```
##          mean    SE
## Income 580.26 22.66
```

And this is estimate is going to be even better than before.

However where should we stop? Should we post-stratify by as many variables as we possibly can?

In general we need to ensure that we have enough individuals in the sample in every cross-classified cell (in this case AgeGroup by Gender) to be able to recalculate the weight. Empty cells are going to cause us a problem, since we'll be discarding the population weight associated with people of that type in the population who happen not to have turned up in the sample.

In population surveys run by Statistics NZ we'll often see post-stratification by age (in 5 year bands), sex, ethnicity (in large amalgamated groups), and large regions.

The origin and nature of nonresponse

Post-stratification is often used to modify weights in the presence of non-response. The idea is that in each post-stratum the people who respond represent well the people who don't respond.

This is a very big and untestable assumption.

The reasons that people do or don't respond are complicated. The greatest risk to a survey is that the non-respondents differ from the respondents on the characteristic of interest.

For example if we're doing a survey of incomes, perhaps all the high income people don't respond - there's no way to adjust for this, because the only way to know about income is to have them answer the survey. So we'll never find out about high income people, nor find out what their incomes actually are.

Post-stratifying the income survey above by age and sex means that we're assuming that the respondents in each age-sex category are a simple random sample from all of the sample members selected into the survey. We can cope with, and adjust for, differing response rates **between** the post-strata, but we can never adjust for different likelihoods of response **within** the post-strata.

A helpful approach to missing data has a typology with three types of missingness.
If we have auxliary data $\mathbf{X}_i$ for each unit, and are interested in an outcome variable Y_i for each unit, then we are interested in the **probability that unit i will respond if selected**

$$\phi_i = \Pr(\text{Unit } i \text{ responds} | \text{Unit } i \text{ is selected})$$

1. **Missing Completely at Random (MCAR)**. This is when the probability of response ϕ_i is the same for all units, no matter what their value of Y_i and $\mathbf{X}_i$.

   This is the best situation: the respondents and nonrespondents do not differ in any important way, and it is as if the nonrespondents had been selected at random from the sample. To analyse data where we have MCAR, we can simply ignore the non-responding units.

2. **Missing at Random (MAR)**. This is also known as 'missing at random given covariates' or alternatively 'ignorable nonresponse.' This occurs when the probability of response ϕ_i depends **only on known quantities**: the auxiliary covariates $\mathbf{X}_i$, but not on the unknown Y_i. We therefore assume that if two units have the same values of $\mathbf{X}$, then their likelihood of response is the same.

   This is a slightly more complex situation than MCAR, but we can still fully account for nonresponse. This is the approach that post-stratification implements: we treat all respondents as equally likely to respond within the post-strata.

	AgeGrp	Nh
15-24	15-24	634920
25-34	25-34	678513
35-45	35-45	659286

Gender	Nh
male	984091
female	988628

3. **Nonignorable Nonresponse.** This is the unfortunate situation where the probability of response ϕ_i depends on the outcome of interest Y_i (as well, possibly, as $\mathbf{X}_i$).

MAR and MCAR are **strong assumptions**. If the data are MAR we can test whether they are MCAR by comparing nonresponse rates in subgroups defined by $\mathbf{X}$ (or by logistic regression if any of the X variables are continuous) and test for any dependence on $\mathbf{X}$.

However it is in general impossible to distinguish whether the data are MAR or whether there is nonignorable response.

We of course exclude from consideration any data which is **missing by design.** There are individuals who if selected turn out to be out of scope (ineligible to participate). These may also be data items which are missing because the respondent is not asked that particular question (e.g. we do not ask for the voting record of children who are not entitled to vote.)

Raking

We may not have full cross-classified benchmarks, for example we might know the totals by AgeGrp and totals by Gender, but not totals by AgeGrp and Gender simultaneously. In that case we can proceed iteratively: we post-stratify first by one variable, and then by other. Each post-stratification on one variable messes up the post-stratification on the other. But eventually the weights will converge to approximately match both sets of benchmarks. This iteration is called **raking** (like raking a lawn one way, then other other), or **Iterative Proportional Fitting**.

R implements raking in with the `rake()` function. We need a separate set of benchmarks for AgeGrp

and for Gender

```
Gender.benchmarks |> kbl() |> kable_styling(full_width=FALSE)
```

With these we can rake the design:

```
surf.nonresp.des.rake <- rake(surf.nonresp.des,
                              sample=list(~AgeGrp, ~Gender),
                              population=list(AgeGrp.benchmarks, Gender.benchmarks))
```

and then calculate our estimates

```
svymean(~Income, design=surf.nonresp.des.rake)
```

```
##          mean     SE
## Income 579.58 22.798
```

We can compare estimated population totals with the Population benchmarks:

Estimated counts with no weight adjustments

AgeGrp	male	female	Total
15-24	324888	310032	634920
25-34	337845	340668	678513
35-45	321358	337928	659286
Total	984091	988628	1972719

	AgeGrp	male	female	SE(male)	SE(female)
15-24	15-24	270977.9	357690.8	50472.45	56491.70
25-34	25-34	260138.8	346851.7	49609.94	55815.54
35-45	35-45	281817.0	455242.8	51307.81	61776.48

Post-stratified by Gender

Post-stratified by Age Group cross classified with Gender: exact match to benchmarks

Post-stratified by raking to Age Group and Gender marginal totals: approximate match to benchmarks

Finally compare the estimates of mean income

The Standard Errors are all similar, though we do get a small reduction in SE with post-stratification. However it is the reduction in the biasing effect of non-response that is most striking.

Another example

In the Academic Performance Indicator dataset we have full population information on types of School (`stype`=Elementary/Middle/High) and whether or not they met their year round performance indicators (`yr.rnd`=Yes/No). These counts will be our benchmarks for post-stratification.

The numbers of schools by type are in `stype.benchmnarks`

The number of schools by achievement of year round objectives are in `yr.rnd.benchmarks`

and we also have these counts **cross-classified**, i.e. numbers of schools for each combination of `stype` and `yr.rnd`:

Select a two stage cluster sample - and then make estimates of the mean school size (`enroll`)

```
set.seed(111)
api2sc <- apipopc |>
  select.2sc(n=40, cfraction=0.30, mmin=2, clustervar="dnum")
api2sc.des <- svydesign(ids=~dnum+snum, fpc=~bigN1+bigN2, data=api2sc)
table(api2sc$stype)/nrow(api2sc)
```

```
##
## Elementary     Middle       High
##  0.6224490  0.1428571  0.2346939
```

	AgeGrp	male	female	SE(male)	SE(female)
15-24	15-24	328030.3	304904.0	53712.42	44259.49
25-34	25-34	314909.1	295664.4	53150.88	43877.23
35-45	35-45	341151.5	388059.6	54225.61	46796.67

	AgeGrp	male	female	SE(male)	SE(female)
15-24	15-24	324888	310032	0	0
25-34	25-34	337845	340668	0	0
35-45	35-45	321358	337928	0	0

	AgeGrp	male	female	SE(male)	SE(female)
15-24	15-24	327978.3	306941.7	36276.56	36276.56
25-34	25-34	348793.8	329719.2	38089.76	38089.76
35-45	35-45	307318.9	351967.1	36046.49	36046.49

Method	Estimate	SE
True	575.3600	0.00000
No adjustment	555.9945	24.50130
PS(Gender)	580.7311	23.35388
PS(AgeGrp*Gender)	580.2640	22.65980
Rake(Age+Gender)	579.5795	22.79842

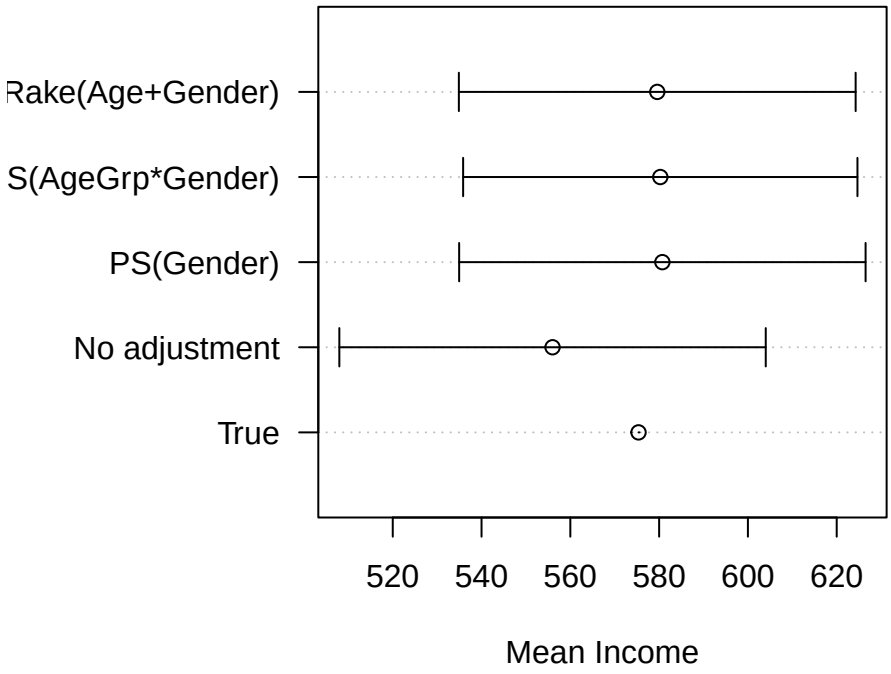


Figure 8.1: Estimates of mean income by adjustment method

stype	Freq
Elementary	4397
Middle	1009
High	751

yr.rnd	Freq
No	874
Yes	5283

stype	yr.rnd	Freq
Elementary	No	791
Middle	No	60
High	No	23
Elementary	Yes	3606
Middle	Yes	949
High	Yes	728

```
svytable(~stype, design=api2sc.des)
```

```
## stype
## Elementary     Middle       High
##  2972.2159   589.3841   946.0500
```

```
svytable(~stype, design=api2sc.des)/sum(api2sc$weight)
```

```
## stype
## Elementary     Middle       High
##  0.6593715  0.1307520  0.2098765
```

```
svymean(~enroll, design=api2sc.des)
```

```
##          mean     SE
## enroll 711.45 81.057
```

```
svyby(~enroll, by=~stype, api2sc.des, svymean)
```

```
##                 stype    enroll        se
## Elementary Elementary  415.6678  46.44827
## Middle         Middle  834.1254 113.51448
## High             High 1564.3007 190.86274
```

Poststratify by stype and reestimate

```
api2sc.des.ps <- postStratify(api2sc.des, strata=~stype, population=stype.benchmarks)
svymean(~enroll, api2sc.des.ps)
```

```
##          mean     SE
## enroll 624.35 53.142
```

Postratify by stype by yr.rnd and reestimate

```
api2sc.des.ps2 <- postStratify(api2sc.des, strata=~stype+yr.rnd,
                               population=stype.yr.rnd.benchmarks)
svymean(~enroll, api2sc.des.ps2)
```

```
##          mean     SE
## enroll 629.45 40.239
```

Poststratify by stype + yr.rnd (i.e. we only have the marginals so we need to rake) and reestimate

	Estimate	SE
Cenus	619.0469	0.00000
2SC	711.4530	81.05714
2SC+PS(stype)	624.3485	53.14154
2SC+PS(stype x yr.rnd)	629.4503	40.23927
2SC+Rake(stype+yr.rnd)	632.0372	46.70502

```
api2sc.des.rake <- rake(api2sc.des,
                        sample=list(~stype, ~yr.rnd),
                        population=list(stype.benchmarks, yr.rnd.benchmarks))
svymean(~enroll, api2sc.des.rake)
```

```
##          mean     SE
## enroll 632.04 46.705
```

Compare

8.2 Replicate weights

For some weight adjustments we no longer have a reliable method of computing the standard errors of our estimates. This can occur when we have very complex designs. It can also be necessary when we don't have access to the frame, and therefore don't have all the information we need (such as population sizes in clusters) to calculate standard errors.

In such circumstances we often create multiple sets of weights in addition to the adjusted weights. These **replicate weights** are what are used to estimate the standard errors, and they have the advantage that they can create standard errors for a wider range of estimators than the usual means and totals.

The basic idea comes from recalling what standard errors are: they are the variability we'd see in an estimate across all possible samples from the population using the chosen sample design and sample size (Figure 8.2).

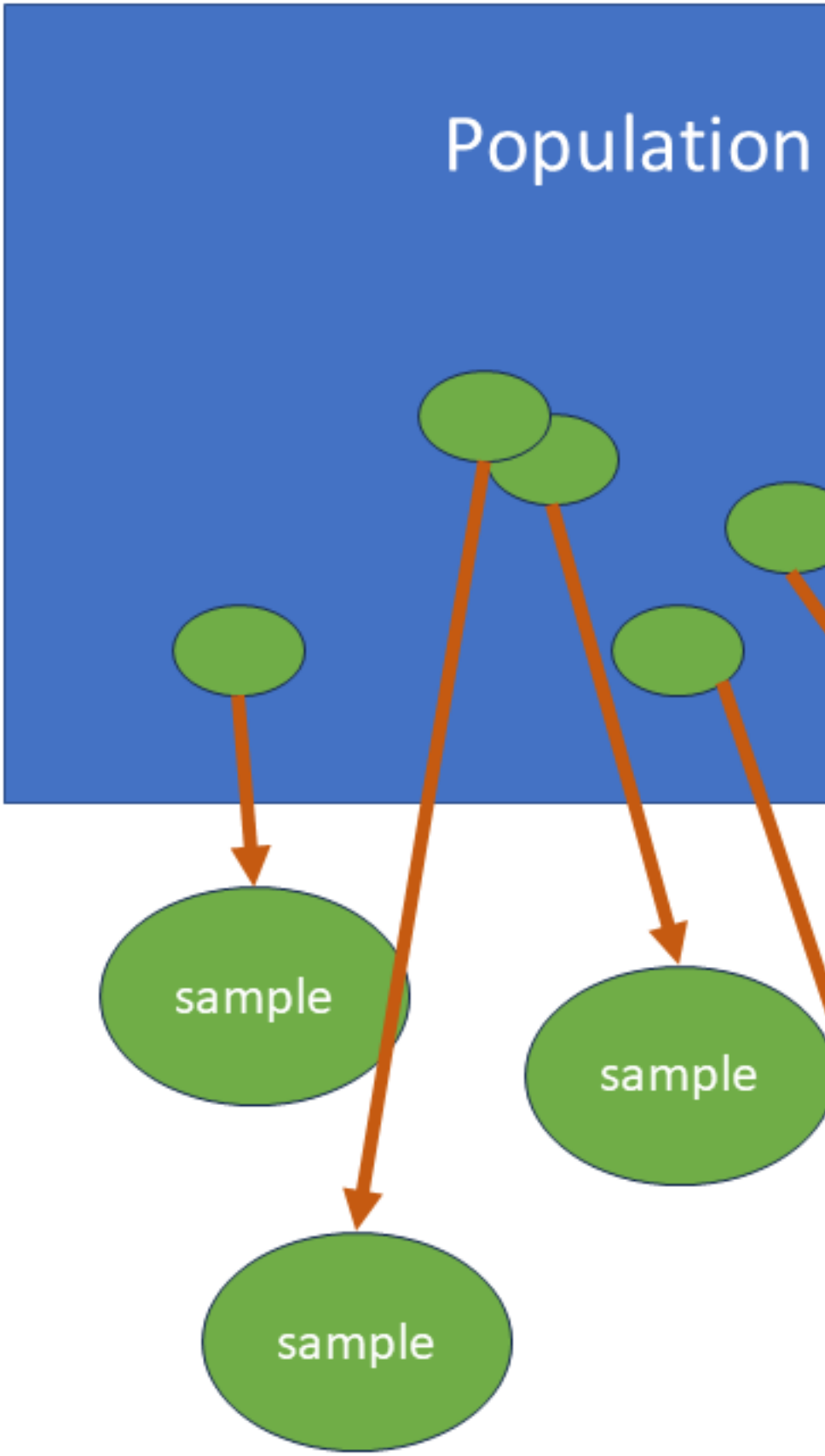
Once we've completed a survey we obviously can't take any more samples from the population. Instead we use the variability we see within the **sample** to capture and estimate the variability we'd see in different samples from the **population** (Figrue 8.3).

Instead of actually resampling from our sample, we instead just vary the weights on some or all of the unit, and create new sets of weights according to a well defined method. We do this multiple times, which is equivalent to taking multiple re-samples of our actual sample. Each re-sample is called a **replicate** and has its own set of weights for all of the units in the sample. e.g. we might have 100 additional sets of replicate weights attached to our survey data set, as well as the actual weights.

We then use these replicate weights to calculate a set of alternative values for our estimate of interest. The variability among these replicate estimates is then used to determine the standard error.

There are a number of different ways to create replicate weights: we'll consider just one known as the **Jackknife**.

Each Jacknife replicate set of weights is calculated by deleting one or more PSUs at the first stage of selection by setting their weights to zero. We then rescale the weights of the remaining units so that they still add up to the same total weight (i.e so they weights still add up to the population size). Table 8.1 shows an example.


Population
sample
sample
sample

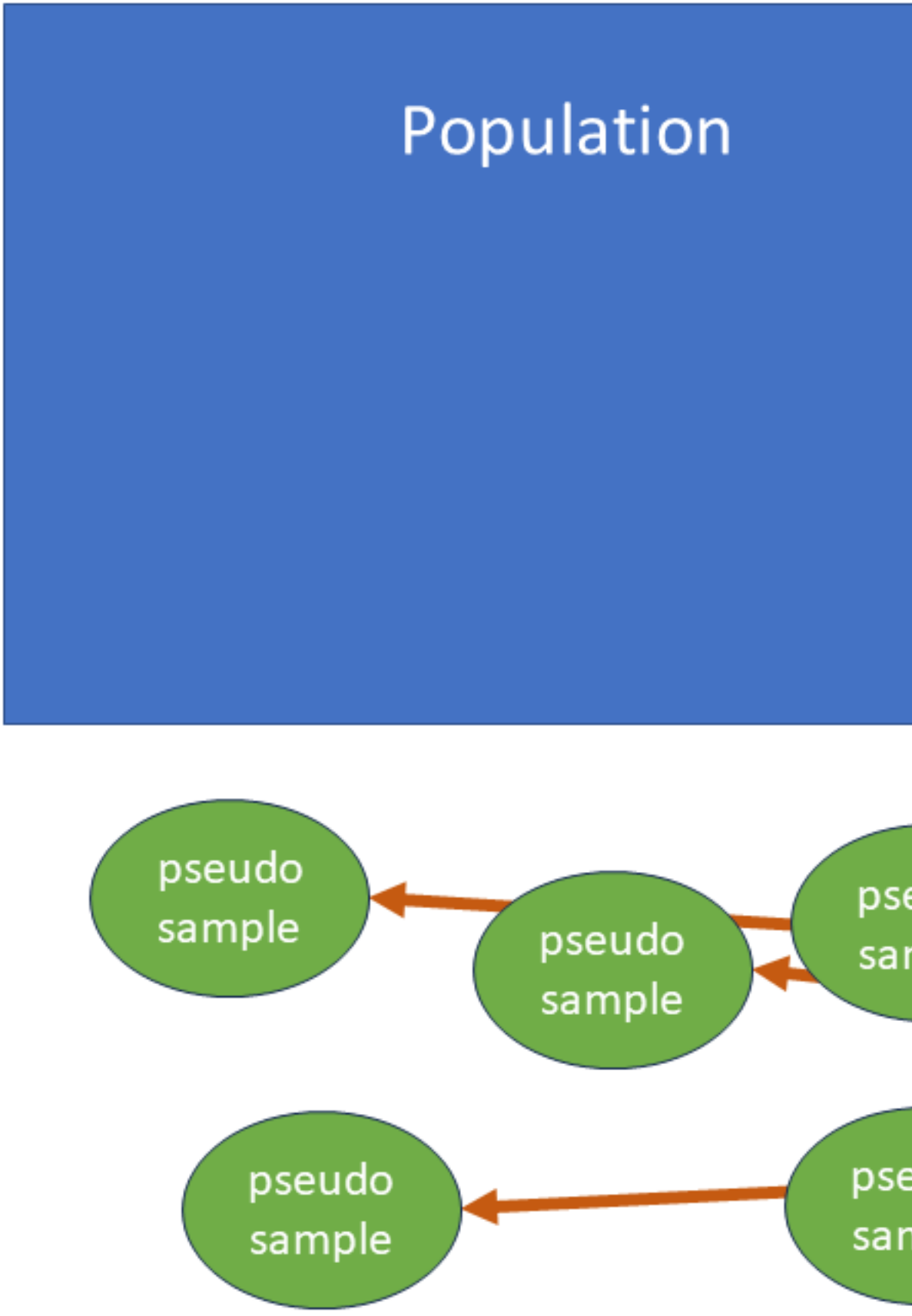

Population
pseudo sample
pseudo sample
pseudo sample

Table 8.1: Example of Jacknife replicate weights: a sample of 4 clusters (PSUs) with sets of replicate weights, each one deleting the weight of a single PSU, and rescaling to maintain the total of the weights.

id	psu	weight	rep01	rep02	rep03	rep04
1	A	100	0.0	138.1	126.1	134.9
2	A	100	0.0	138.1	126.1	134.9
3	A	100	0.0	138.1	126.1	134.9
4	B	80	107.9	0.0	100.9	107.9
5	B	80	107.9	0.0	100.9	107.9
6	B	80	107.9	0.0	100.9	107.9
7	B	80	107.9	0.0	100.9	107.9
8	C	120	161.9	165.7	0.0	161.9
9	C	120	161.9	165.7	0.0	161.9
10	D	100	134.9	138.1	126.1	0.0
11	D	100	134.9	138.1	126.1	0.0
12	D	100	134.9	138.1	126.1	0.0
Total		1160	1160.0	1160.0	1160.0	1160.0

We can create a set of replicate weights for the two stage cluster sample from the API data set as follows:

```
api2sc.jkdes <- as.svrepdesign(api2sc.des, type="JK1")
```

```
## Warning in as.svrepdesign.default(api2sc.des, type = "JK1"): Finite population
## corrections after first stage have been dropped
```

There is an informative warning here: no fpc can apply at second stage of selection (nor any subsequent stages).

The specification **JK1** here indicates that the design is not stratified.

Here are the first 5 rows and 8 columns of replicate weights: each row refers to a PSU (i.e. a district `dnum`) and each column is a replicate.

```
api2sc.jkdes$repweights$weights[1:5,1:8]
```

```
##          [,1]     [,2]     [,3]     [,4]     [,5]     [,6]     [,7]     [,8]
## [1,] 0.000000 1.025641 1.025641 1.025641 1.025641 1.025641 1.025641 1.025641
## [2,] 1.025641 0.000000 1.025641 1.025641 1.025641 1.025641 1.025641 1.025641
## [3,] 1.025641 1.025641 0.000000 1.025641 1.025641 1.025641 1.025641 1.025641
## [4,] 1.025641 1.025641 1.025641 0.000000 1.025641 1.025641 1.025641 1.025641
## [5,] 1.025641 1.025641 1.025641 1.025641 0.000000 1.025641 1.025641 1.025641
```

Replicate Weights are $n/(n-1) = 40/39 = 1.025641$ - drops one of the 40 selected districts `dnum` out for each replicate, setting the weight for all units selected in that district to zero.

These are called **scaling weights**, and they need to be multiplied by the original sampling weight to get the actual replicate weights.

We can use this design just as in earlier examples to get estimates and standard errors:

```
svymean(~enroll, design=api2sc.jkdes)
```

```
##          mean     SE
## enroll 711.45 82.848
```

	Estimate	SE
Cenus	619.0469	0.00000
2SC	711.4530	81.05714
2SC+PS(stype)	624.3485	53.14154
2SC+PS(stype x yr.rnd)	629.4503	40.23927
2SC+Rake(stype+yr.rnd)	632.0372	46.70502
JK1	711.4530	82.84780
JK1+PS(stype)	624.3485	60.60973

This gives us a good estimate of the SE, similar to what we get from the original sample design specification:

```
svymean(~enroll, design=api2sc.des)
```

```
##          mean     SE
## enroll 711.45 81.057
```

Note that the estimate itself is identical: in both cases they come from the same single set of sample weights.

If we are needing to post-stratify our weights to account for non-response, then we need also to poststratify each set of replicate weights.

```
api2sc.jkdes.ps <- postStratify(api2sc.jkdes, strata=~stype, population=stype.benchmarks)
svymean(~enroll, design=api2sc.jkdes.ps)
```

```
##          mean    SE
## enroll 624.35 60.61
```

Here post-stratification gives us a strong reduction of the standard errors, as we saw earlier.

Here's a comparison of the various estimates and standard errors from the various methods we've seen so far:

Important points to note:

- the SE is reduced by post-stratification, and
- we get similar SEs when we use the jacknife estimation method.

Let's take a look at the replicate weights after post-stratification

```
api2sc.jkdes.ps$repweights$weights[1:5,1:8]
```

```
##      [,1]    [,2]     [,3]     [,4]     [,5] [,6]    [,7]     [,8]
## 1 0.00000 1.00628 1.047964 1.019081 1.000000 1.00 1.00628 1.012640
## 3 0.00000 1.00000 1.000000 1.000000 1.049550 1.00 1.00000 1.067176
## 1 1.00945 0.00000 1.047964 1.019081 1.000000 1.00 1.00628 1.012640
## 1 1.00945 1.00628 0.000000 1.019081 1.000000 1.00 1.00628 1.012640
## 2 1.00000 1.00000 0.000000 1.000000 1.030303 1.02 1.00000 1.000000
```

They're no longer the same in the different PSUs.

This seems to be a lot of additional work: we get the same SEs and estimates using the standard formula-based estimation methods for SEs. Why bother with replicate weights in that case?

There are three reasons:

dnum	pweight	repweight001	repweight002	repweight003	repweight004	repweight005	re
31	41.16340	0.00000	41.42192	43.13775	41.94883	41.16340	
31	47.63519	0.00000	47.63519	47.63519	47.63519	49.99550	
44	27.44227	27.70160	0.00000	28.75850	27.96589	27.44227	
46	100.62165	101.57255	101.25359	0.00000	102.54158	100.62165	
46	53.99346	53.99346	53.99346	0.00000	53.99346	55.62963	
46	100.62165	101.57255	101.25359	0.00000	102.54158	100.62165	
51	41.16340	41.55241	41.42192	43.13775	0.00000	41.16340	
51	41.16340	41.55241	41.42192	43.13775	0.00000	41.16340	
67	47.63519	49.99550	47.63519	47.63519	47.63519	0.00000	
67	22.08824	22.08824	22.08824	23.79930	22.08824	0.00000	

- For some very **complex designs** there are no formula based estimation methods that can be used;
- For some **estimates**, such as medians and quartiles, replicate methods are more reliable than formula based methods;
- Replicate based methods are robust in the presence of small samples and outliers, and can also be used to estimate bias (as well as standard errors).

It's commonly the case that rather than calculating them for ourselves we might inestead have been **supplied** replicate weights, i.e. someone else might have created them. This often happens when we have been granted access to a data set collected by a government department or agency concerned about confidentiality, and in those instances we are not given access to the frame.

In which case the weights in the first 10 rows of our API two stage cluster dataset would look like this

Unlike the example earlier, these replicate weights are **combined weights** - i.e. they do not need to be multiplied by the original weight, and they each separately add up to the same value: the population size.

We use a data set like this to specify the sample design to R using just the probability weights `pweight`and the set of replicate weights `repweight001-repweight040`. We don't specify the cluster variables when specifying the design.

```
pweightcol <- match("pweight",names(api2sc.withrep.ps)) # which column name is exactly
repweightcols <- grep("^repweight",names(api2sc.withrep.ps)) # which columnnames start
varcols <- (1:ncol(api2sc.withrep.ps))
varcols <- varcols[!(varcols%in%c(pweightcol,repweightcols))]
n <- length(unique(api2sc$dnum))
api2sc.repdes.ps <- svrepdesign(variables=api2sc.withrep.ps[,varcols],
                                weights=api2sc.withrep.ps[,pweightcol],
                                repweights=api2sc.withrep.ps[,repweightcols],
                                combined.weights=TRUE,
                                scale=(n-1)/n,
                                type="JK1")
svymean(~enroll, api2sc.repdes.ps)
```

```
##          mean     SE
## enroll 624.35 62.313
```

The consequence of only being given the weights (rather than the full clustering information) means that the first stage design is treated as PPSWR rather than SRSWOR, and there is no adjustment for the fpc. That means the SEs are slightly larger.

	Estimate	SE
Cenus	619.0469	0.00000
2SC	711.4530	81.05714
2SC+PS(stype)	624.3485	53.14154
2SC+PS(stype x yr.rnd)	629.4503	40.23927
2SC+Rake(stype+yr.rnd)	632.0372	46.70502
JK1	711.4530	82.84780
JK1+PS(stype)	624.3485	60.60973
JK1+PS(stype); weights only	624.3485	62.31259

The method "JK1" is used for unstratified designs. For stratified designs the jackknife is specified by "JKn": we don't just drop out one PSU over the whole sample in each replicate, instead we drop out a PSU from every stratum simultaneously.

E.g. draw a two stage cluster sample within two strata: schools which are or are not eligible for an awards programme:

```
set.seed(999)
apist2sc <- apipopc |>
  select.st2sc(n=40, stratumvar="awards", cfraction=0.20, clustervar="dnum")
apist2sc.des <- svydesign(id=~dnum+snum, strata=~awards, fpc=~bigN1+bigN2,
                          data=apist2sc, nest=TRUE)
apist2sc.des.ps <- postStratify(apist2sc.des, strata=~stype, population=stype.benchmarks)
```

Note: we need `nest=TRUE` here because districts `dnum` do not nest within strata. I.e. it is the **combination** of `awards` and `dnum` that actually defines the clusters here.

```
svymean(~enroll, design=apist2sc.des)
```

```
##          mean     SE
## enroll 497.75 64.645
```

```
svymean(~enroll, design=apist2sc.des.ps)
```

```
##          mean     SE
## enroll 523.97 63.001
```

```
apist2sc.jkdes <- as.svrepdesign(apist2sc.des, type="JKn")
```

```
## Warning in as.svrepdesign.default(apist2sc.des, type = "JKn"): Finite
## population corrections after first stage have been dropped
```

```
svymean(~enroll, design=apist2sc.jkdes)
```

```
##          mean     SE
## enroll 497.75 84.997
```

```
apist2sc.jkdes.ps <- postStratify(apist2sc.jkdes, strata=~stype, population=stype.benchmarks)
svymean(~enroll, design=apist2sc.jkdes.ps)
```

```
##          mean     SE
## enroll 523.97 82.125
```

There are alternative methods to Jacknife generation of replicate weights. You may hear of Bootstrap weights, and Balanced Repeated Replication. Some of these have multiple variants, and there are settings in which one outperforms the others. Jackknife variance estimation is the most common however.

Chapter 9

Imputation

In the previous chapter we saw methods of dealing with **unit non-response**: i.e. we have units that are selected, but we collect no data at all from them.

Another common type of non-response is where a unit does not provide all the information requested: certain data items are therefore missing. This is called **item non-response**.

We could just do a complete case analysis - in other words throw all of the data away for such units, and treat them as unit non-responders with zero weight. This wastes data and effort, and misses the opportunity to use the data we have collected. It also leads to a loss of precision due to reduced sample size.

An alternative option is **imputation**: this means that we **fill in (i.e. invent) the missing data** using some kind of model.

All such models involve untestable assumptions, and we run the same risks as when we make weight adjustments for non-response. There is a substantial risk of bias in our estimates whatever we do: leaving these units fully out of our analysis risks missing an important part of the population. Incorrectly guessing their responses is of course also a risk if we do so incorrectly. We may be **introducing** bias in doing so.

Whatever we do our imputation must:

- be **minimal** - we should not impute more than a few per cent of our data;
- **never be done on our key variables** - if we're trying to find out about political views then the last thing we should be doing is guessing them. Less significant variables, e.g. household income, will have a lesser effect on our estimates;
- be **documented** - we should always know how much imputation has been done, on which variables, and most importantly **how** the imputation was done.

Let's take as an example the SURF income survey synthetic dataset of 200 individuals aged 15-45 from the NZ population. The top 10 rows are:

Let's remove some data: delete 10 entries at random from each of marital status and income.

```
set.seed(111)
msurf <- surf
msurf$Marital[sample(nrow(surf),10)] <- NA
msurf$Income[sample(nrow(surf),10)] <- NA
```

Here are the counts of the various responses to the `Marital` variable after removing 10 values.

Marital	Freq
married	68
never	82
other	20
previously	20
NA	10

```
library(mice)
library(lattice)
```

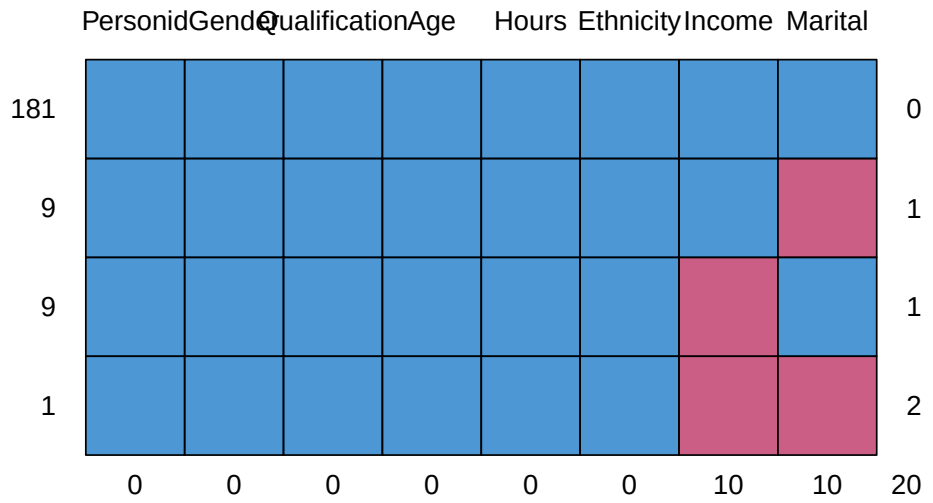
The R package `mice` provides for multiple imputation methods, and provides some demonstration datasets and some visualisation tools. However it's a bit tricky to use, so we'll use a simpler function `impute()`.

The `mice` package can draw a nice picture of the pattern of missingness in the SURF data, which is a mixture of numeric and categorical variables

```
summary(msurf)
```

```
##     Personid         Gender          Qualification           Age
##  Min.   :  1.00   Length:200         Length:200         Min.   :15.00
##  1st Qu.: 50.75   Class :character   Class :character   1st Qu.:22.75
##  Median :100.50   Mode  :character   Mode  :character   Median :32.00
##  Mean   :100.50                                         Mean   :30.69
##  3rd Qu.:150.25                                         3rd Qu.:38.00
##  Max.   :200.00                                         Max.   :45.00
##
##      Hours           Income         Marital           Ethnicity
##  Min.   : 2.00   Min.   :  11.0   Length:200         Length:200
##  1st Qu.:20.00   1st Qu.: 355.2   Class :character   Class :character
##  Median :40.00   Median : 532.5   Mode  :character   Mode  :character
##  Mean   :33.71   Mean   : 586.9
##  3rd Qu.:45.00   3rd Qu.: 819.8
##  Max.   :70.00   Max.   :1789.0
##                  NA's   :10
```

```
md.pattern(msurf)
```




```
##     Personid Gender Qualification Age Hours Ethnicity Income Marital
## 181        1      1             1   1     1         1      1       1  0
## 9          1      1             1   1     1         1      1       0  1
## 9          1      1             1   1     1         1      0       1  1
## 1          1      1             1   1     1         1      0       0  2
##            0      0             0   0     0         0     10      10 20
```

An example dataset supplied with `mice` is `nhanes`, and is a tiny extract of 25 rows from the US National Health and Nutrition Examination Survey (NHANES). Note all variables are stored numeric, though the `age` and `hyp` column are actually categorical.

We'll create a copy of the dataset, call it `mnhanes` and recode the factor levels.

```
mnhanes <- mice::nhanes
mnhanes$age <- factor(mnhanes$age)
mnhanes$hyp <- factor(mnhanes$hyp)
levels(mnhanes$age) <- c("20-39","40-59","60+")
levels(mnhanes$hyp) <- c("No","Yes")
mnhanes <- data.frame(id=factor(1:nrow(mnhanes)),mnhanes)
```

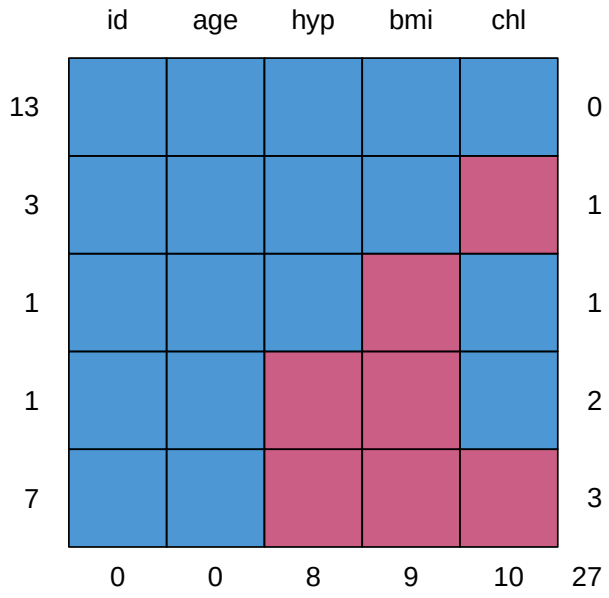
Here are the first 10 rows of the data, which include a lot of missing data for the variables `bmi` (body mass index, kg/m^2), `hyp` (presence of hypertension, 1=no, 2=yes) and `chl` (total serum cholesterol level, mg/dL).

Item non-response is indicated by an `NA` value. Notice that units 1, 4 and 10 provided no data at all apart from their ages.

Here's the missingness pattern in the NHANES dataset.

```
md.pattern(mnhanes)
```

id	age	bmi	hyp	chl
1	20-39	NA	NA	NA
2	40-59	22.7	No	187
3	20-39	NA	No	187
4	60+	NA	NA	NA
5	20-39	20.4	No	113
6	60+	NA	NA	184
7	20-39	22.5	No	118
8	20-39	30.1	No	187
9	40-59	22.0	No	238
10	40-59	NA	NA	NA




```
##    id age hyp bmi chl
## 13  1   1   1   1   1  0
## 3   1   1   1   1   0  1
## 1   1   1   1   0   1  1
## 1   1   1   0   0   1  2
## 7   1   1   0   0   0  3
##     0   0   8   9  10 27
```

9.1 Mean, median or mode imputation

The simplest method of imputing a variable is just to replace any missing value with a **typical** value from the units that did provide a response.

For numerical variables that can mean replacing missing values with the mean or the median. (We prefer the median of course if there are outliers.) For categorical variables the mode is the only `typical` value we have.

id	age	bmi	hyp	chl	bmi.imp
1	20-39	26.5625	NA	NA	TRUE
2	40-59	22.7000	No	187	FALSE
3	20-39	26.5625	No	187	TRUE
4	60+	26.5625	NA	NA	TRUE
5	20-39	20.4000	No	113	FALSE
6	60+	26.5625	NA	184	TRUE
7	20-39	22.5000	No	118	FALSE
8	20-39	30.1000	No	187	FALSE
9	40-59	22.0000	No	238	FALSE
10	40-59	26.5625	NA	NA	TRUE

id	age	bmi	hyp	chl	bmi.imp	hyp.imp	chl.imp
1	20-39	26.5625	No	187	TRUE	TRUE	TRUE
2	40-59	22.7000	No	187	FALSE	FALSE	FALSE
3	20-39	26.5625	No	187	TRUE	FALSE	FALSE
4	60+	26.5625	No	187	TRUE	TRUE	TRUE
5	20-39	20.4000	No	113	FALSE	FALSE	FALSE
6	60+	26.5625	No	184	TRUE	TRUE	FALSE
7	20-39	22.5000	No	118	FALSE	FALSE	FALSE
8	20-39	30.1000	No	187	FALSE	FALSE	FALSE
9	40-59	22.0000	No	238	FALSE	FALSE	FALSE
10	40-59	26.5625	No	187	TRUE	TRUE	TRUE

Let's replace the missing values of `bmi` in the `NHANES`mnhanes' dataset with the mean of the responding values:

```
mnhanes1 <- mnhanes
mnhanes1$bmi.imp <- is.na(mnhanes1$bmi)
mnhanes1$bmi[is.na(mnhanes1$bmi)] <- mean(mnhanes1$bmi, na.rm=TRUE)
```

The code above does two things: in a new copy of the data set `mnhanes1` it creates a new column `bmi.imp` which is `TRUE` if the `bmi` value is missing.
It then replaces `NA` values in the `bmi` column with the mean value 26.5625.

Here are the top 10 rows of the dataset after imputation.

The function `impute()` does the same thing: we just specify the variable name and the method (mean, median or mode).

```
mhnanes1 <- impute(mnhanes, varname="bmi", method="mean")
```

We can impute the categorical `hyp` variable using mode imputation and the numerical `chl` variable using median imputation

```
mnhanes1 <- mnhanes1 |>
  impute(varname="hyp", method="mode") |>
  impute(varname="chl", method="median")
```

This leads to the fully imputed dataset (first 10 rows only):

There's an obvious risk here: by filling in the most common or most typical value in the dataset we are **reducing variability** among the respondents and making the sample dominated by these typical values.

For example in Figure 9.1 the proportion of people with hypertension reduces from 4/17=24% to 4/25=16% because all of the missing individuals are assigned to the typical reponse of 'no hypertension'.

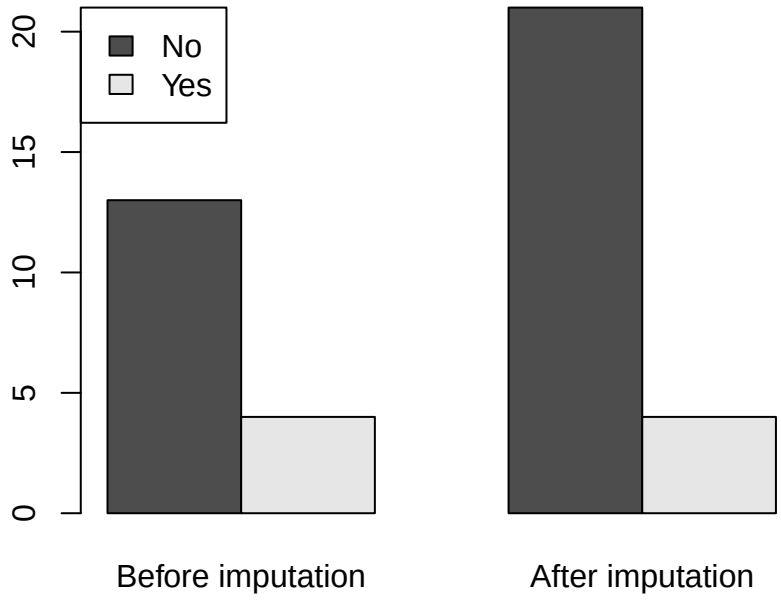


Figure 9.1: Distribution of the Hypertension variable before and after mean imputation

Likewise the distribution of BMI after mean imputation is far from realistic with a large artificial peak at the mean (Figure 9.2).

If we are only imputing a small proportion of values this shouldn't matter, but we'd prefer to have an imputation method that didn't introduce such artefacts into the data.

9.2 Hot deck imputation

An attractive alternative to replacing an observation with the mean, median or mode is the choose another respondent at random, and copy over their data. These respondents are referred to as 'donors', who 'donate' a copy of their data to the respondent with missing data.

The `impute()` function does this, by specifying the method "sample":

```
set.seed(222)
mnhanes2 <- mnhanes |>
  impute(varname="bmi", method="sample") |>
  impute(varname="hyp", method="sample") |>
  impute(varname="chl", method="sample")
```

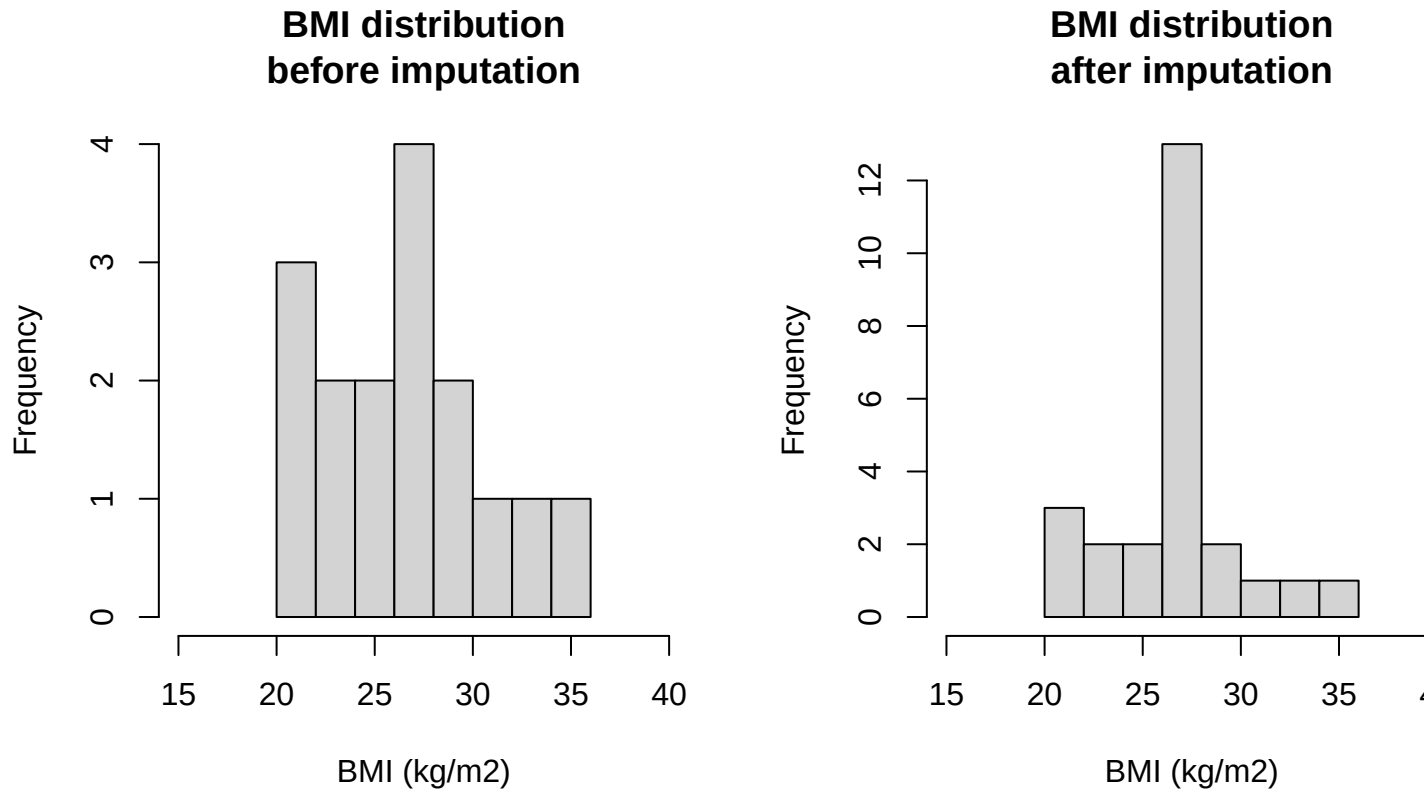


Figure 9.2: BMI distributions before and after mean imputation

The improved effect of doing so can be seen in Figures 9.3 (where tbe before and after proportions of people with hypertension are 4/17=24% and 5/20=20%) and 9.4.

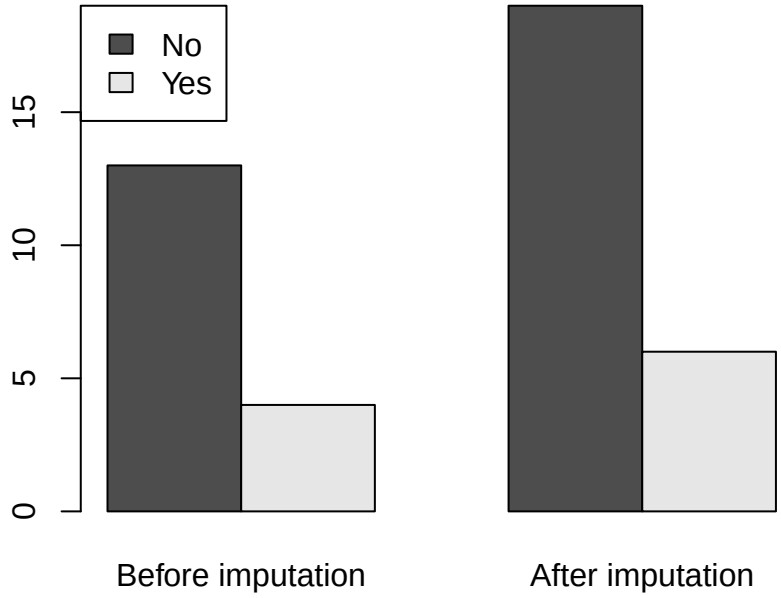


Figure 9.3: Distribution of the Hypertension variable before and after Hot Deck imputation

The Hot Deck method gets its name from the 'deck' (like a deck of cards) of respondents with complete information from which we draw at random. 'Hot' refers to the fact that we are using respondents from the same survey as the donors. 'Cold' deck imputation uses data from another external data source as a source of donors.

Important note - you'll get a different set of imputed values every time you do a hot deck imputation - so two analysts imputing data on the same data set will end up with different imputed values and therefore (slightly) different final estimates. If imputation is **minimal** then this shouldn't be a problem.

9.3 Cell methods

We can do better still. We know that characteristics such as BMI, hypertension and cholesterol correlate strongly with age, and we know the age of everyone.

It makes sense to do mean, median, mode **and** hot deck imputation within **imputation cells**. These cells are groups of respondents who have the same characteristics: specified by a (set of) categorical variables. We have to know which imputation cell the respondent

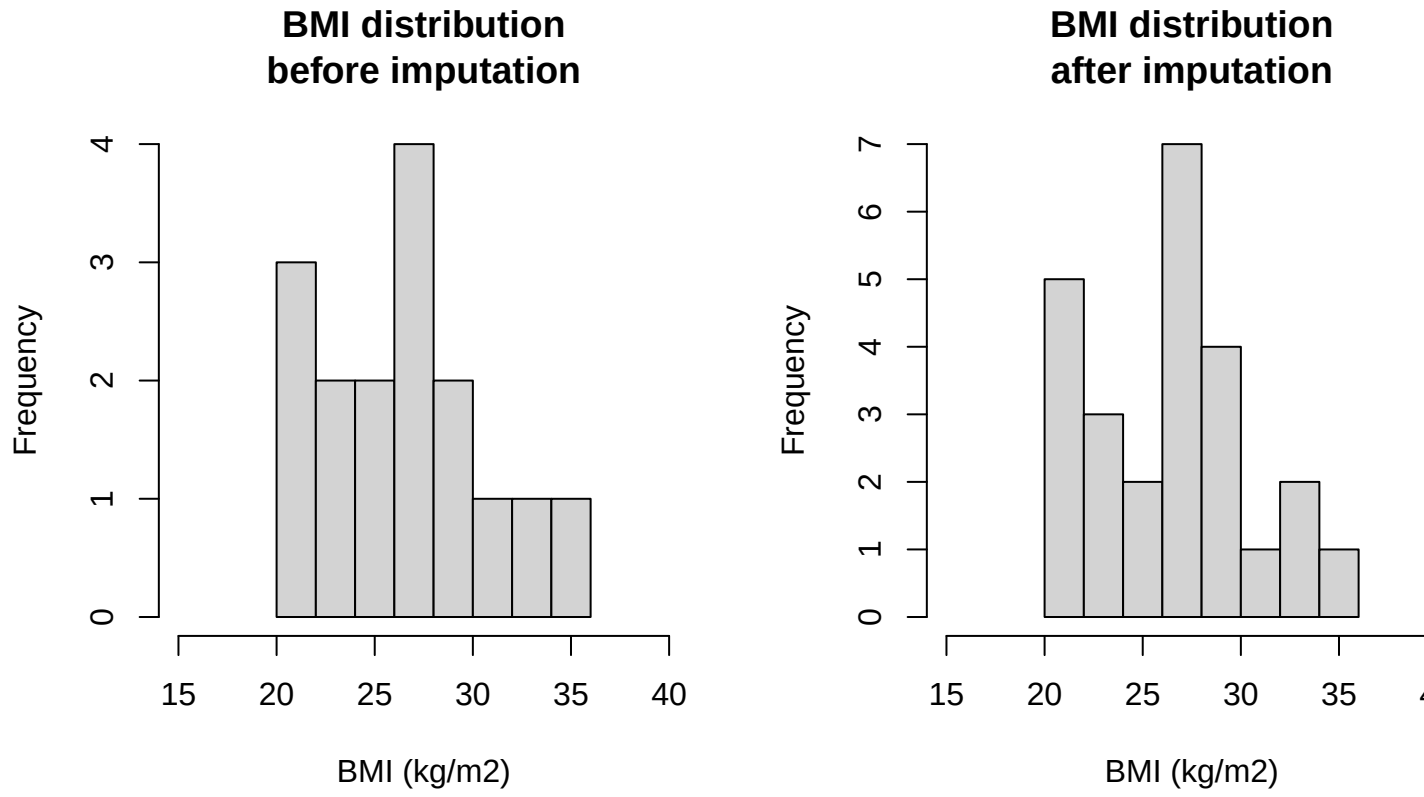


Figure 9.4: BMI distributions before and after Hot Deck imputation

with missing data belongs to, so we must have complete data for the variables defining the imputation cells.

Once we have imputation cells defined we can do mean, median, mode or hot deck imputation within the cells.

In the `impute()` function we define the cells with the right hand side of a model formula which should only include categorical variables. If it includes numerical variables then it is unlikely that there will be anyone in the same cell as the respondent who needs a donor. So with a variable like age we use age groups to define the cells, rather than year of age.

Let's do cell mean imputation for BMI, cell hot deck for hypertension, and cell median for cholesterol, all with cells defined by age group.

```
set.seed(333)
mnhanes3 <- mnhanes |>
  impute(varname="bmi", method="mean", formula=~age) |>
  impute(varname="hyp", method="sample", formula=~age) |>
  impute(varname="chl", method="median", formula=~age)
```

We can compare BMI values by age group in our original and three imputed datasets

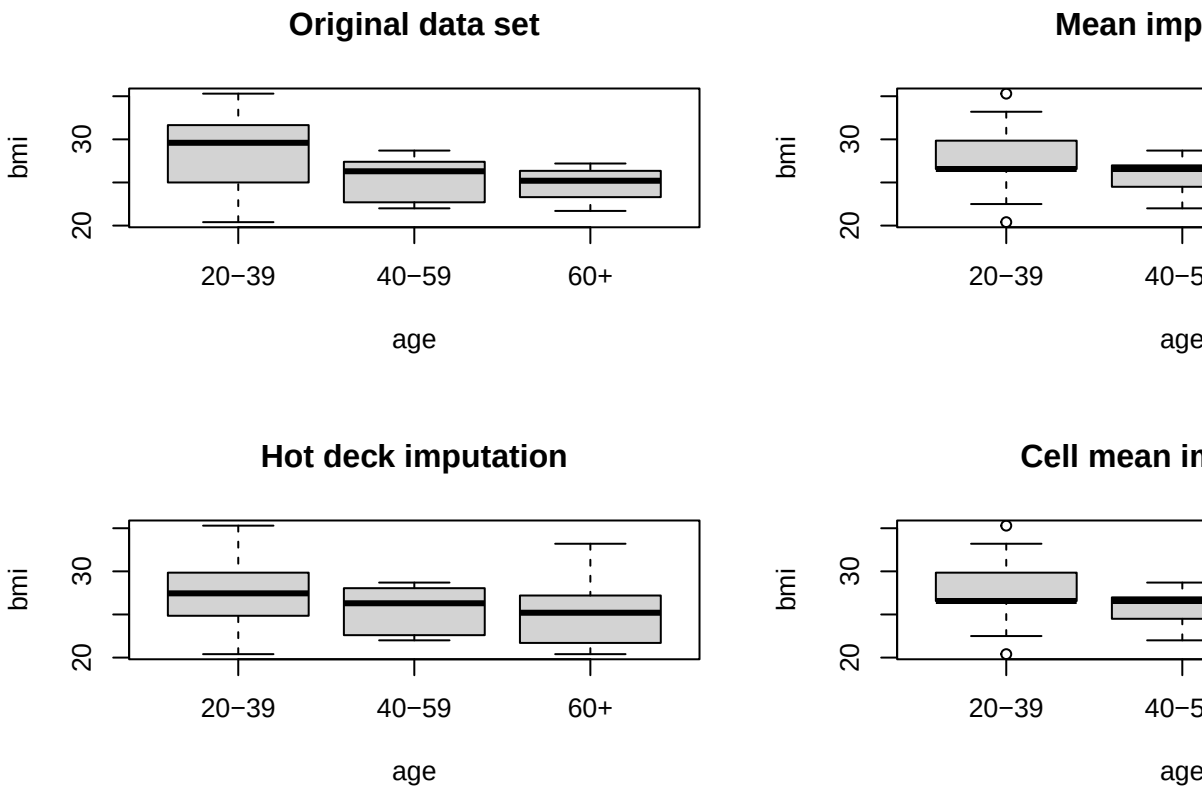


Figure 9.5: BMI distribution by age group after various imputation methods

9.4 Regression methods

A further extension of imputation methods leads us to **regression imputation**: using the complete data respondents we fit a regression model for our variable of interest, and then use it to predict the missing values. This works for numerical as well as categorical predictors, and includes cell mean imputation as a special case. As with the cell methods we need the respondents who need their values imputed to have complete data on the predictor variables.

Let's switch to imputing the 10 missing weekly incomes in the SURF dataset `msurf`.

We first note that income depends clearly both on Gender and weekly hours worked:

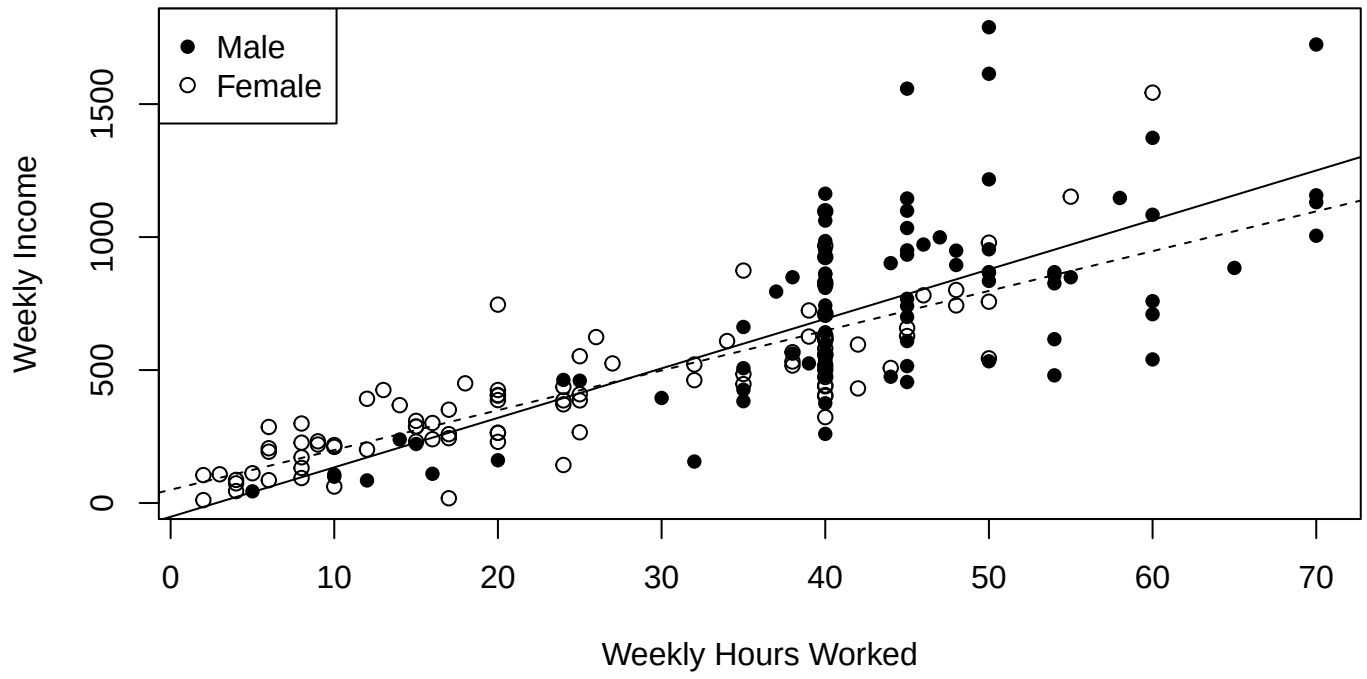


Figure 9.6: Weekly Income and Weekly Hours worked by Gender. The solid line shows the relationship for Males and the dashed line for Females

Let's impute the missing values by mean imputation and and regression imputation. Our regression model will be

$$\text{Income}_i = \beta_0 + \beta_1(\text{Hours}_i) + \beta_2 I(\text{Gender}_i{=}\text{Female}) + \varepsilon_i \qquad \text{where } \varepsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2)$$

```
msurf1 <- msurf |>
  impute(varname="Income", method="mean")
msurf2 <- msurf |>
  impute(varname="Income", method="lm.mean", formula=~Hours+Gender)
msurf3 <- msurf |>
  impute(varname="Income", method="lm.sim", formula=~Hours+Gender)
```

The results of the three imputation methods are shown in Figure 9.7.

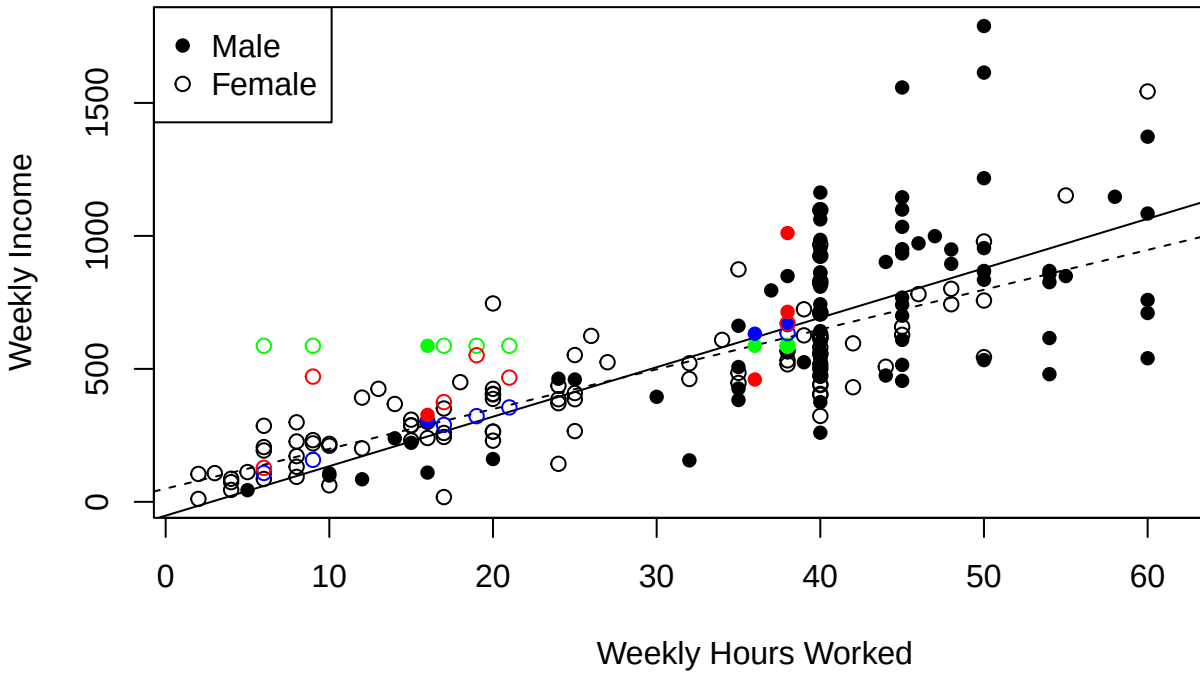


Figure 9.7: Weekly Income and Weekly Hours worked by Gender. The solid line shows the relationship for Males and the dashed line for Females. The green points are cell mean imputations, the blue points are regression mean imputations and the red points are regression simulation imputations.

The cell mean imputations (shown in green on the plot) are clearly very poor - everyone gets the same value, and in general these values are much to high since our missing values correspond to workers with in general few weekly hours.

The first regression imputation has method `lm.mean` fits the model in equation (9.4), and then predicts directly onto the fitted regression line:

$$\text{ImputedIncome}_i = \hat{\beta}_0 + \hat{\beta}_1(\text{Hours}_i) + \hat{\beta}_2 I(\text{Gender}_i\text{=Female})$$

using the fitted regression parameters $(\hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2)$. These imputed values are shown in blue, and are exactly where we'd expect a person of the given Gender and Hours worked to be earning.

This is very good, but in the spirit of hot deck imputation we can do even better: we know that observations should scatter around the the regression line, rather than lying directly on it. So the second regression method draws from the error distribution

$$\hat{\varepsilon}_i \sim N(0, \hat{\sigma}^2)$$

using the fitted standard deviation ($\hat{\sigma}$). So the imputed value is the expected value plus this additional error, which bumps the prediction off the regression line:

$$\text{ImputedIncome}_i = \hat{\beta}_0 + \hat{\beta}_1(\text{Hours}_i) + \hat{\beta}_2 I(\text{Gender}_i\text{=Female}) + \hat{\varepsilon}_i$$

These imputed values are shown in red in Figure 9.7, and they look just like real data. To get these imputed values we change the method from "lm.mean" to "lm.sim" to change from the mean (expected) income, to a simulated value.

9.5 Multiple imputation

Instead of imputing a **single** value for each missing observation, we impute several possible imputed values - we end up with multiple completed data sets, in something of the same spirit as the replicate weights we get with the Jackknife.. The variability among estimates from these separate completed data sets gives us additional insight into the effect of our imputations, and the uncertainty we have associated with having missing data.

The confidence we can have regarding the true scale of the uncertainty depends on how strongly we believe in the model we've used to impute.

The `mice` package implements multiple imputation, that's its major function.
It's more detail then we need for this course, but you should be aware that you'll see multiple imputation methods from time to time.

Chapter 10

Designing Surveys

The design of surveys is a massive topic, and we can't cover much here. The key principle is that everything that is done in the design of a survey should be done to meet the Objectives (the research goals) of the survey. Nothing else matters.

For our purposes calculation of the sample size required when designing a survey is the most significant single decision to make since it determines the standard errors for the survey estimates. In real life the decision about sample size is often severely constrained by available resources.

10.1 Another interpretation of the Deff

Assume that for sample size n the variance of an estimator in a specific design has the approximate formula

$$\mathbf{Var}[\widehat{T}_{\text{design}}] = \frac{c_{\text{design}}}{n}$$

then using the definition of the Deff, the ratio of the variance with respect to SRSWOR, then

$$\begin{aligned}
\mathbf{Deff}[\widehat{T}_{\text{design}}] &= \frac{\mathbf{Var}[\widehat{T}_{\text{design}}]}{\mathbf{Var}[\widehat{T}_{\text{SRSWOR}}]} && (10.1)\\
&= \frac{c_{\text{design}}/n}{c_{\text{SRSWOR}}/n} && (10.2)\\
&= \frac{c_{\text{design}}}{c_{\text{SRSWOR}}} && (10.3)
\end{aligned}$$

So if we want to find the sample sizes at which the variances in the design of interest and SRSWOR are equal, then we set

$$\begin{aligned}
V_{\text{design}} &= V_{\text{SRSWOR}} && (10.4)\\
\frac{c_{\text{design}}}{n_{\text{design}}} &= \frac{c_{\text{SRSWOR}}}{n_{\text{SRSWOR}}} && (10.5)\\
n_{\text{design}} &= \frac{c_{\text{design}}}{c_{\text{SRSWOR}}} n_{\text{SRSWOR}} && (10.6)\\
&= \mathbf{Deff}[\widehat{T}_{\text{design}}] n_{\text{SRSWOR}} && (10.7)
\end{aligned}$$

This means that a rough way to compute the sample size required in a complex design is to first calculate the sample size we'd need in SRSWOR, and then scale it by the Deff for the complex design.

If the Deff is 2 (as it may be in a cluster design for example), then we need twice the sample size an SRSWOR would need in order to achieve the same standard error.

10.2 Computing the Required Sample Size

1. Identify your **design estimate** – this is the estimate that will underlie a headline you could imagine being written about your survey.
2. Identify the desired MOE, m, for this design estimate, noting whether you want this MOE to apply to the overall estimate, or for that estimate in each of, say, H subgroups/strata.
3. Compute the sample size required, n_1, assuming that you're going to select a Simple Random Sample and ignoring the fpc.

   For estimation of a mean $\bar{Y}$ of some characteristic y:

   $$n_1 = \left(\frac{Z}{MOE}\right)^2 s^2$$

   where Z is appropriate for the desired level of confidence and s is the estimated population standard deviation of the characteristic y.

   For estimation of a proportion P:

   $$n_1 = \left(\frac{Z}{MOE}\right)^2 p(1-p)$$

   where p is the estimated proportion, or 0.5 if really unknown.
4. If the sample size is a substantial proportion of the population or subgroup size N, then apply the fpc adjustment:

   $$n_2 = \frac{n_1}{1 + \frac{n_1}{N}}$$

   This adjusts for the fact that in a finite population we don't need quite as large a sample to achieve the same standard error. However if the sampling fraction (n/N: the proportion of the population that is selected) is small, then the fpc adjustment has very little effect.
5. If you have H subgroups and are using equal allocation then multiply by H

   $$n_3 = Hn_2$$

   or alternatively sum over the n_2 values in each of the strata you've defined.
6. Apply the design effect (Deff) appropriate for the actual design you'll be using.

   $$n_4 = \text{Deff} \times n_3$$

7. Adjust for anticipated non-response: if response rate ϕ is expected then

   $$n_5 = \frac{n_4}{\phi}$$

   (If non-response is likely to vary between strata, then apply this correction to n_2, within each stratum.)

References

Bibliography

Cochran, W. G. (1977). *Sampling Techniques*. Wiley, New York, third edition.

Kish, L. (1965). *Survey Sampling*. Wiley, New York.

Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*. Wiley, Hoboken, second edition.

Lohr, S. L. (1999). *Sampling: Design and Analysis*. Duxbury Press, Pacific Grove, CA, USA.

Lumley, T. (2004). Analysis of complex survey samples. *Journal of Statistical Software*, 9(1):1–19. R package verson 2.2.

Lumley, T. S. (2010). *Complex Surveys: A Guide to Analysis Using R (Wiley Series in Survey Methodology)*. Wiley.

Särndal, C.-E., Swensson, B., and Wretmann, J. (1992). *Model Assisted Survey Sampling*. Springer-Verlag, New York.

Scheaffer, R. L., III, W. M., and Ott, R. L. (2006). *Elementary Survey Sampling*. Duxbury, Belmont, sixth edition.

Zealand, S. N. (1995). *A Guide to Good Survey Design*. Statistics New Zealand, Wellington, second edition.